

NATURAL LANGUAGE PROCESSING

RÉSEAUX DE NEURONES RÉCURRENTS

Sarah Malaeb
2025-26

RÉFÉRENCES PRINCIPALES DE CE COURS

Deep Learning A-Z, Kirill Eremenko, Hadelin de Ponteves, @SuperDataScience

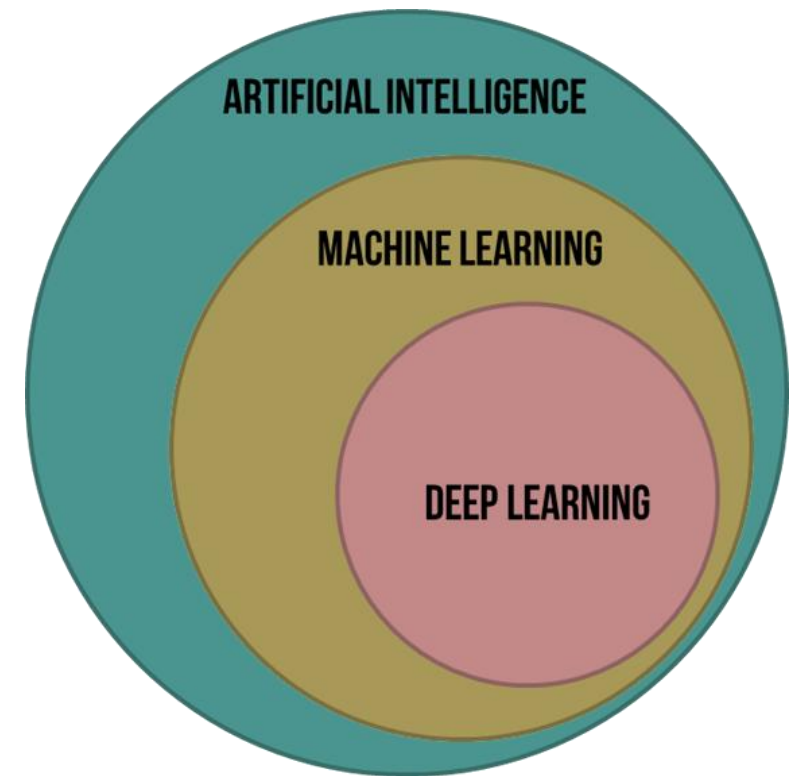
StatQuest with Josh Starmer, www.youtube.com/watch?v=AsNTP8Kwu80

Simply Learn, www.simplilearn.com

**RÉCAP :
RÉSEAU DE NEURONES ARTIFICIELS
ANN (ARTIFICIAL NEURAL NETWORK)**

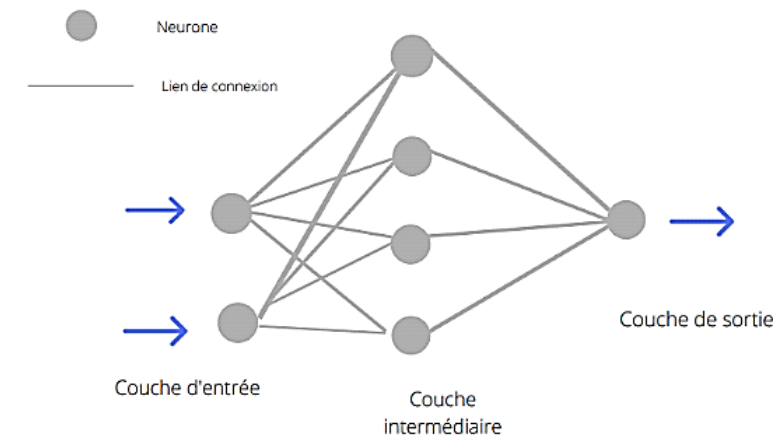
IA VS. ML VS. DL

- **IA** : est l'ingénierie de la création de machines et de programmes intelligents.
- **ML** : est la capacité à apprendre automatiquement sans être explicitement programmé.
- **Deep Learning** : est une branche plus évoluée de l'apprentissage automatique : l'apprentissage basé sur des réseaux neuronaux profonds imitant le cerveau humain dans la prise de décision.



IMITATION DU CERVEAU HUMAIN

- Avec le Deep Learning, les ordinateurs essaient de copier le cerveau qui est très puissant pour apprendre, adapter des compétences et les appliquer.
- Il imite et recrée le cerveau humain avec une structure artificielle appelée le réseau neuronal artificiel, où l'on trouve :
 - des nœuds (ou neurones) pour les valeurs d'entrée,
 - des sorties qui représentent les données prédictives,
 - et entre les deux, il y a des couches cachées (des milliards de neurones qui relient les neurones d'entrée aux neurones de sortie).



RÉSEAUX DE NEURONES

- Un réseau de neurones artificiels est un modèle informatique inspiré du fonctionnement du cerveau humain, composé de multiples unités de traitement (neurones) interconnectées.
- Ces neurones transmettent des informations à travers des connexions pondérées, permettant au réseau de réaliser des tâches d'apprentissage automatique telles que la reconnaissance de motifs, la classification et la résolution de problèmes complexes.

DIFFÉRENTS TYPES DE RÉSEAUX DE NEURONES

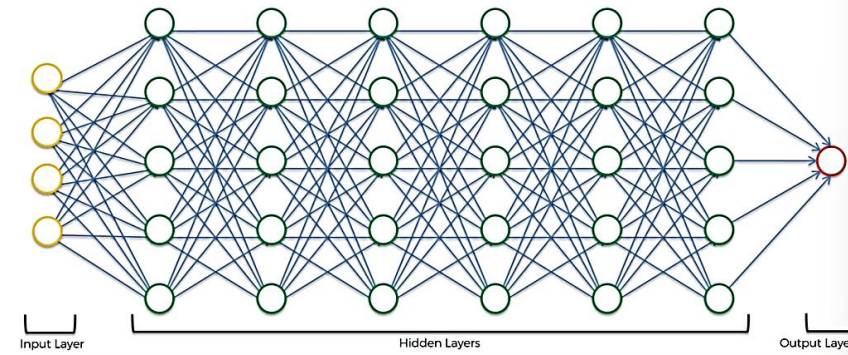
- Il existe plusieurs types de réseaux de neurones artificiels, chacun adapté à des tâches et des architectures spécifiques :
 - Réseaux de Neurones à Alimentation Avant (Feed-forward Neural Networks) :
 - Ces réseaux sont les plus simples, avec une propagation de l'information dans une seule direction, de l'entrée à la sortie. Ils sont couramment utilisés pour la classification et la régression.
 - Réseaux de Neurones Convolutionnels (Convolutional Neural Networks CNN) :
 - Spécialisés dans le traitement d'images, les CNN utilisent des couches de convolution pour extraire des caractéristiques visuelles hiérarchiques, ce qui les rend efficaces pour la classification d'images et la détection d'objets.
 - Réseaux de Neurones Récurrents (Recurrent Neural Networks RNN) :
 - Ils sont conçus pour traiter des données séquentielles, comme des séquences temporelles, du texte ou de la parole, en utilisant des boucles de rétroaction. Les RNN sont adaptés aux tâches de traitement du langage naturel, de traduction automatique, et de prédiction de séquences.

DIFFÉRENTS TYPES DE RÉSEAUX DE NEURONES

- Réseaux de Neurones Génératifs (Generative Neural Networks) :
 - Ces réseaux, tels que les GAN (Generative Adversarial Networks) et les VAE (Variational Autoencoders), sont utilisés pour générer de nouvelles données réalistes, comme des images ou du texte.
- Réseaux de Neurones Autoencodeurs (Autoencoder Neural Networks) :
 - Utilisés pour la réduction de dimension et la reconstruction de données, les autoencodeurs sont utiles dans la compression d'informations et la détection d'anomalies.
- etc.

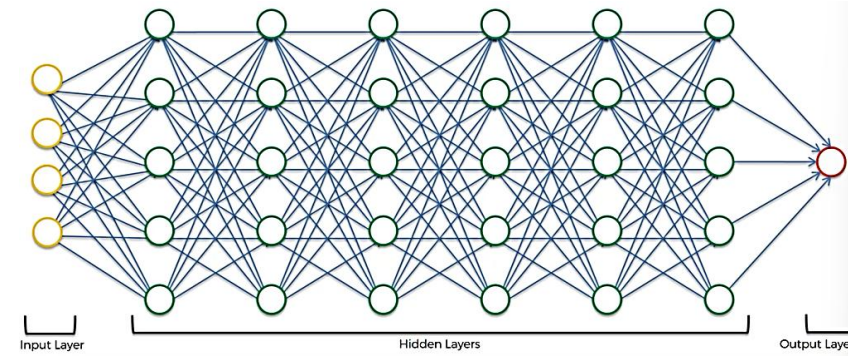
RÉSEAUX DE NEURONES ARTIFICIELS

ANN Artificial Neural Network



- Un réseau de neurones artificiels est un modèle informatique qui s'inspire du fonctionnement des neurones biologiques du cerveau humain.
- Il est composé de multiples unités de traitement appelées neurones, interconnectées par des connexions pondérées.
- Ces neurones, organisés en couches, transmettent et traitent l'information en utilisant des calculs mathématiques complexes.
- Les réseaux de neurones artificiels sont utilisés dans le domaine de l'apprentissage automatique pour résoudre une variété de tâches telles que la classification, la régression, la reconnaissance de motifs, et bien d'autres, en adaptant les poids des connexions lors de l'entraînement pour apprendre à partir des données.

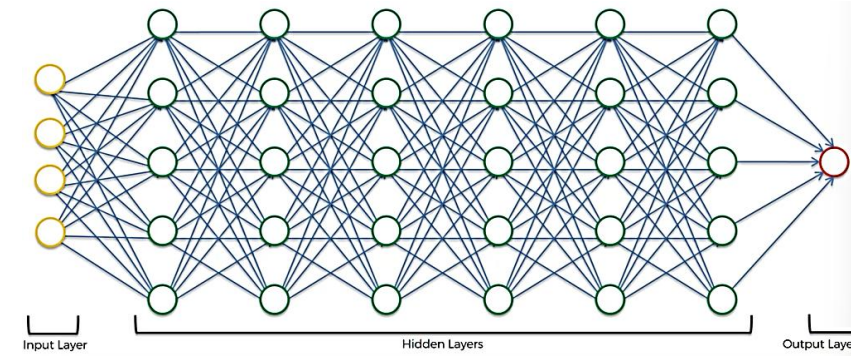
RÉSEAUX DE NEURONES ARTIFICIELS



Réseau de Neurones Artificiel Feed-Forward

- Le réseau de neurones le plus simple est le **Réseau de Neurones Artificiel Feed-Forward**.
- Le **Réseau de Neurones Artificiel Feed-Forward**, également connu sous le nom de **Perceptron Multicouche (MLP)**.
- Un **réseau Feed-Forward** possède des connexions dans une seule direction ; il n'existe donc **aucune boucle**.
- Un réseau feed-forward représente une **fonction de son entrée actuelle** ; ainsi, il **ne possède aucun état interne** autre que les **poids eux-mêmes**.
- Les réseaux feed-forward sont généralement **organisés en couches**, de sorte que chaque unité reçoit des entrées uniquement des unités de la couche immédiatement précédente. Dans un réseau multicouche, une ou plusieurs **couches de neurones cachées** qui ne sont pas directement connectées aux sorties du réseau peuvent exister.

RÉSEAUX DE NEURONES ARTIFICIELS

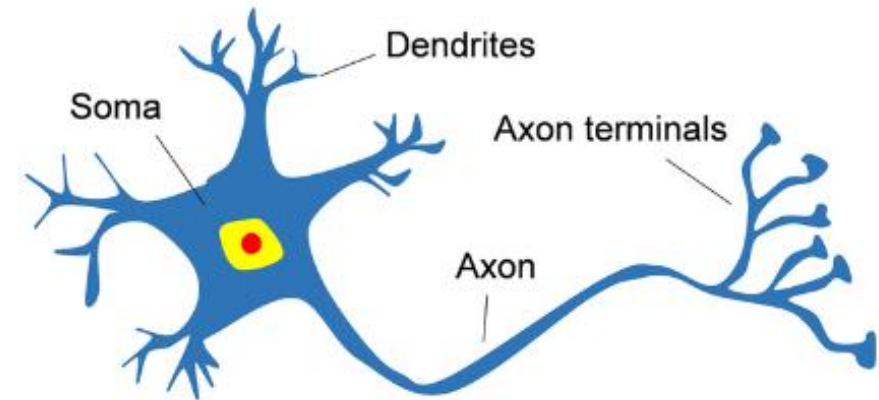


Rétropropagation (Backpropagation)

- La rétropropagation (Backpropagation) est le cœur de tout réseau de neurones.
- La rétropropagation sert à calculer les gradients de manière efficace.
- Les optimiseurs sont utilisés pour entraîner le réseau de neurones, en utilisant les gradients calculés grâce à la rétropropagation.
- L'algorithme de rétropropagation est apparu dans les années 1960.

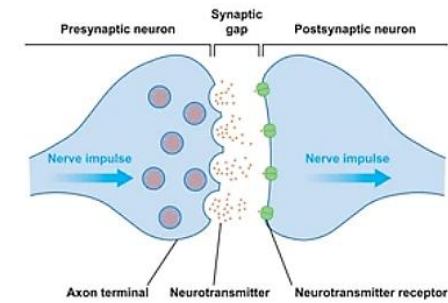
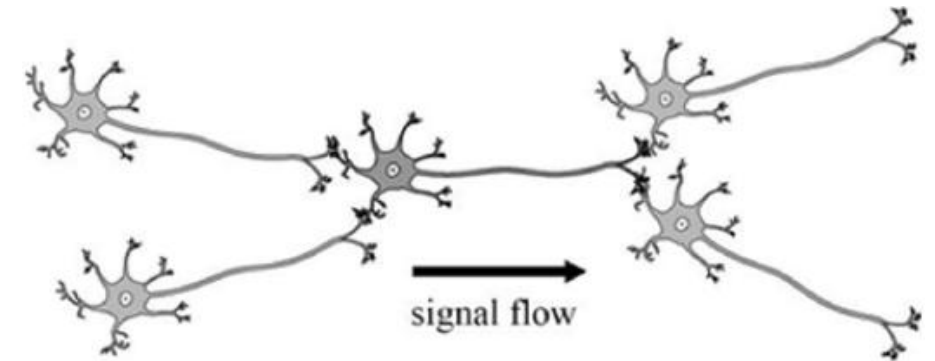
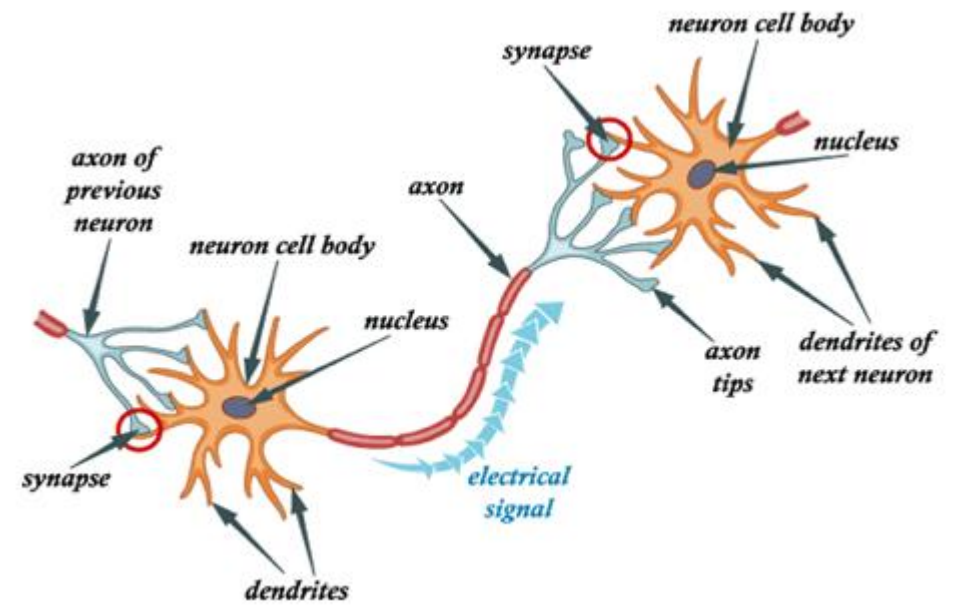
NEURONE BIOLOGIQUE

- Le cerveau humain est le système d'apprentissage le plus puissant de la planète.
C'est pourquoi la création de réseaux neuronaux est très intéressante.
- Pour créer un réseau neuronal, il faut d'abord créer des neurones.
- Les dendrites sont les récepteurs du signal.
- L'axone est le transmetteur du signal.

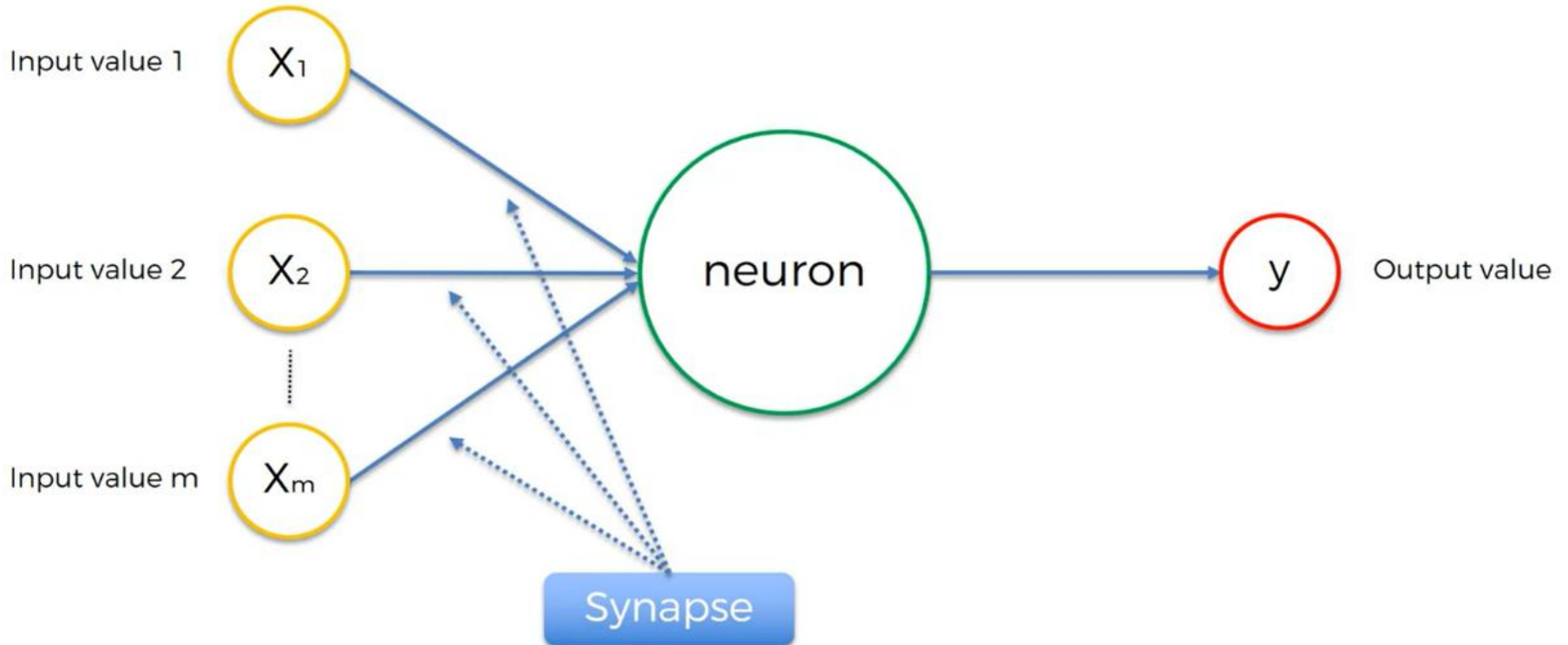


NEURONE BIOLOGIQUE

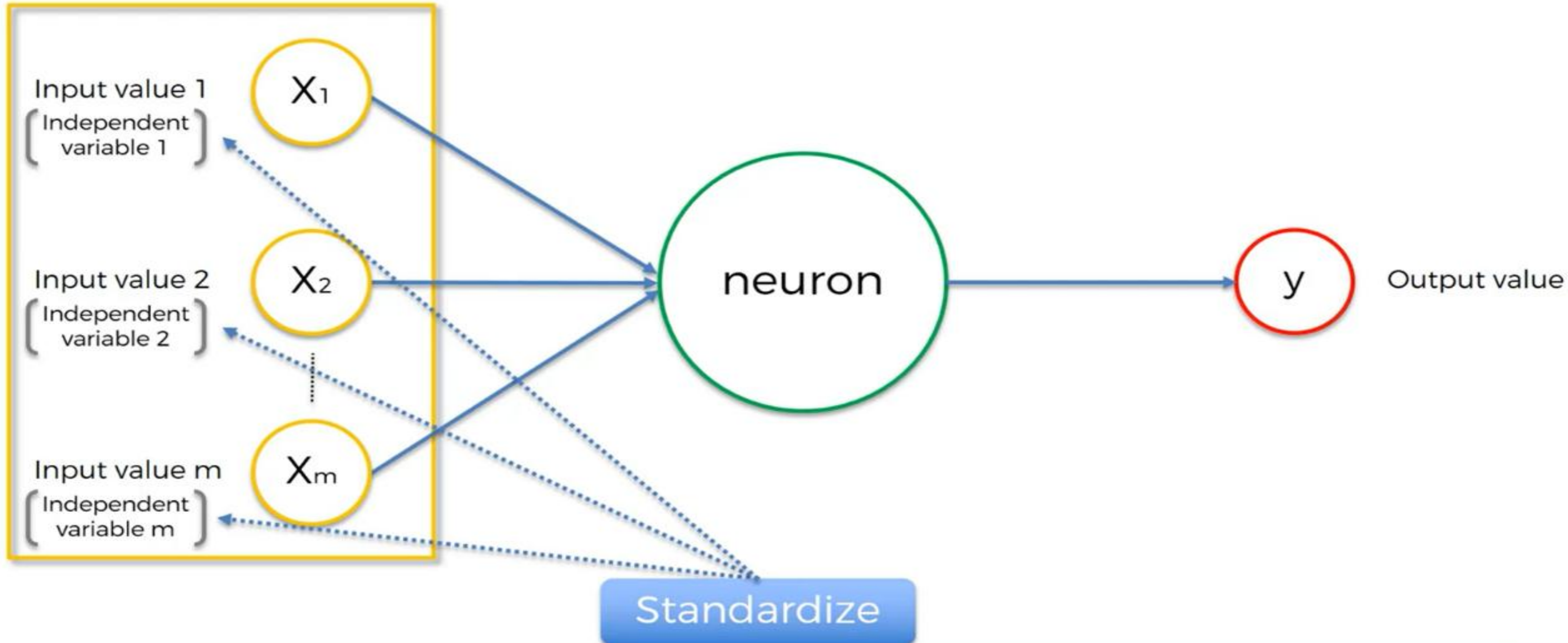
- Les **dendrites** d'un neurone sont **connectées aux axones des autres neurones**.
- Le **signal** d'un neurone voyage à travers l'**axone** et passe **sur les dendrites** du neurone suivant.
- Il est important de noter **qu'il n'existe aucune connexion physique entre la dendrite et l'axone** (ils ne se touchent pas réellement).
→ La connexion entre eux est le signal, qui correspond à la **synapse**.



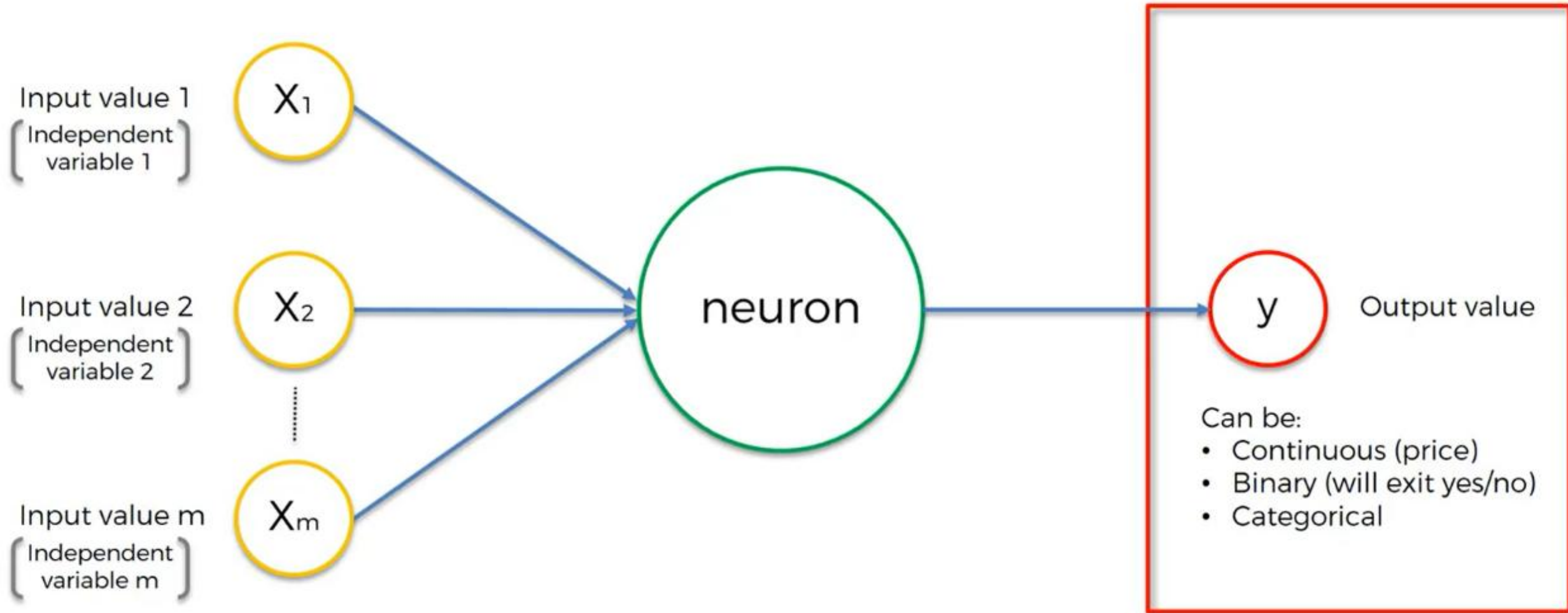
NEURONE DU RÉSEAU DE NEURONES



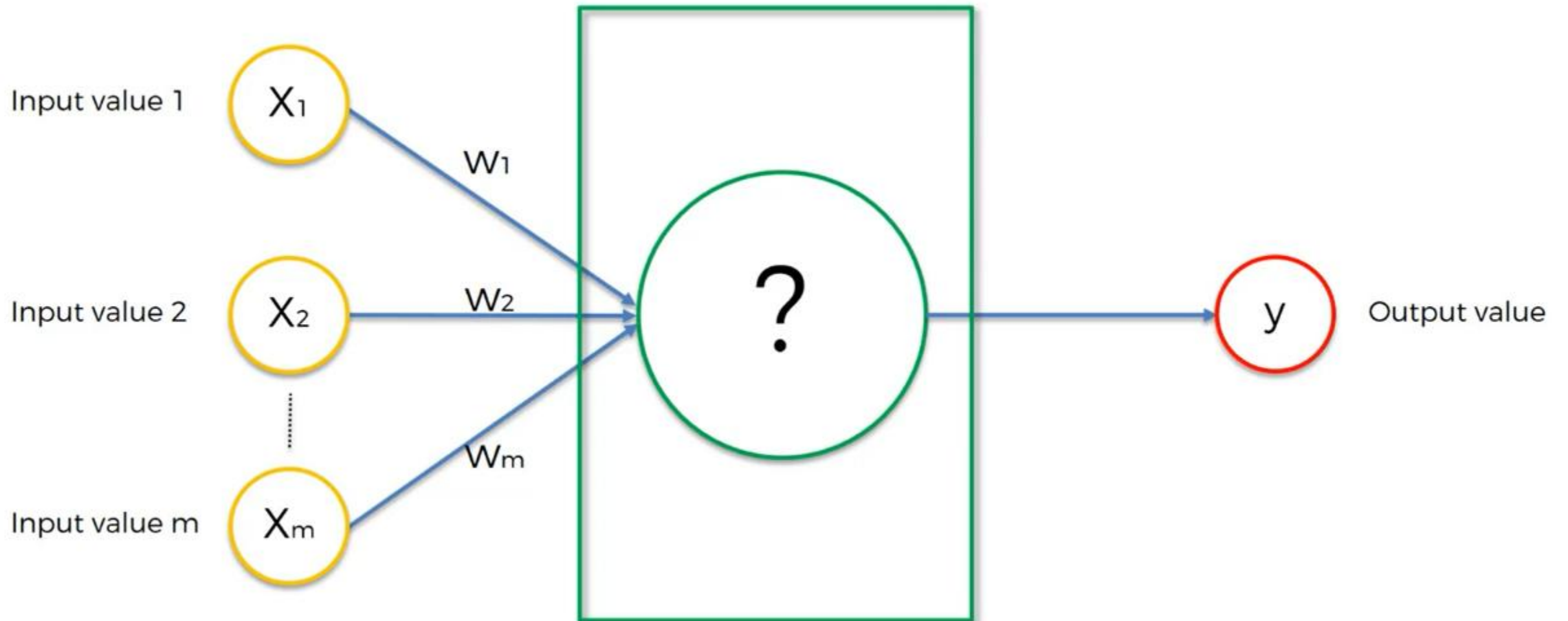
NEURONE DU RÉSEAU DE NEURONES



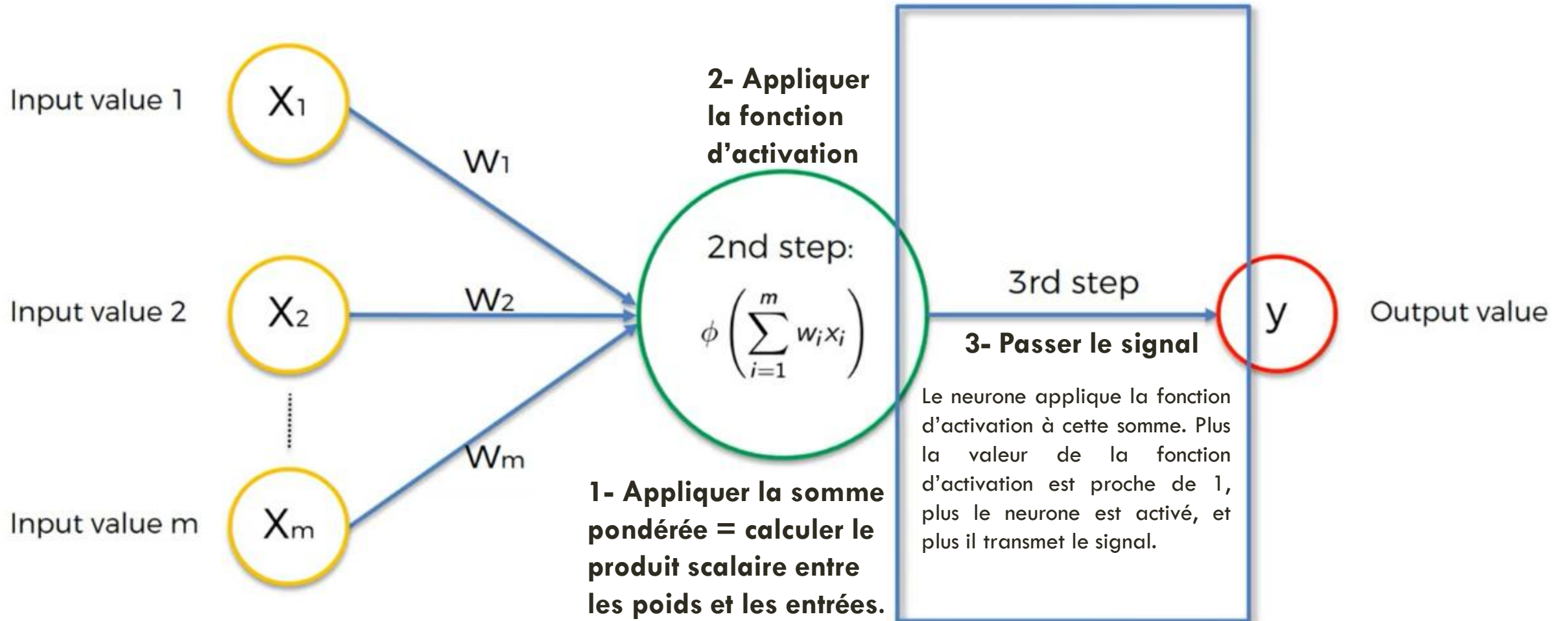
NEURONE DU RÉSEAU DE NEURONES



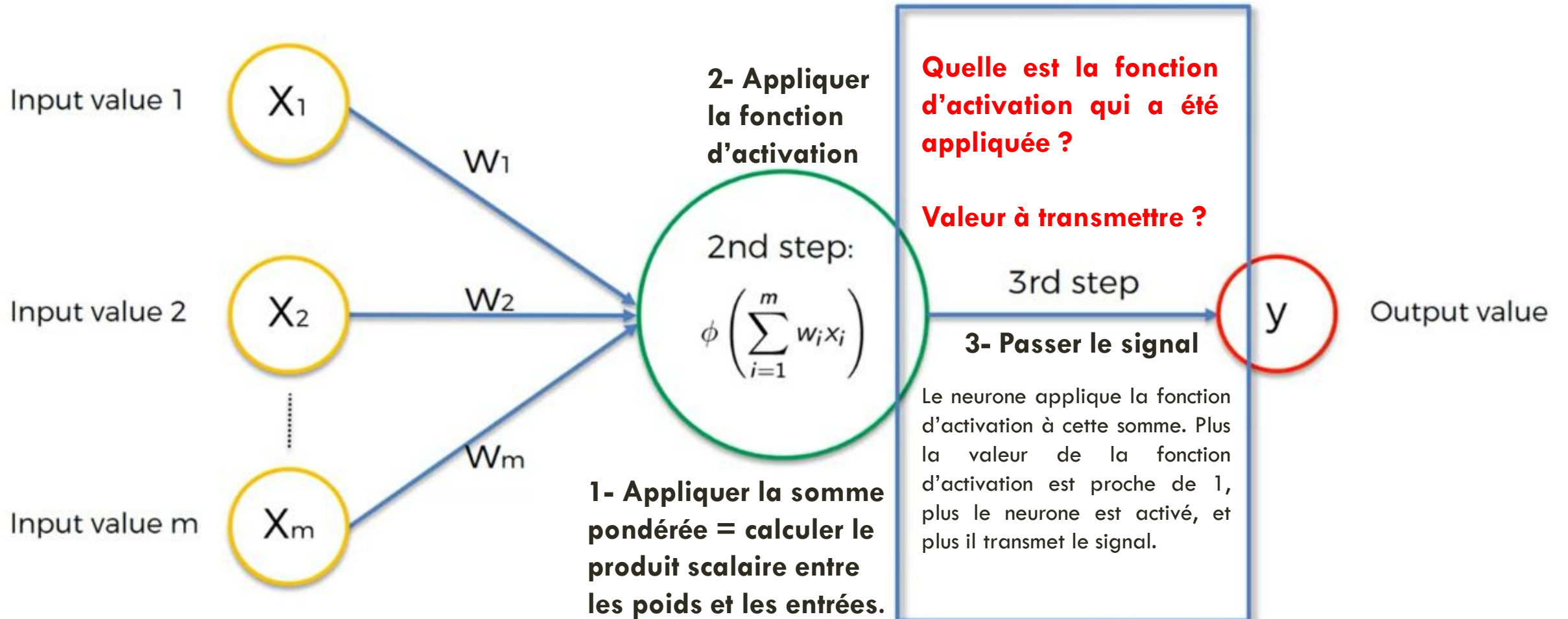
NEURONE DU RÉSEAU DE NEURONES



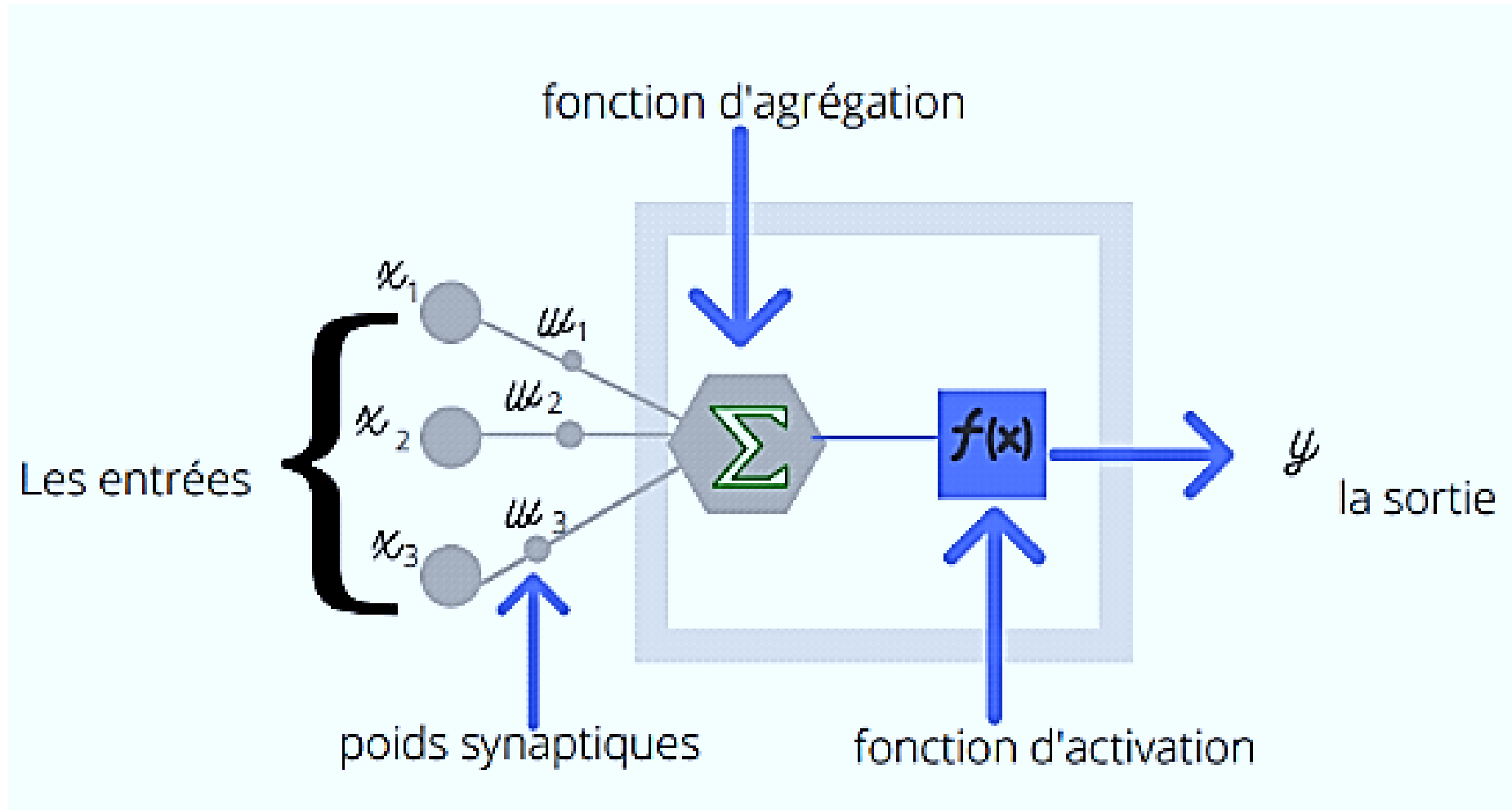
FONCTION D'ACTIVATION



FONCTION D'ACTIVATION



ARCHITECTURE DU RÉSEAU DE NEURONE ARTIFICIEL



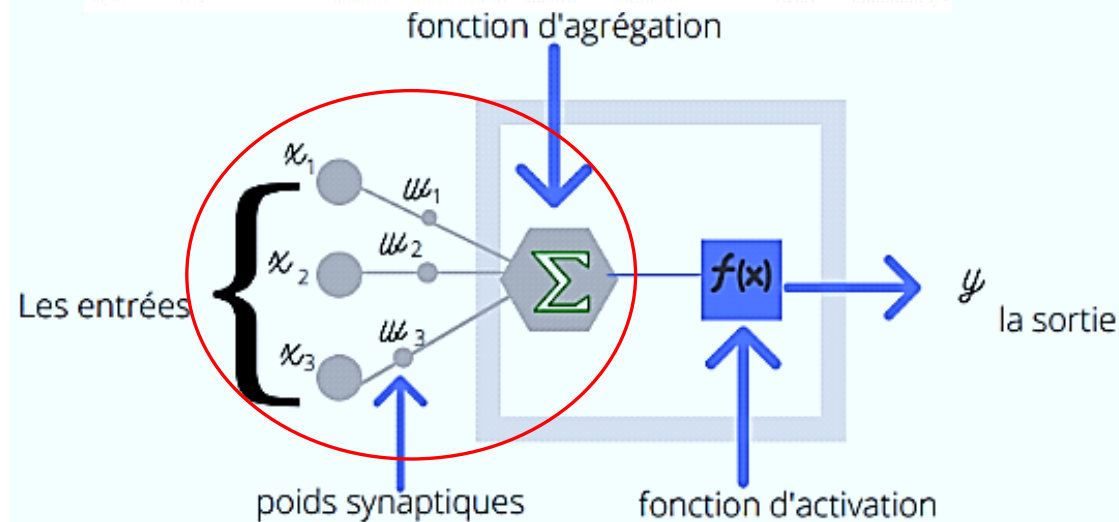
LA FONCTION D'AGRÉGATION

- C'est la somme pondérée des poids w_i multiplié par la valeur de l'entrée x_i .

Fonction de combinaison

$$\sum = x_1.w_1 + x_2.w_2 + \dots + x_n.w_n$$

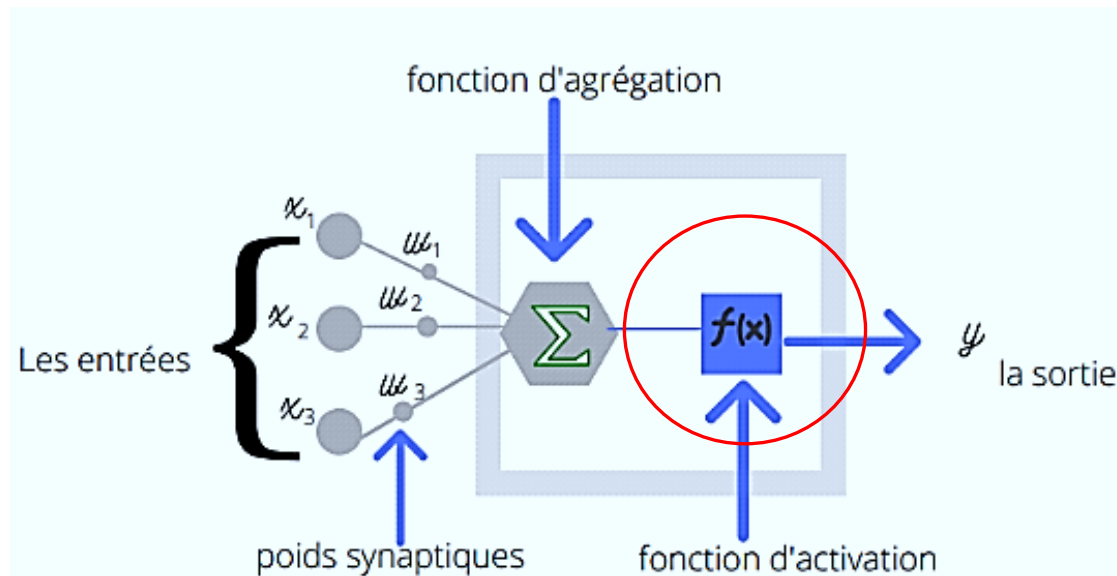
Combine le résultats des signaux d'entrée
 x_i avec les poids du neurone w_i
(entièrement linéaire)



RÉSEAUX DE NEURONES ARTIFICIELS

La fonction d'activation

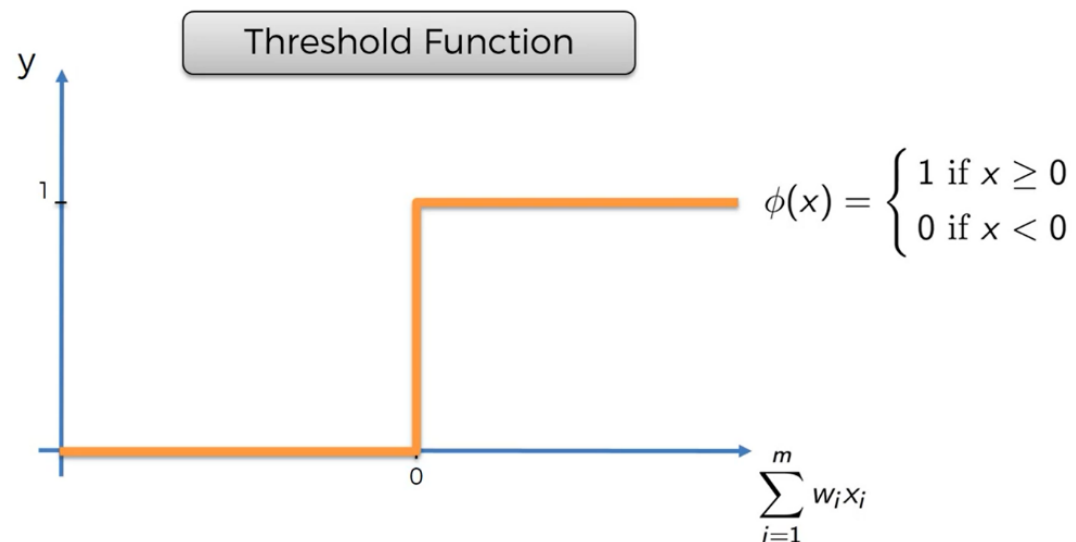
- La fonction d'activation peut être envisagée comme l'équivalent du "seuil d'activation" présent dans les neurones biologiques.
- Elle joue un rôle crucial en déterminant si un neurone artificiel doit être activé, et si c'est le cas, à quel niveau cette activation doit avoir lieu.
- Il existe plusieurs fonctions d'activation couramment utilisées dans les réseaux de neurones.



RÉSEAUX DE NEURONES ARTIFICIELS

La fonction d'activation

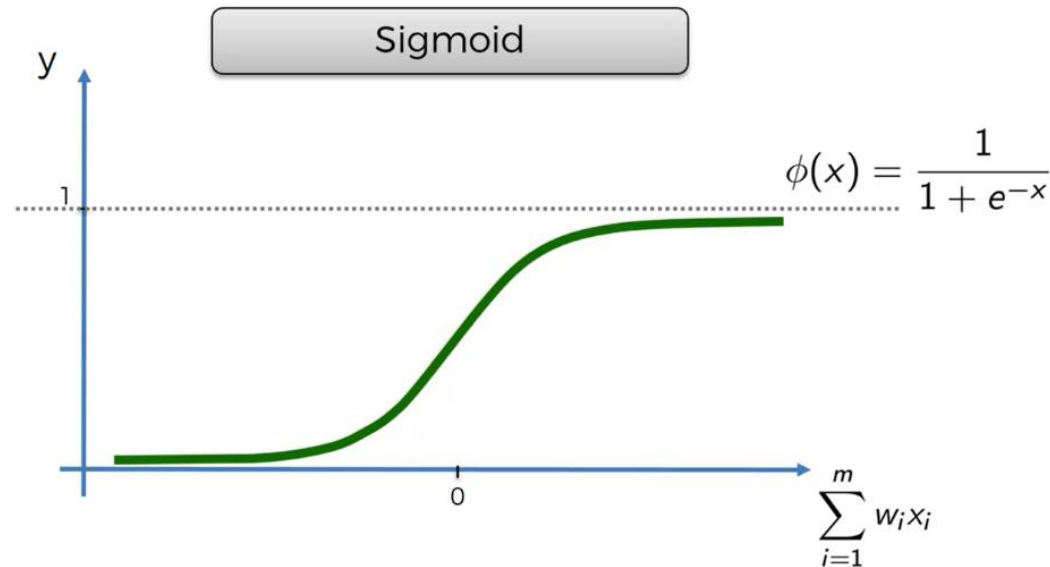
- La fonction Threshold (seuil):
 - Le seuil, dans ce contexte, représente un seuil numérique au-dessus duquel un neurone artificiel est activé, c'est-à-dire qu'il produit une sortie non nulle, tandis qu'en dessous de ce seuil, le neurone reste inactif avec une sortie nulle.



RÉSEAUX DE NEURONES ARTIFICIELS

La fonction d'activation

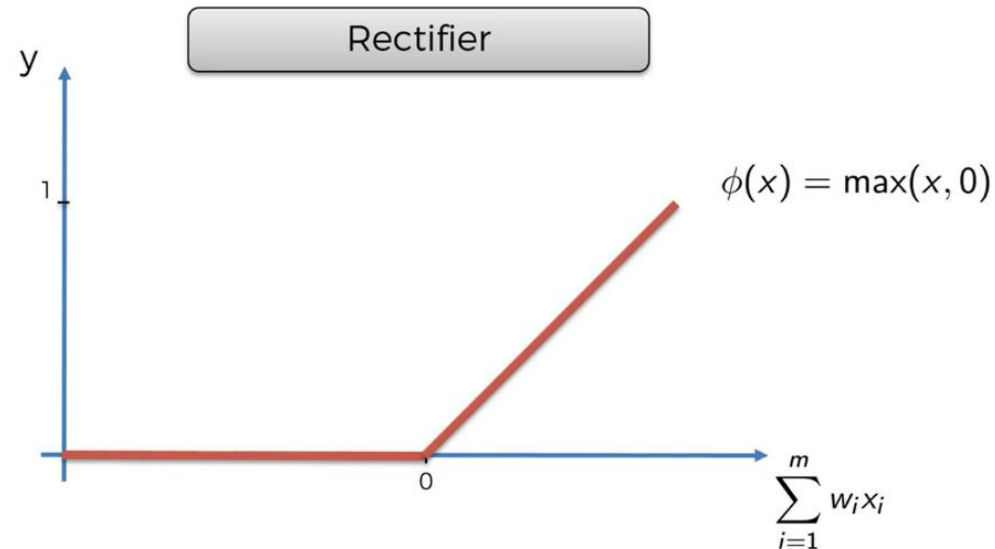
- La fonction Sigmoid :
 - Cette fonction transforme la somme pondérée des entrées en une valeur comprise entre 0 et 1.



RÉSEAUX DE NEURONES ARTIFICIELS

La fonction d'activation

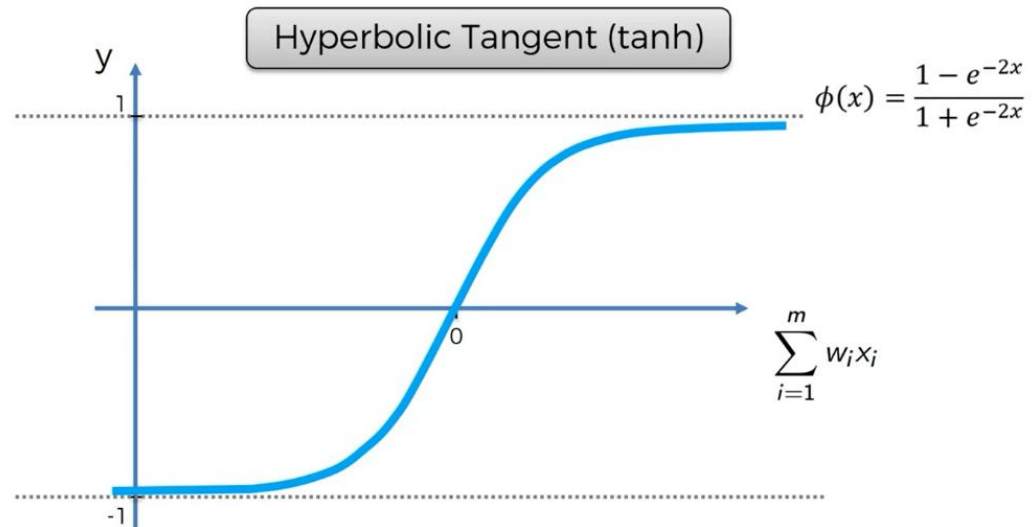
- Fonction Rectifier ou ReLU (Rectified Linear Unit) :
 - Elle transforme les valeurs d'entrée de manière linéaire si elles sont positives, et les réduit à zéro si elles sont négatives. Cela permet d'accélérer l'apprentissage et de mieux gérer les problèmes de disparition du gradient.



RÉSEAUX DE NEURONES ARTIFICIELS

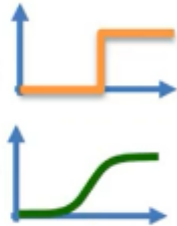
La fonction d'activation

- Fonction Tanh (Tangente Hyperbolique) :
 - Elle ressemble à la sigmoïde mais comprime les valeurs entre -1 et 1.



EXEMPLE — QUELLE FONCTION D'ACTIVATION UTILISER?

Assuming the dependant variable is binary ($Y=0$ or 1)

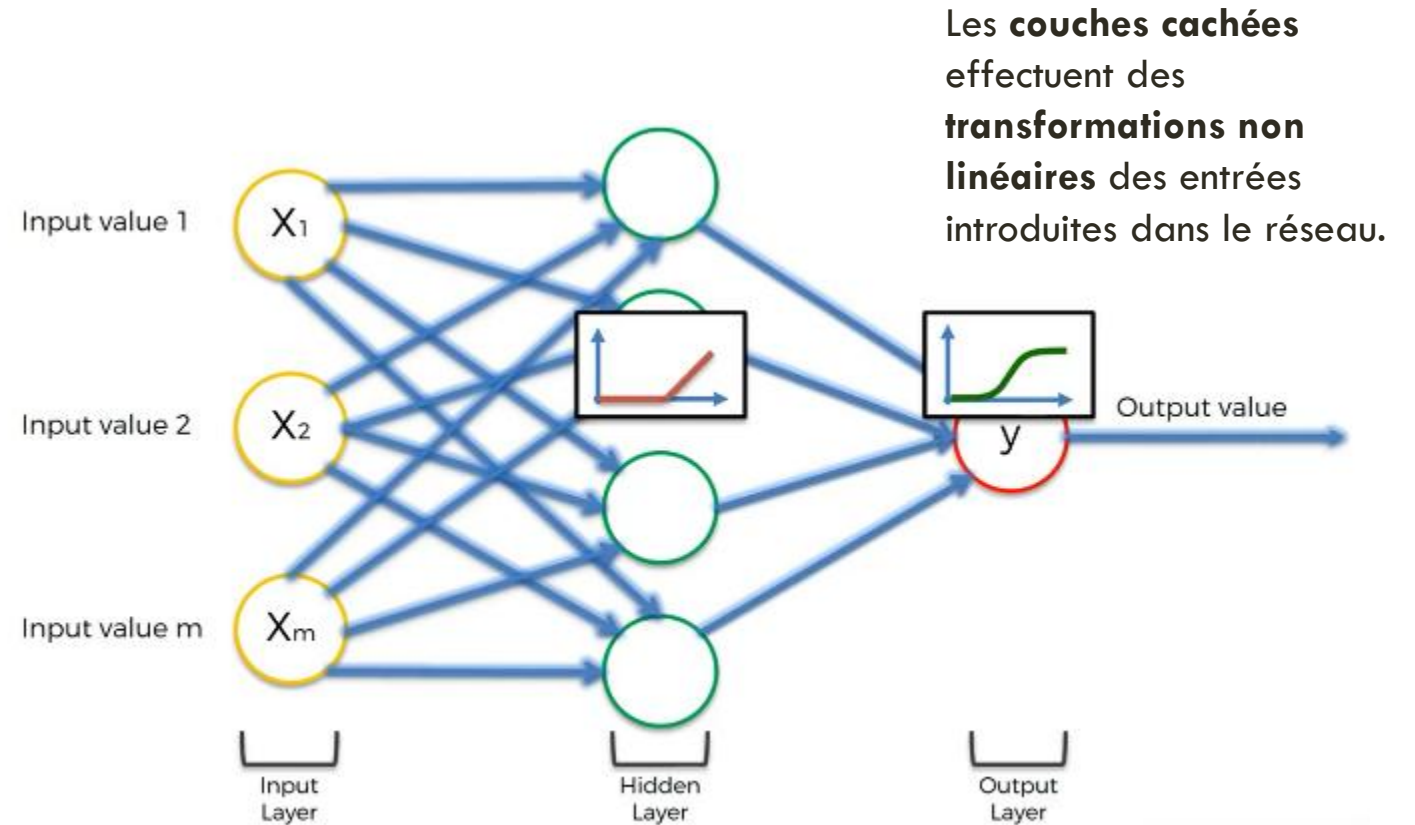


Nous appliquons
Soit une fonction de Seuil (Threshold)
Soit une fonction Sigmoïde

e.g. :

Probabilité entre 0 et 0,5 : équivalente à $y = 0$

Probabilité entre 0,5 et 1 : équivalente à $y = 1$



Dans la couche cachée, nous appliquons la fonction Rectifier (ReLU).
Dans la couche de sortie, nous appliquons la fonction Sigmoïde.

LE PROCESSUS DE L'APPRENTISSAGE DU ANN

Initialisation des poids et biais

Propagation de l'information à travers les fonctions d'agrégation et d'activation

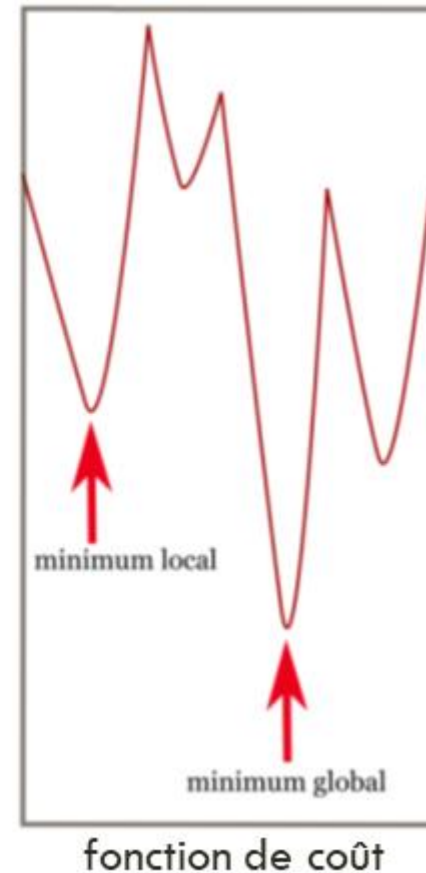
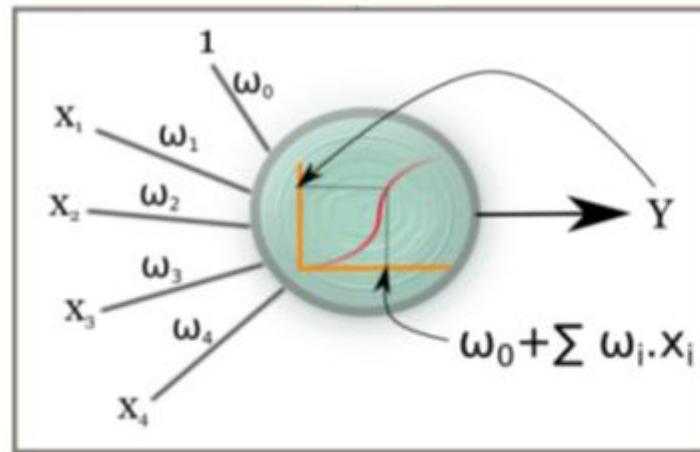
Calcul des sorties correspondantes

Calcul de la somme d'erreurs quadratiques entre les sorties prédites et les sorties actuelles

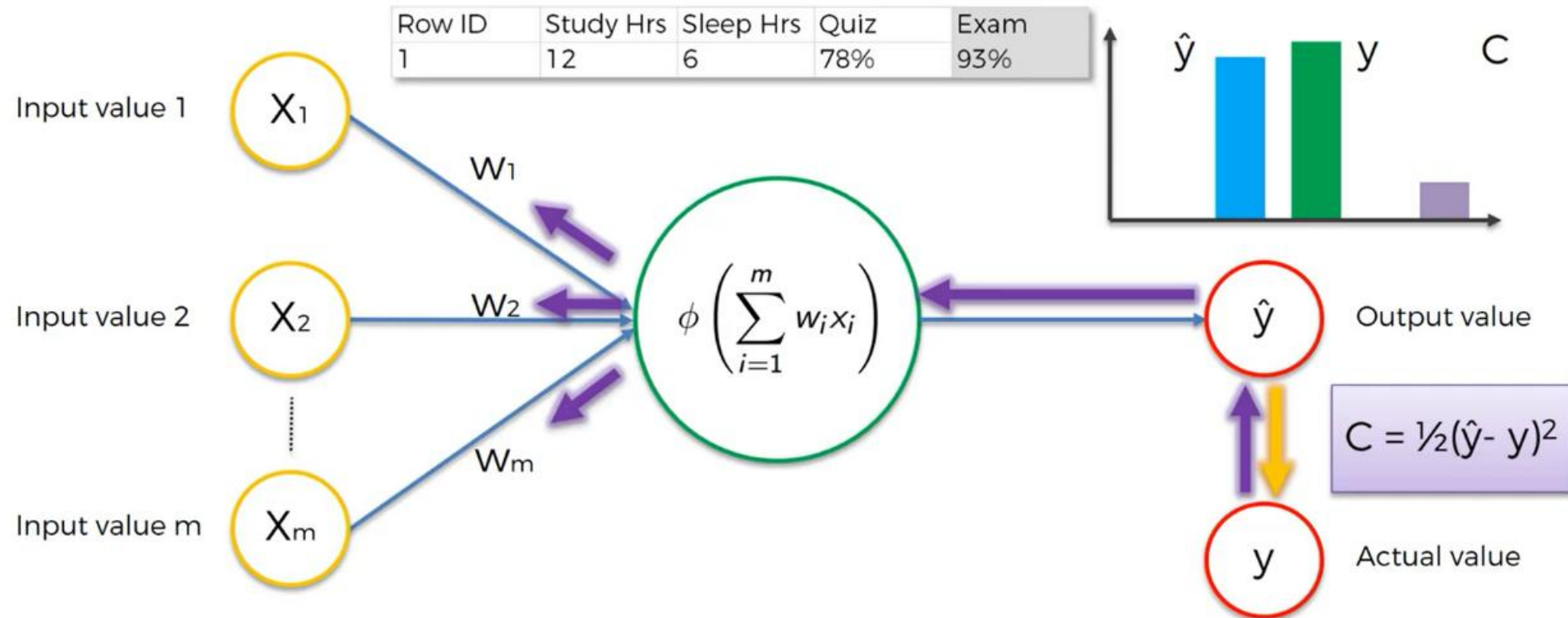
Rétropropagation basée sur le gradient pour l'optimisation du ANN en ajustant les poids

COMMENT OPTIMISER LE ANN ?

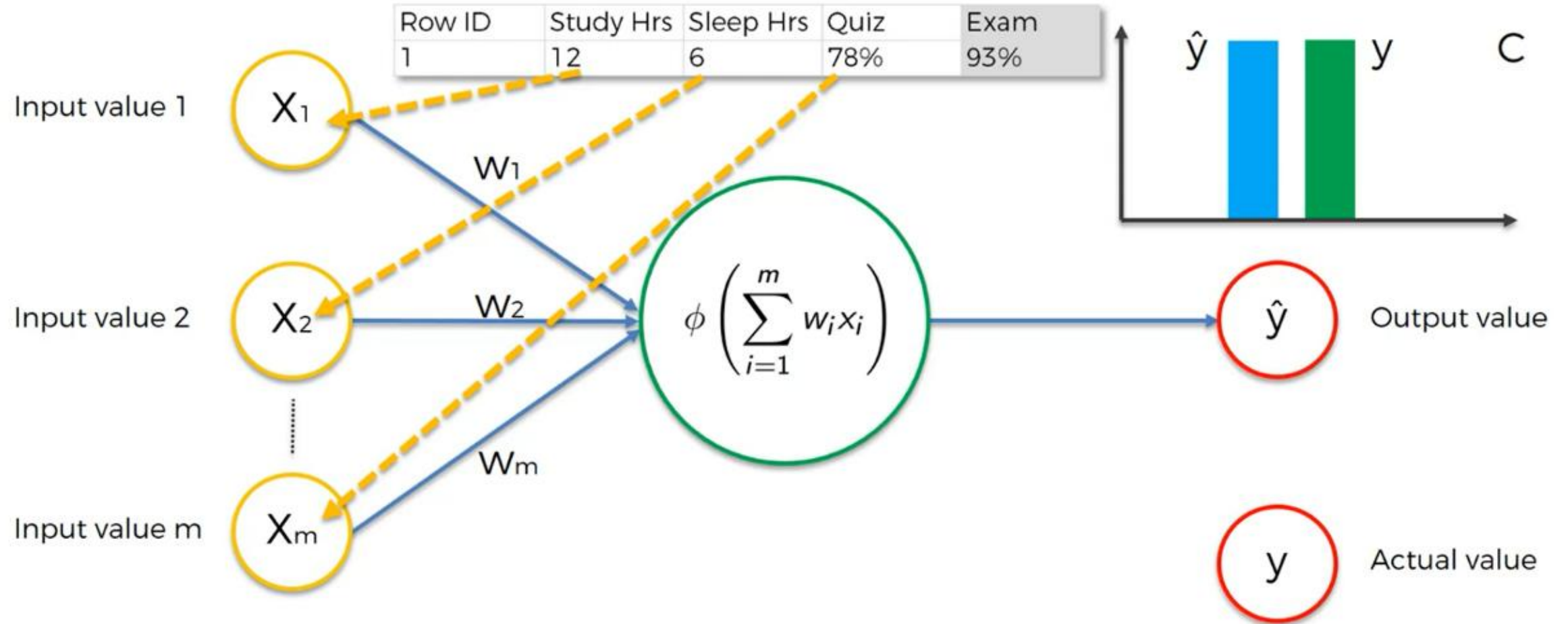
- L'objectif est de mémoriser les poids optimaux et les biais avec un taux d'erreur minimal.



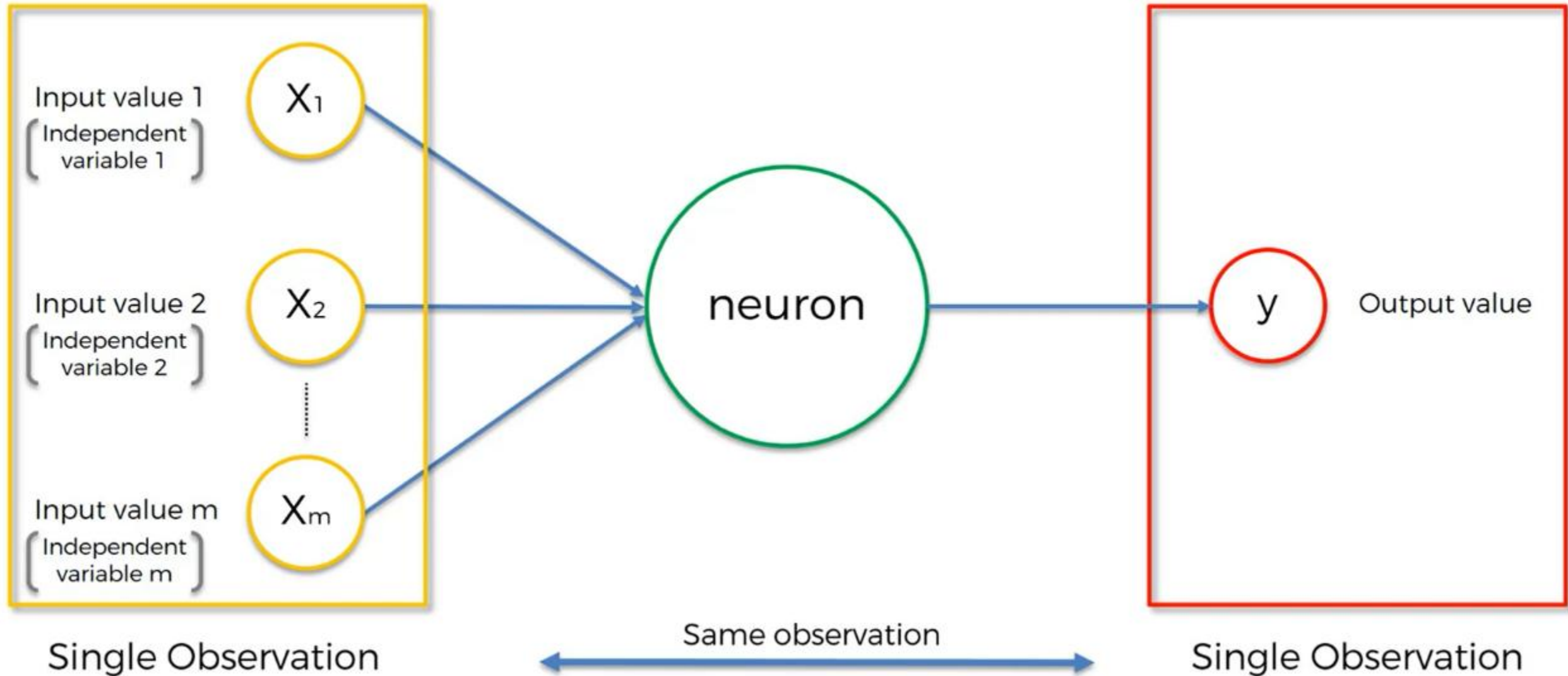
COMMENT OPTIMISER LE ANN ?



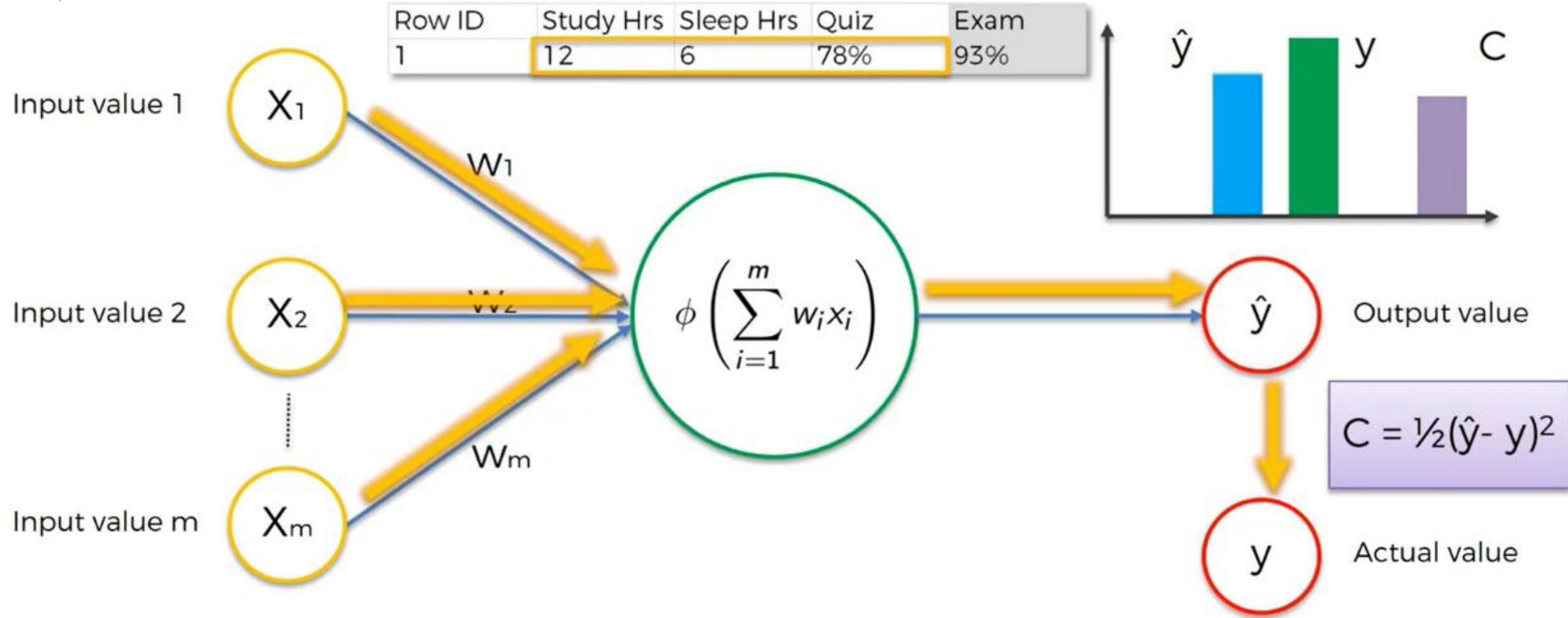
COMMENT LE ANN APPREND ?



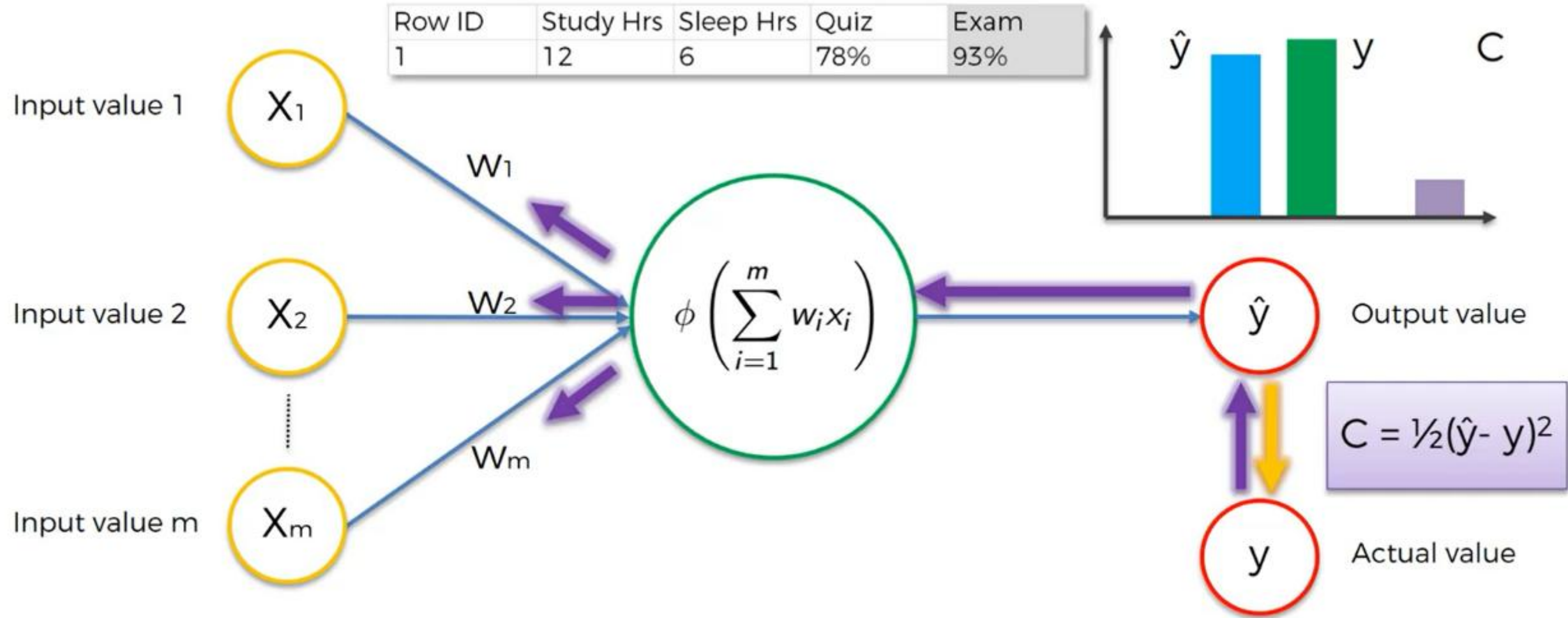
COMMENT LE ANN APPREND ?



COMMENT LE ANN APPREND ?



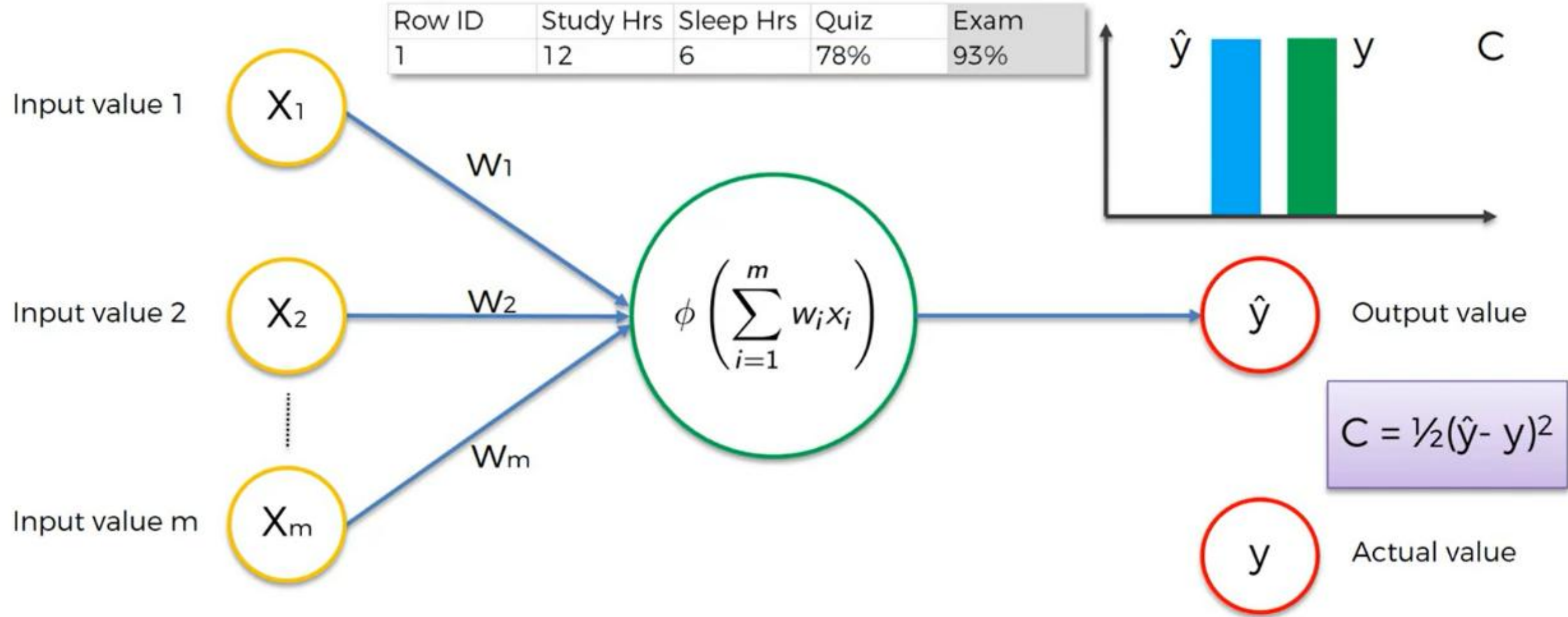
COMMENT LE ANN APPREND ?



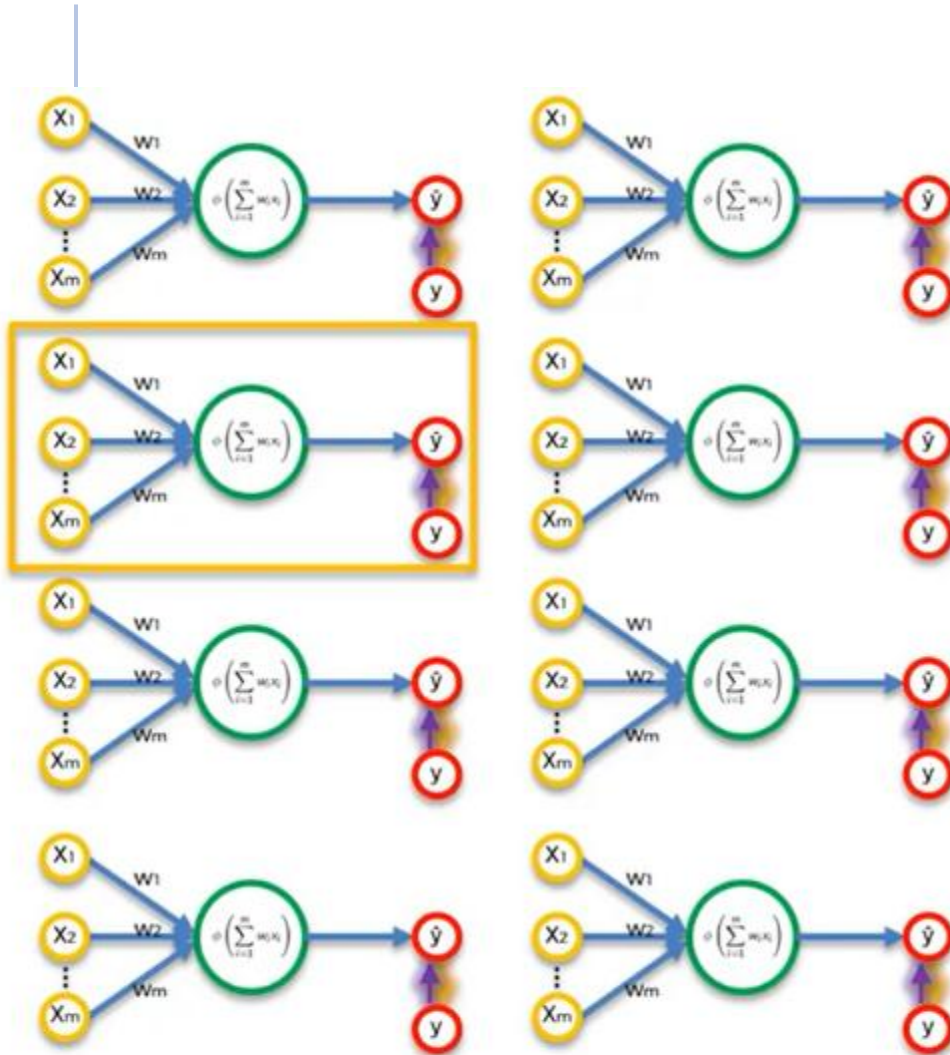
We repeat the same process.

Our purpose is trying to minimize that cost function, so Y and $\hat{Y}$

COMMENT LE ANN APPREND ?



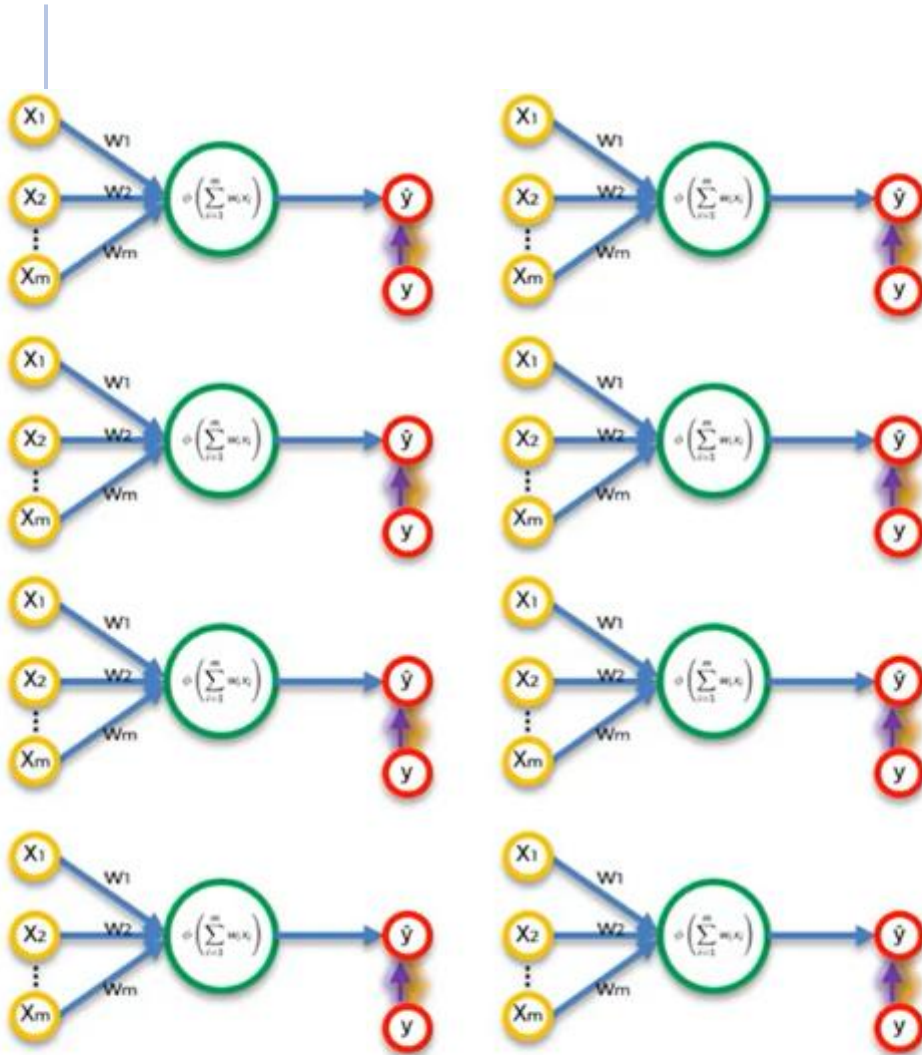
COMMENT LE ANN APPREND ?



Row ID	Study Hrs	Sleep Hrs	Quiz	Exam
1	12	6	78%	93%
2	22	6.5	24%	68%
3	115	4	100%	95%
4	31	9	67%	75%
5	0	10	58%	51%
6	5	8	78%	60%
7	92	6	82%	89%
8	57	8	91%	97%



COMMENT LE ANN APPREND ?



Row ID	Study Hrs	Sleep Hrs	Quiz	Exam
1	12	6	78%	93%
2	22	6.5	24%	68%
3	115	4	100%	95%
4	31	9	67%	75%
5	0	10	58%	51%
6	5	8	78%	60%
7	92	6	82%	89%
8	57	8	91%	97%

$$C = \sum \frac{1}{2}(\hat{y} - y)^2$$

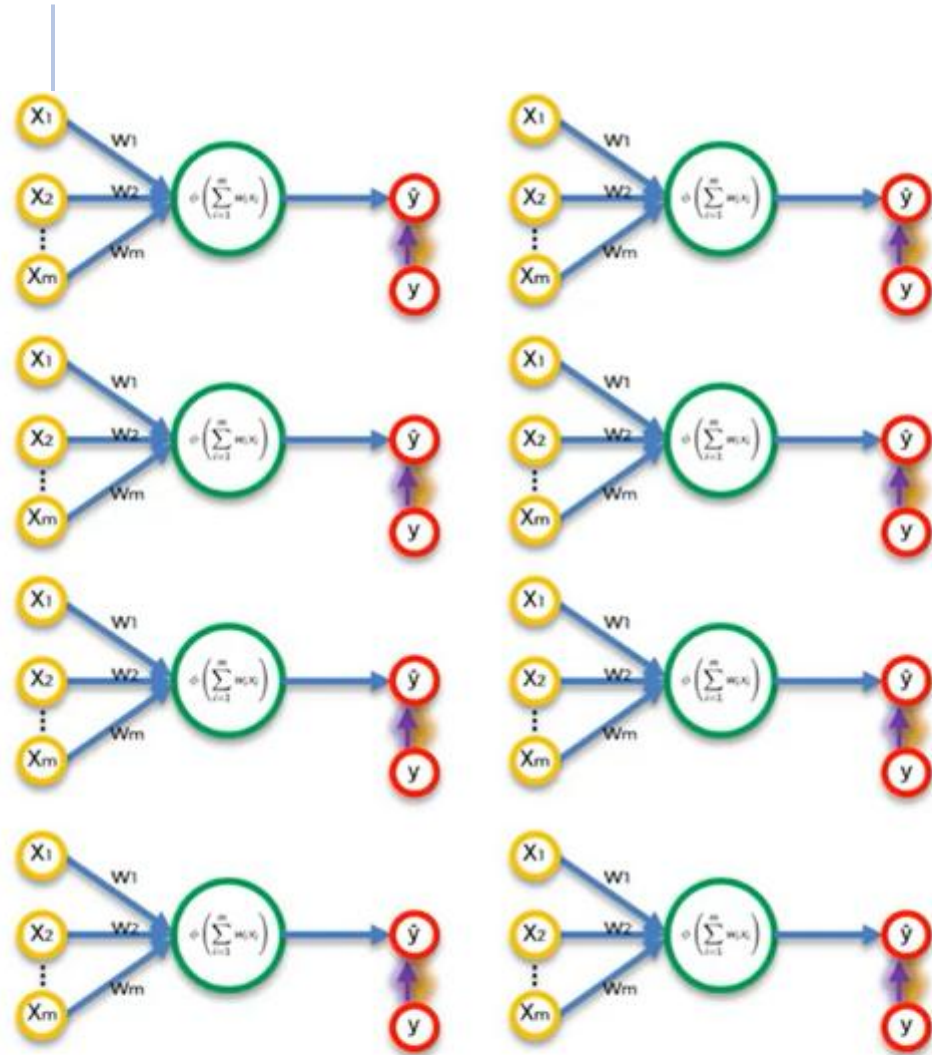


All these perceptrons are the same NN

We train our NN with all the rows, until we have all the Y and $\hat{Y}$
We calculate the Cost function, and we go back, and adjust the weights

Sarah Malaeb

COMMENT LE ANN APPREND ?

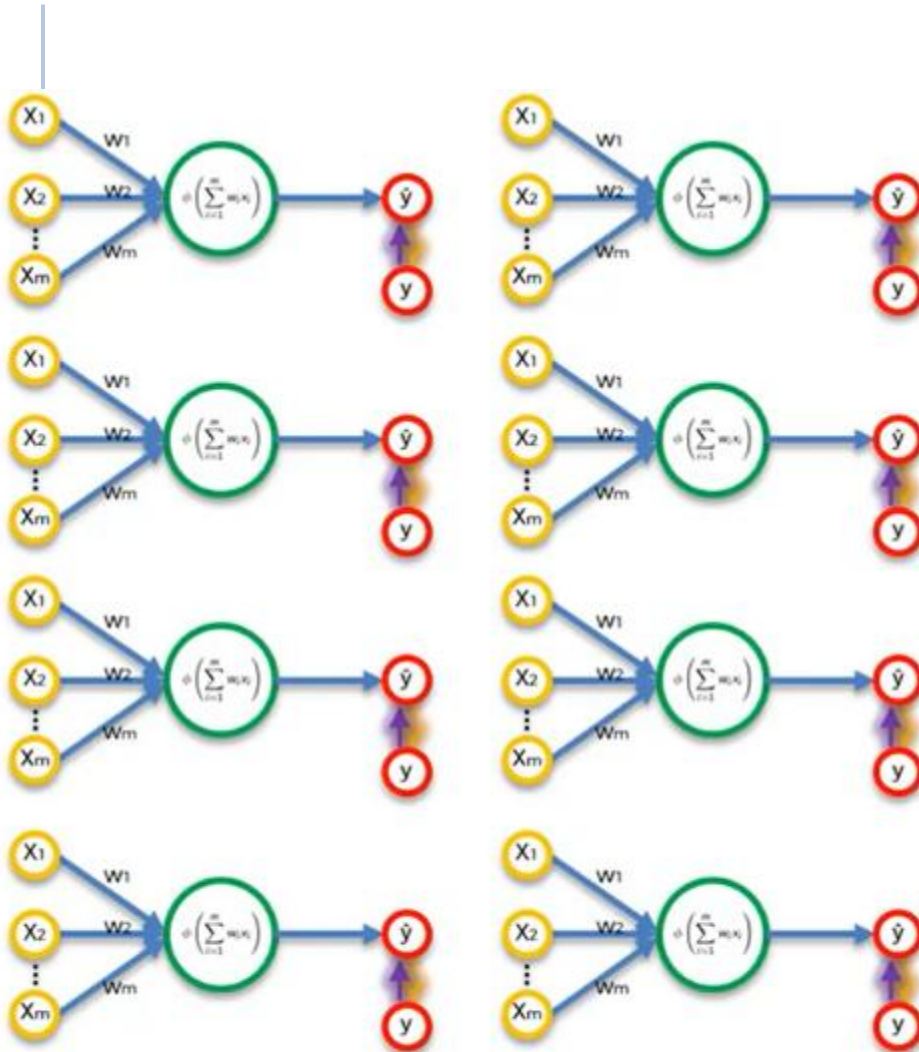


Row ID	Study Hrs	Sleep Hrs	Quiz	Exam
1	12	6	78%	93%
2	22	6.5	24%	68%
3	115	4	100%	95%
4	31	9	67%	75%
5	0	10	58%	51%
6	5	8	78%	60%
7	92	6	82%	89%
8	57	8	91%	97%

$$C = \sum \frac{1}{2}(\hat{y} - y)^2$$



COMMENT LE ANN APPREND ?



Row ID	Study Hrs	Sleep Hrs	Quiz	Exam
1	12	6	78%	93%
2	22	6.5	24%	68%
3	115	4	100%	95%
4	31	9	67%	75%
5	0	10	58%	51%
6	5	8	78%	60%
7	92	6	82%	89%
8	57	8	91%	97%

$$C = \sum \frac{1}{2}(\hat{y} - y)^2$$



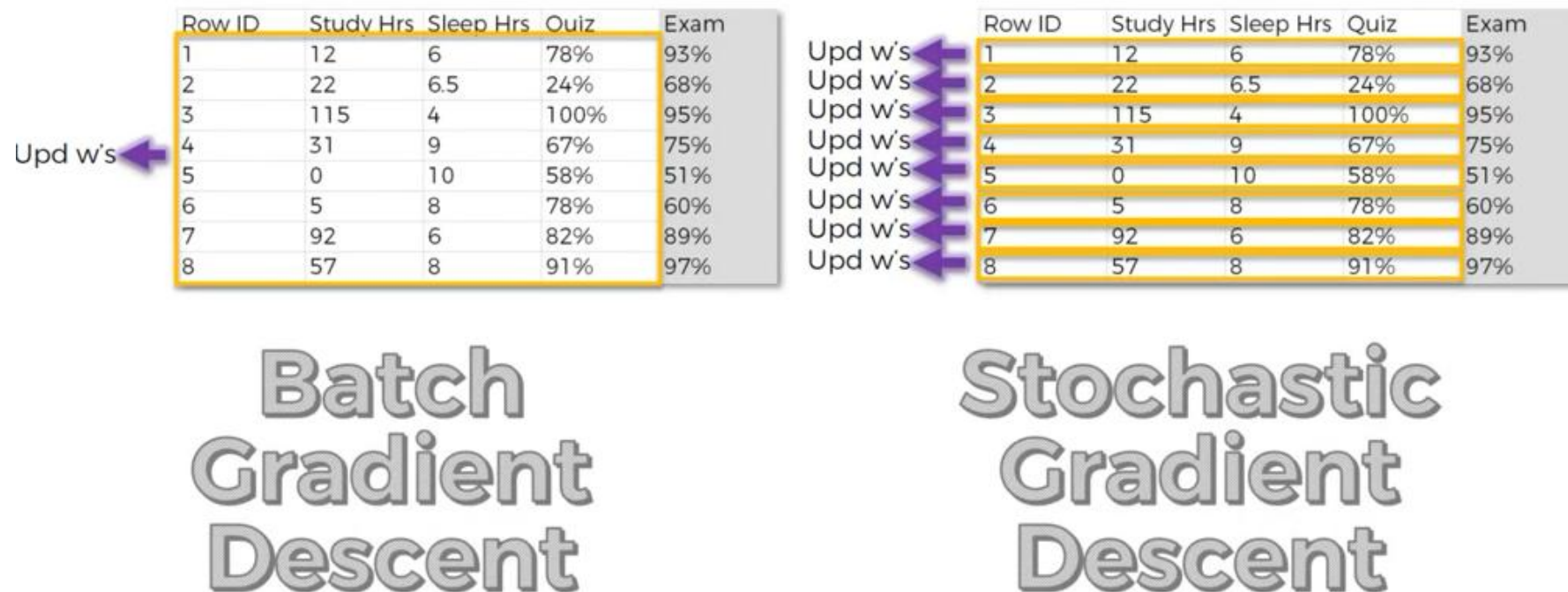
La valeur C doit être minimisée, ainsi nous trouvons les poids optimaux pour notre réseau de neurones (NN) et atteignons le réseau final.

Nous passons ensuite à la phase de test.

Tout ce processus est appelé rétropropagation (Back Propagation). Sarah Malaeb

COMMENT OPTIMISER LE ANN ?

- Pour calculer la fonction de coût, les techniques les plus couramment utilisées sont:
- la descente de gradient par mini-lots (Batch Gradient Descent) et la descente de gradient stochastique (Stochastic Gradient Descent).

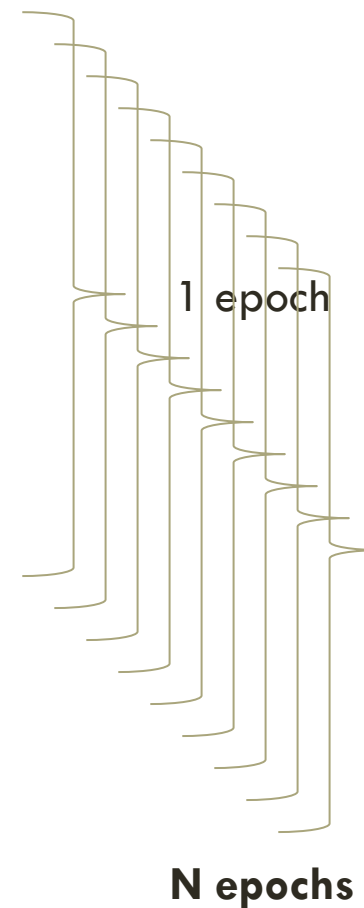
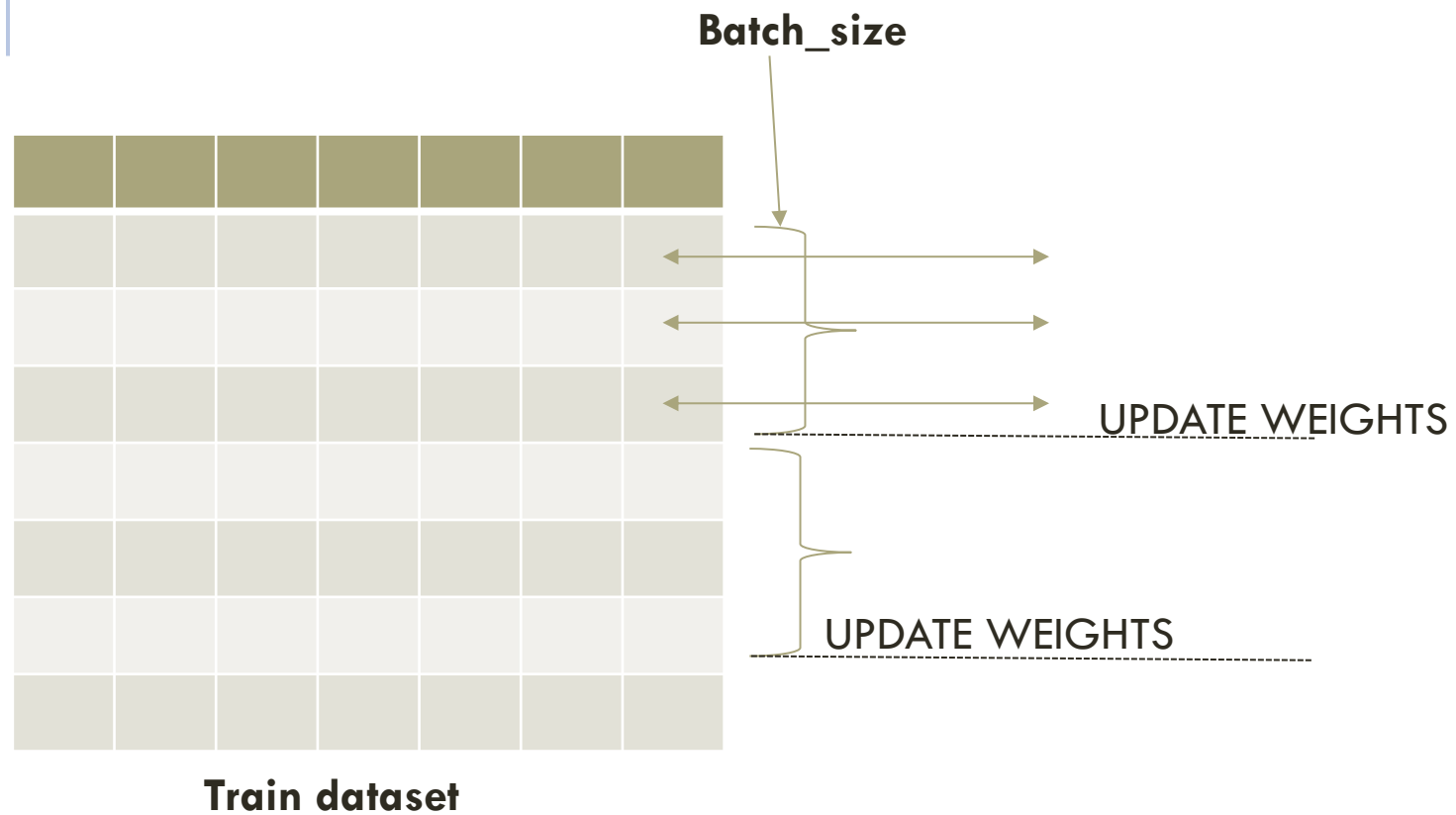


COMMENT OPTIMISER LE ANN ?

- **Le processus de validation** des ANN permet d'évaluer les performances du modèle sur des données non vues. Les hyperparamètres tels que le taux d'apprentissage et la taille du lot seront ajustés, généralement à l'aide de techniques d'optimisation comme la recherche par grille (Grid search) ou l'optimisation bayésienne, afin d'optimiser la performance du modèle.
- En parallèle, **la descente de gradient** est utilisée pour mettre à jour les poids du réseau de manière itérative en minimisant l'erreur entre les prédictions du modèle et les valeurs réelles, permettant ainsi au réseau de converger vers une meilleure représentation des données et d'améliorer ses performances prédictives.
- **L'écho (ou rétropropagation)** est un processus essentiel dans les ANN où l'erreur est propagée en arrière à travers les couches du réseau pour ajuster les poids des connexions.
- **Le batch** fait référence à la pratique de mettre à jour les poids du réseau en utilisant un groupe de plusieurs exemples d'entraînement à la fois plutôt qu'un seul, ce qui peut accélérer la convergence de l'apprentissage.

BATCH_SIZE?

EPOCHS?



EXEMPLE PYTHON UTILISANT LA BIBLIOTHÈQUE KERAS (INTÉGRÉE À TENSORFLOW)

```
# Importation des bibliothèques nécessaires
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

# Initialisation du modèle
classfier = Sequential()

# Couche d'entrée + première couche cachée
classfier.add(Dense(
    units=6,                      # nombre de neurones dans la couche cachée
    kernel_initializer='uniform', # initialisation des poids
    activation='relu',            # fonction d'activation ReLU
    input_dim=11))               # nombre de variables d'entrée (features)

# Deuxième couche cachée (optionnelle)
classfier.add(Dense(
    units=6,
    kernel_initializer='uniform',
    activation='relu'))

# Couche de sortie
classfier.add(Dense(
    units=1,                      # 1 neurone car classification binaire
    kernel_initializer='uniform',
    activation='sigmoid'))        # sortie entre 0 et 1 → probabilité
```

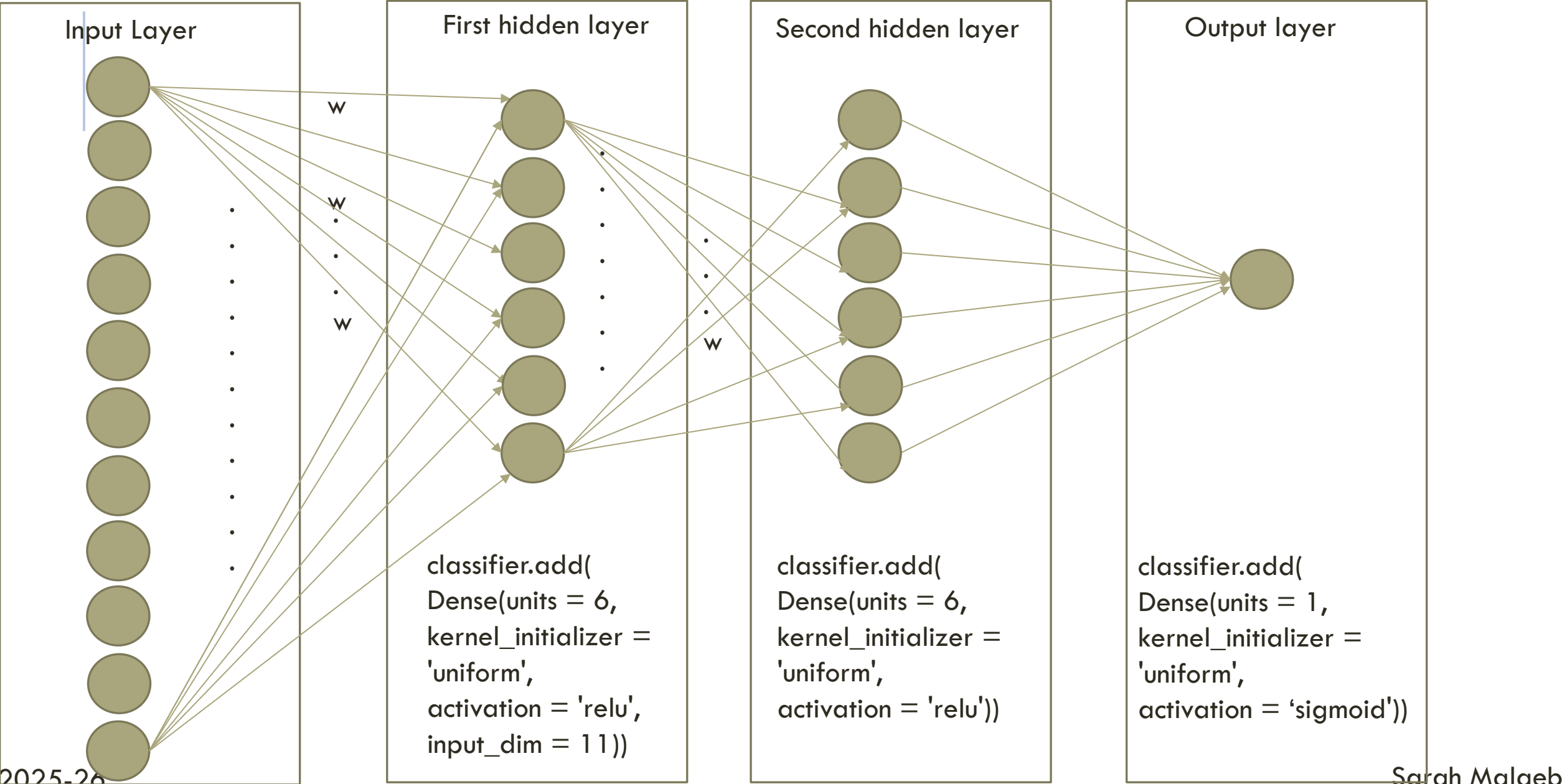
```
# Entraînement du modèle
classfier.fit(
    X_train,          # variables explicatives d'entraînement
    y_train,          # étiquettes correspondantes
    batch_size=10,    # nombre d'exemples par lot
    epochs=100,       # nombre d'itérations complètes sur le dataset
    verbose=1         # afficher la progression
)

# Évaluation du modèle sur les données de test
loss, accuracy = classfier.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy*100:.2f}%")

# Prédictions
y_pred = classfier.predict(X_test)
y_pred = (y_pred > 0.5) # seuil pour transformer les probabilités en classes (0 ou 1)

# Exemple d'affichage
print("Exemple de prédiction :", y_pred[:10])
```

EXEMPLE PYTHON UTILISANT LA BIBLIOTHÈQUE KERAS (INTÉGRÉE À TENSORFLOW)



RÉSEAUX DE NEURONES ARTIFICIELS

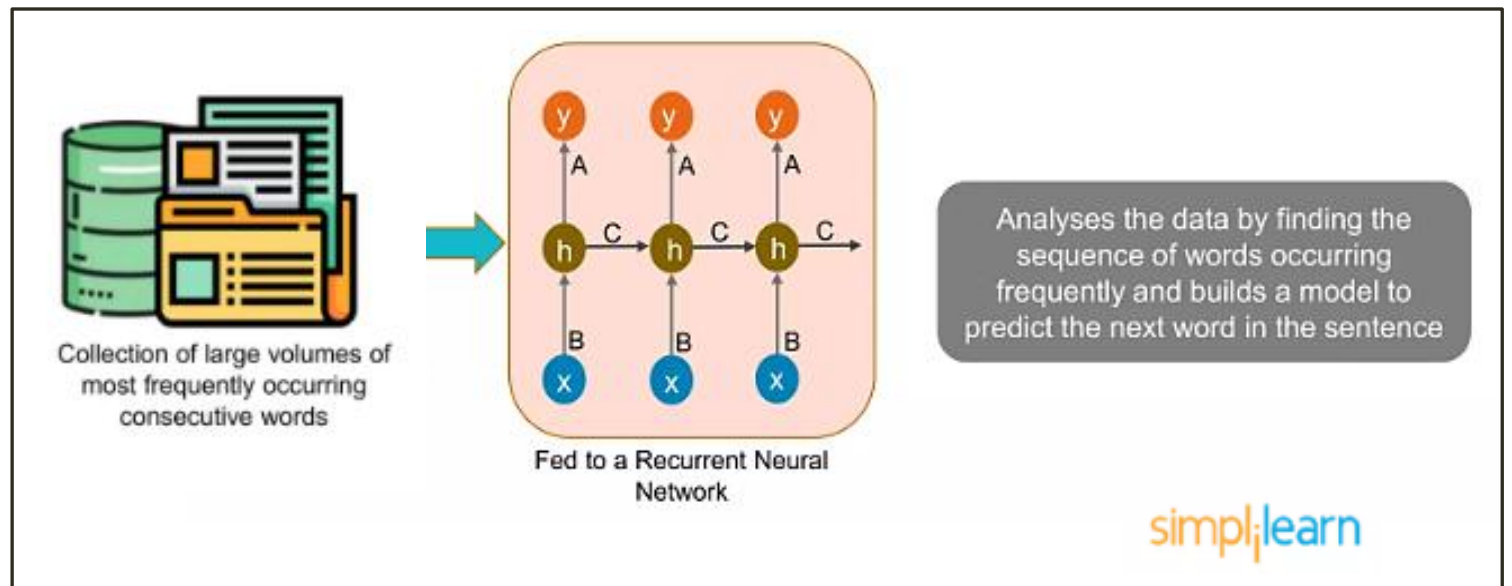
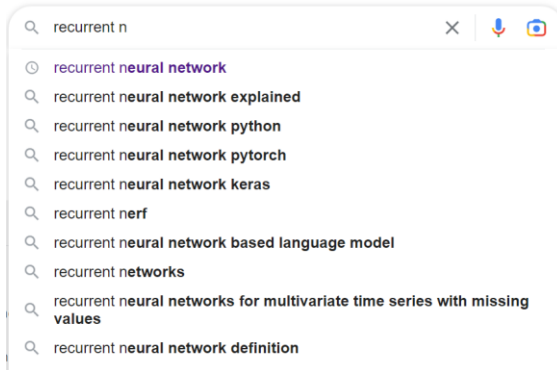
Conclusion

- L'apprentissage supervisé, la rétropropagation, et la descente de gradient sont au cœur du processus d'entraînement des ANN.
- L'ajustement des hyperparamètres est crucial pour optimiser les performances des ANN sur des tâches spécifiques.
- Les réseaux de neurones profonds ont permis des avancées significatives dans divers domaines tels que la vision par ordinateur, le traitement du langage naturel et la reconnaissance de motifs, etc.
- Les réseaux de neurones artificiels ne représentent qu'une partie de l'éventail des techniques d'apprentissage automatique. Il existe en effet de nombreux autres types de réseaux de neurones spécialisés, tels que les réseaux de neurones récurrents (RNN), les réseaux de neurones convolutionnels (CNN), les réseaux adversariaux génératifs (GAN) et bien d'autres, chacun ayant ses propres avantages et domaines d'application.

RÉSEAUX DE NEURONES RÉCURRENTS

INTRODUCTION AUX RÉSEAUX DE NEURONES RÉCURRENTS (RNN)

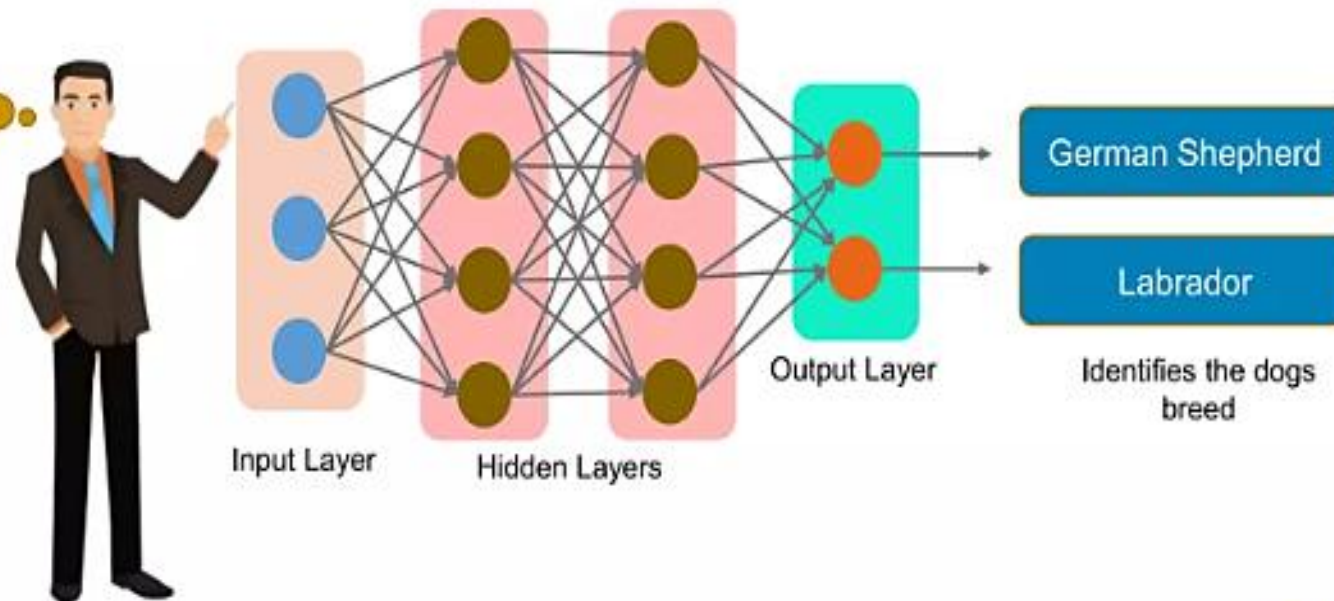
Savez-vous comment la fonction d'autocomplétion de Google prédit la suite des mots qu'un utilisateur est en train de taper ?



INTRODUCTION AUX RÉSEAUX DE NEURONES RÉCURRENTS (RNN)

Such networks do not
require memorizing the
past output

De tels réseaux ne nécessitent pas de mémoriser les sorties passées.

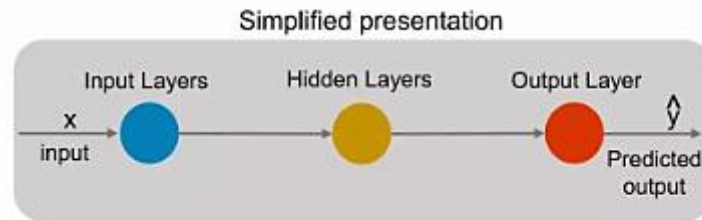
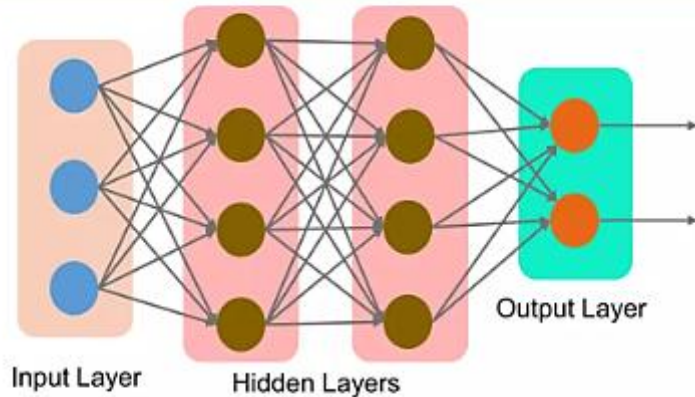


simpli|learn

INTRODUCTION AUX RÉSEAUX DE NEURONES RÉCURRENTS (RNN)

Pourquoi les RNNs ?

In a Feed-Forward Network, information flows only in forward direction, from the input nodes, through the hidden layers (if any) and to the output nodes. There are no cycles or loops in the network.



- Decisions are based on current input
- No memory about the past
- No future scope

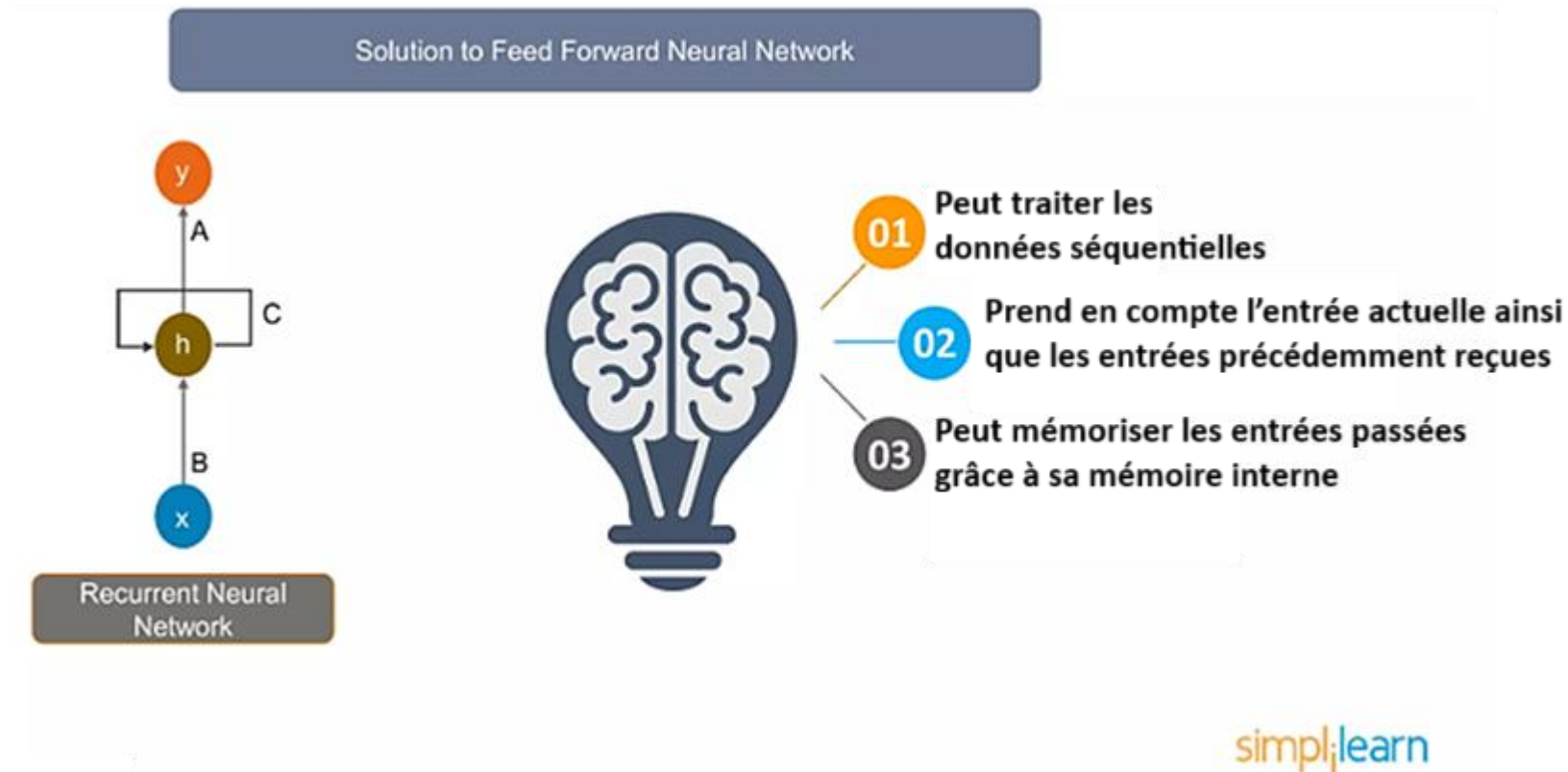
Problèmes avec les Réseaux de Neurones Feed-Forward :



- 01 Ne peut pas traiter les données séquentielles
- 02 Ne considère que l'entrée actuelle
- 03 Ne peut pas mémoriser les entrées précédentes

INTRODUCTION AUX RÉSEAUX DE NEURONES RÉCURRENTS (RNN)

Pourquoi les RNNs ?



EXEMPLE DE DONNÉES SÉQUENTIELLES

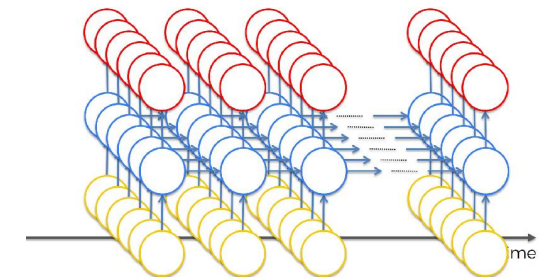
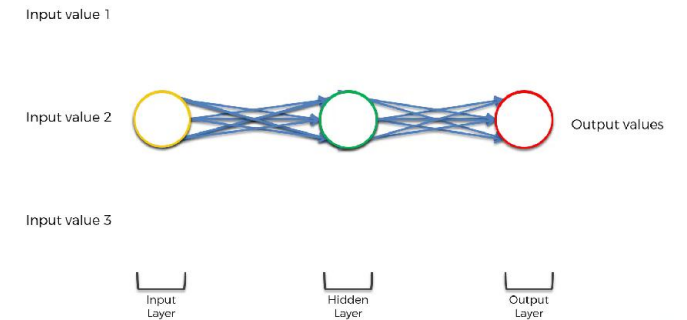
Exemple de données séquentielles : le texte

- L'extraction d'entités dans un texte est un excellent exemple de la manière dont les données dans différentes parties d'une séquence peuvent s'influencer mutuellement.
- Avec les entités, les mots qui précèdent et qui suivent l'entité dans la phrase ont un effet direct sur la façon dont elles sont classées.
- Afin de traiter des données temporelles ou séquentielles, comme les phrases, nous devons utiliser des algorithmes conçus pour apprendre à partir des données passées et des données futures dans la séquence.

EXEMPLE DE DONNÉES SÉQUENTIELLES

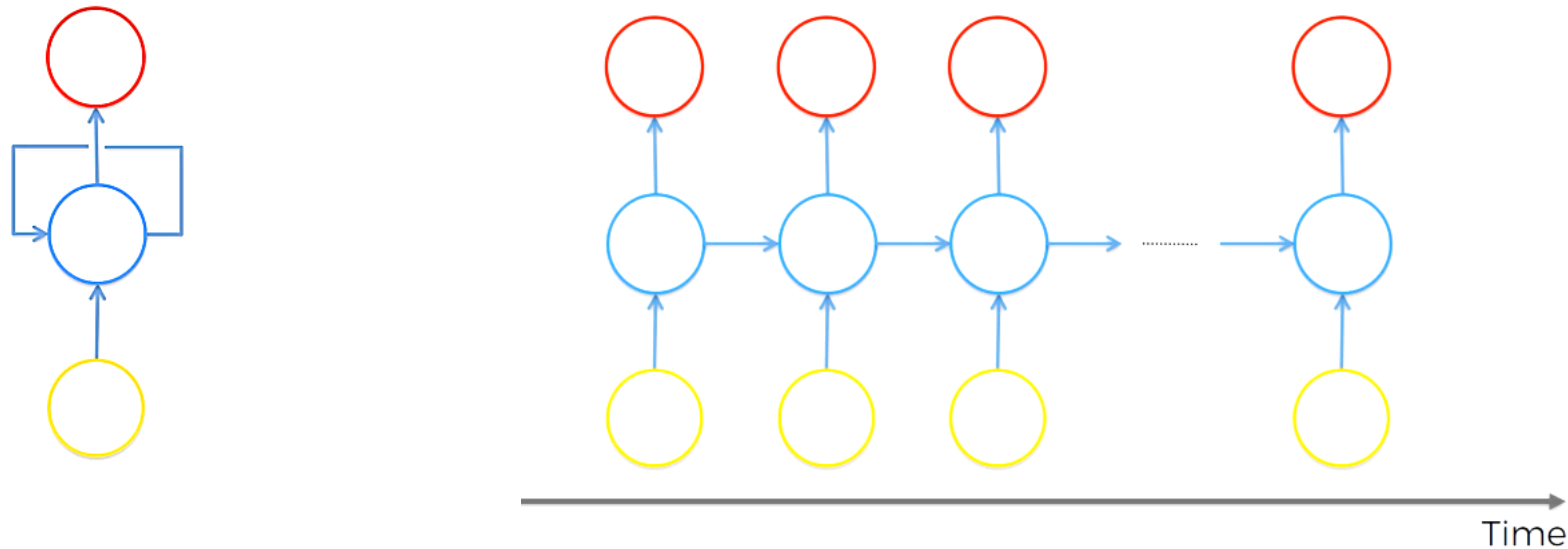
Par exemple, si vous fournissez à un **réseau de neurones feed-forward** une **séquence de lettres comme « N-E-U-R-O-N »**, au moment où il arrive à « R », il aura probablement oublié qu'il vient juste de lire « U ». C'est un gros problème et cela rend ces réseaux de neurones peu utiles.

En revanche, les **RNNs** collectent continuellement les entrées qu'ils viennent de traiter, et les stockent également. Par exemple, dans « N-E-U-R-O-N », ils **se souviendront non seulement de « R » mais aussi du « U » qui le précédait**. **C'est très utile : le modèle fera un meilleur travail pour prédire la suite de la séquence.**



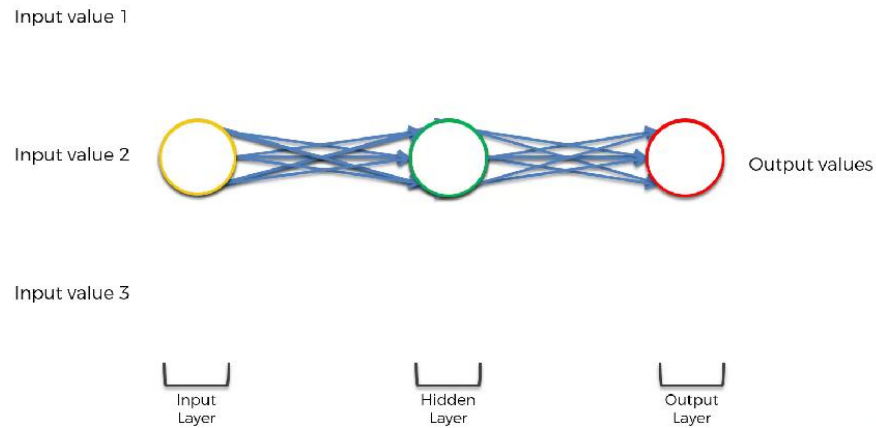
RÉSEAU DE NEURONES RÉCURRENTS (RNN)

Un RNN est une classe de réseaux de neurones artificiels dans laquelle **les connexions entre les nœuds peuvent créer une boucle**, permettant à la sortie de certains nœuds d'influencer les entrées suivantes de ces mêmes nœuds. **Cela lui permet de présenter un comportement dynamique temporel.**

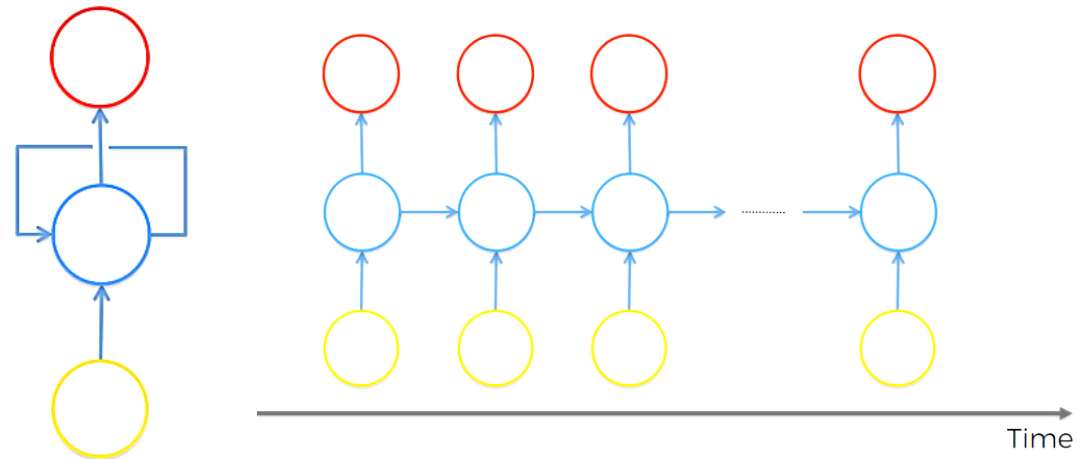


RÉSEAU DE NEURONES RÉCURRENTS (RNN)

ANN



RNN

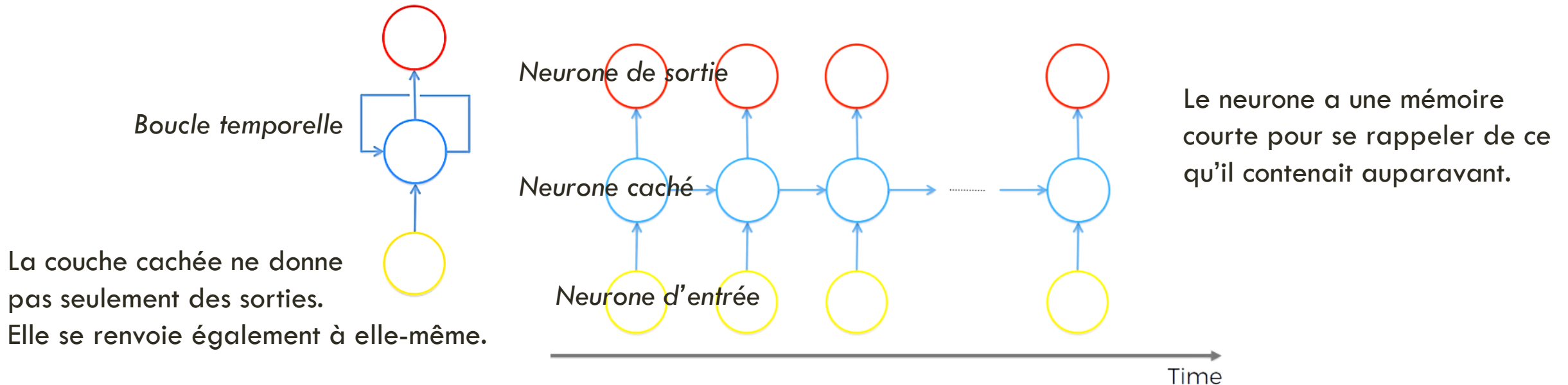


La boucle au niveau de la couche cachée se renvoie également à elle-même.

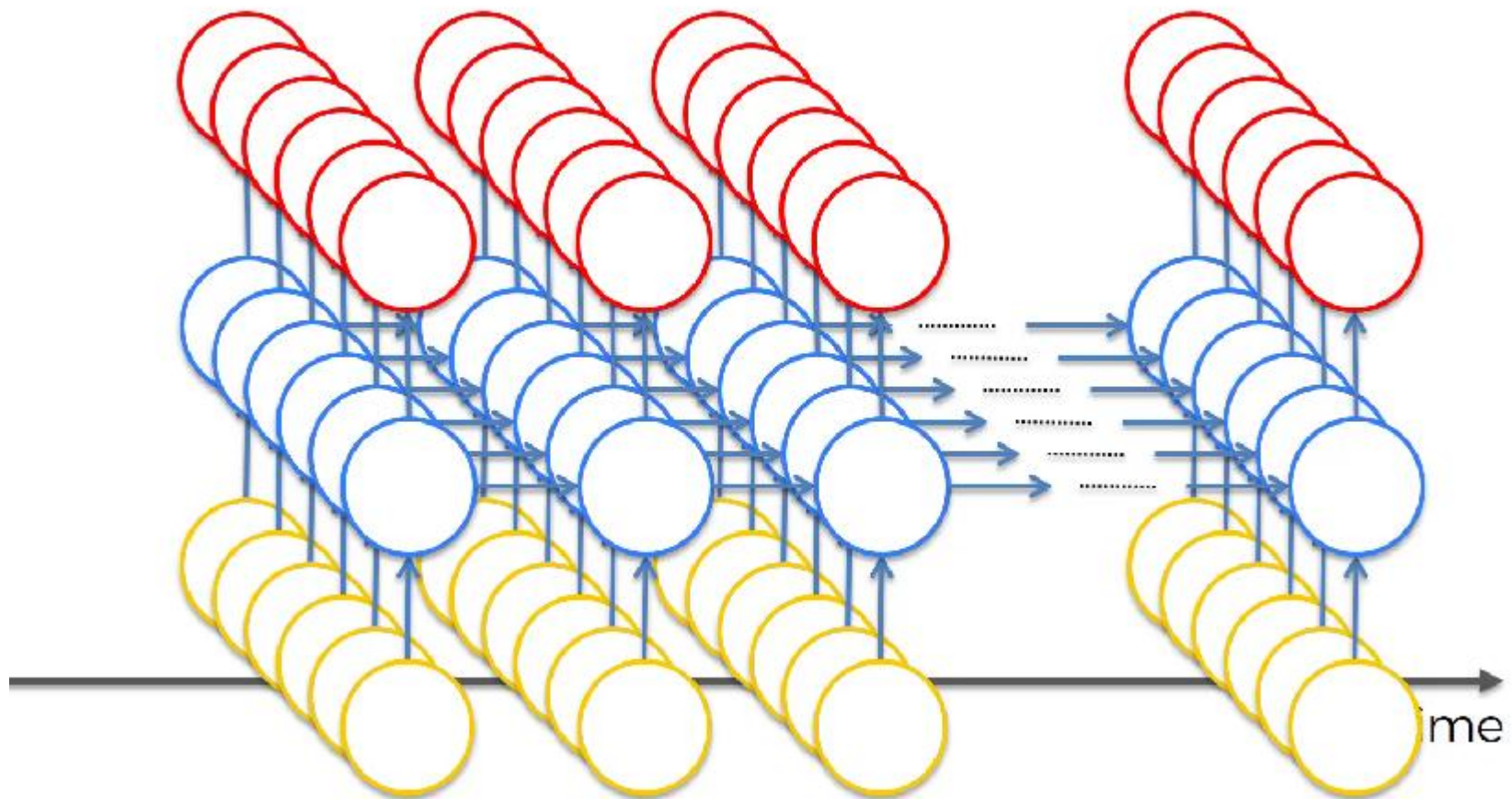
RÉSEAU DE NEURONES RÉCURRENTS (RNN)

Ils permettent la **répétition** : les RNN sont conçus pour interpréter des informations temporelles ou séquentielles.

Ces réseaux **utilisent d'autres points de données dans une séquence pour faire de meilleures prédictions**. Ils le font **en prenant une entrée et en réutilisant les activations des nœuds précédents ou suivants dans la séquence pour influencer la sortie**.



RÉSEAU DE NEURONES RÉCURRENTS (RNN)



CNN RÉSEAUX DE NEURONES CONVOLUTIFS VS. RNN RÉSEAUX DE NEURONES RÉCURRENTS

Quelle est la différence entre un CNN et un RNN ?

La principale différence entre un CNN et un RNN est la capacité à traiter des informations temporelles — des données qui arrivent sous forme de séquence, comme une phrase.

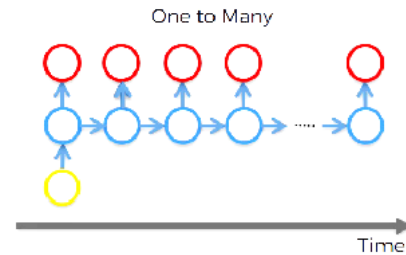
Les RNNs sont conçus précisément pour traiter les informations temporelles.

Les CNNs ne sont pas capables d'interpréter efficacement les informations temporelles.

Par conséquent, les CNN et les RNN sont utilisés pour des finalités complètement distinctes, et il existe des différences dans la structure même des réseaux afin de répondre à ces usages différents.

RÉSEAU DE NEURONES RÉCURRENTS (RNN)

Un Input – Plusieurs Outputs



Exemple : Une image en entrée, l'ordinateur décrit l'image, plusieurs sorties : toutes les descriptions (chien noir et blanc sautant au-dessus d'une barre).

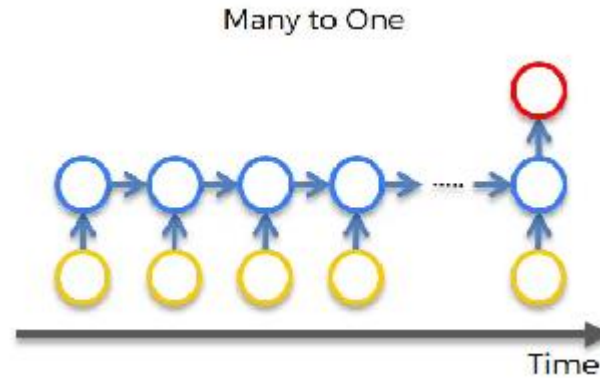


"black and white
dog jumps over
bar."

karpathy.github.io

RÉSEAU DE NEURONES RÉCURRENTS (RNN)

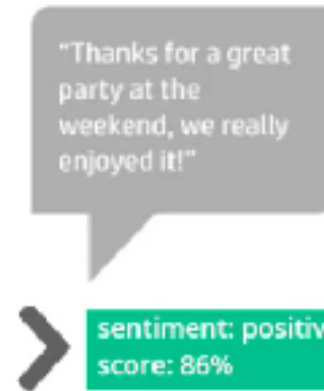
Plusieurs Inputs – Un Output



Exemple : Analyse de sentiments

Entrées multiples : nombreux textes (émotionnels, comportementaux...).

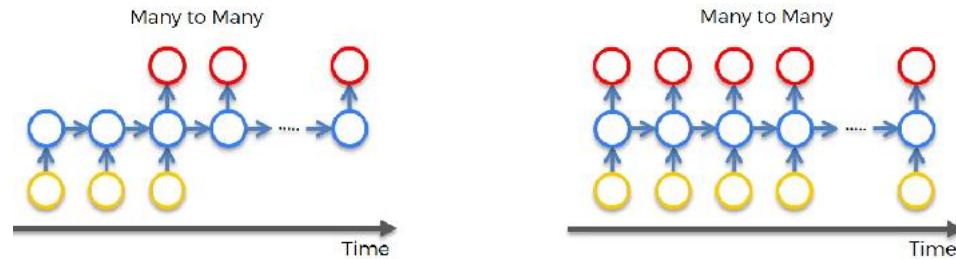
Une sortie : le score sentimental.



dev.havenondemand.com

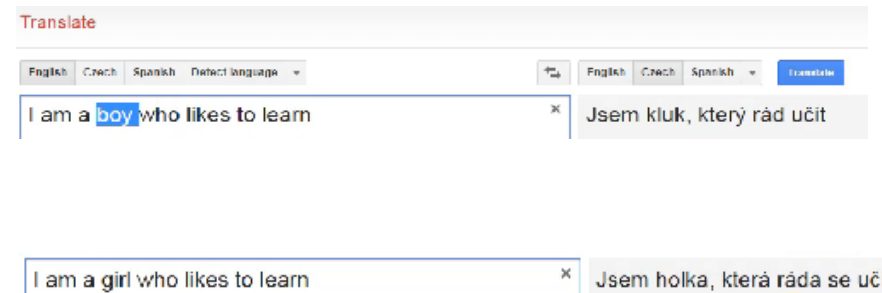
RÉSEAU DE NEURONES RÉCURRENTS (RNN)

Plusieurs Inputs – Plusieurs Outputs



Example: Traduction, chaque mot ou certains mots peuvent influencer le résultat de la traduction, par exemple les mots féminins/masculins entraînent des sorties différentes.

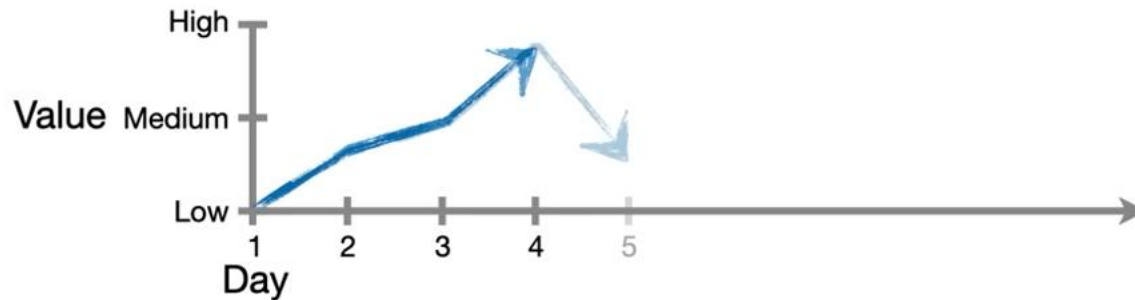
Un autre exemple : Film écrit par un algorithme



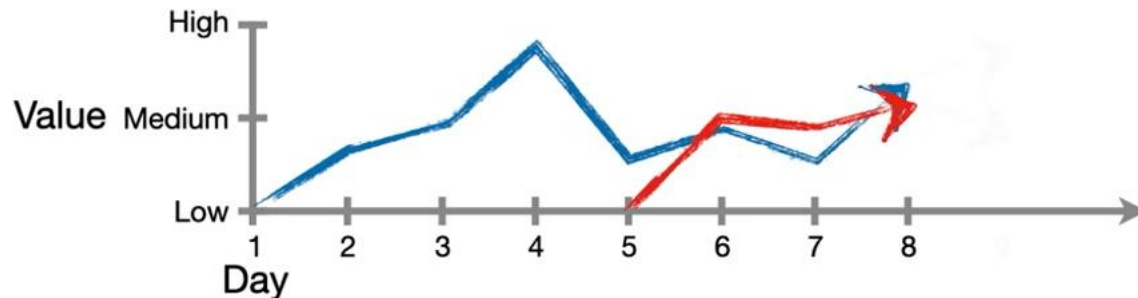
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

Les prix des actions ont tendance à évoluer au fil du temps.



Plus une entreprise est cotée depuis longtemps en bourse, plus nous avons de données à son sujet.

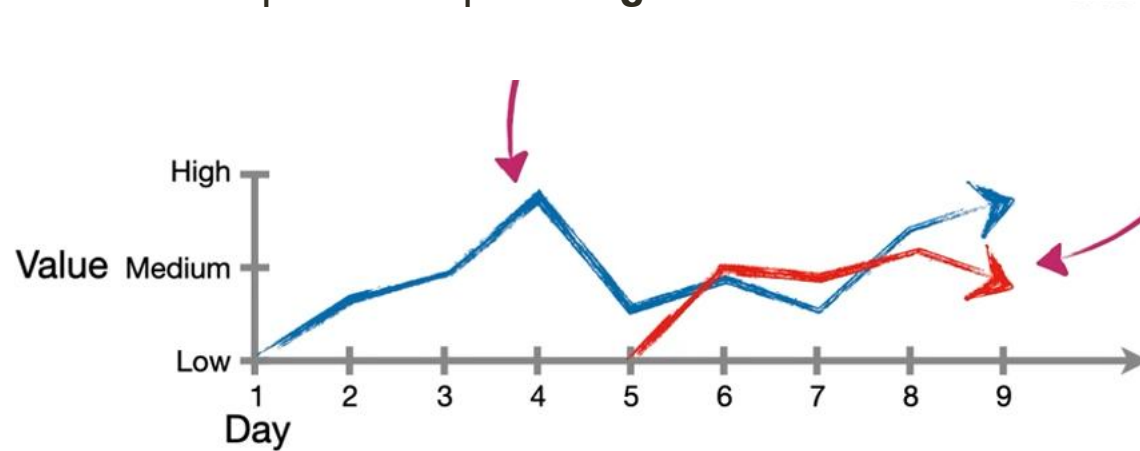


EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

Par exemple, nous avons **plus de points temporels** pour l'entreprise représentée par la **ligne bleue**...

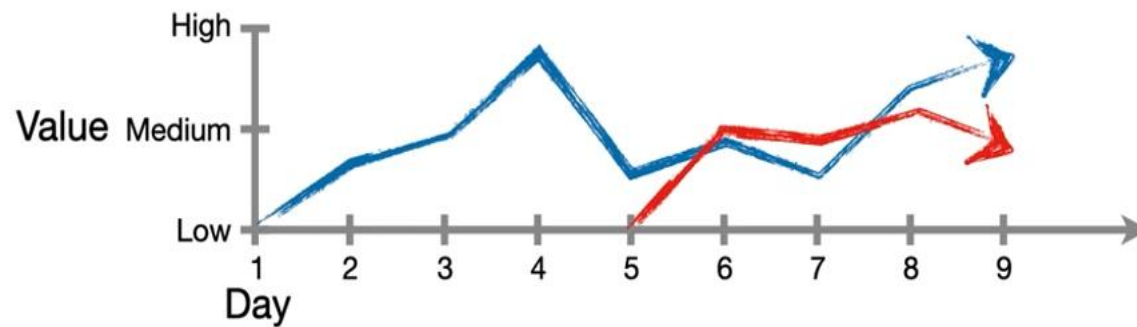
...que pour l'entreprise représentée par la **ligne rouge**.



EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

...alors nous avons besoin d'un **réseau de neurones** capable de **fonctionner avec différentes quantités de données séquentielles**.

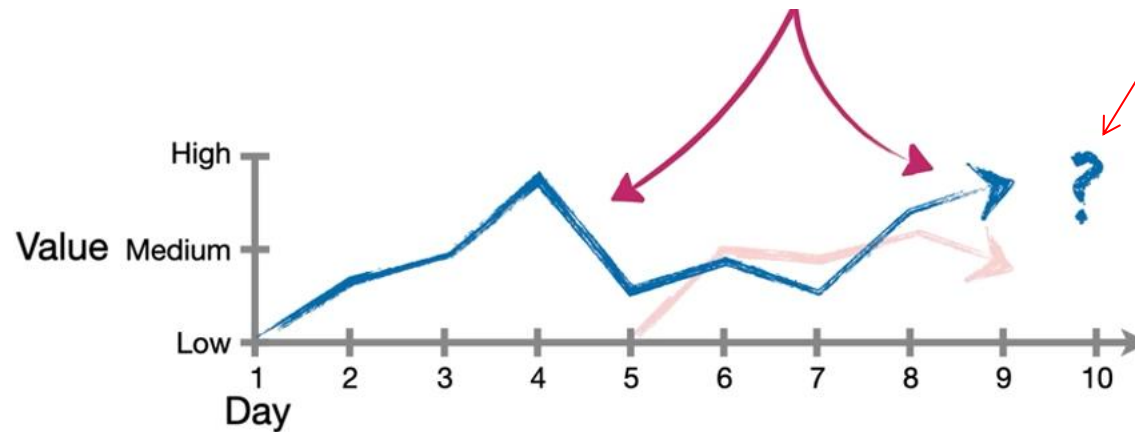


EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

En d'autres termes, si nous voulons **prédire le prix de l'action** de l'entreprise représentée par la ligne bleue au jour 10...

...alors nous pourrions vouloir utiliser **les données des 9 jours précédents**.

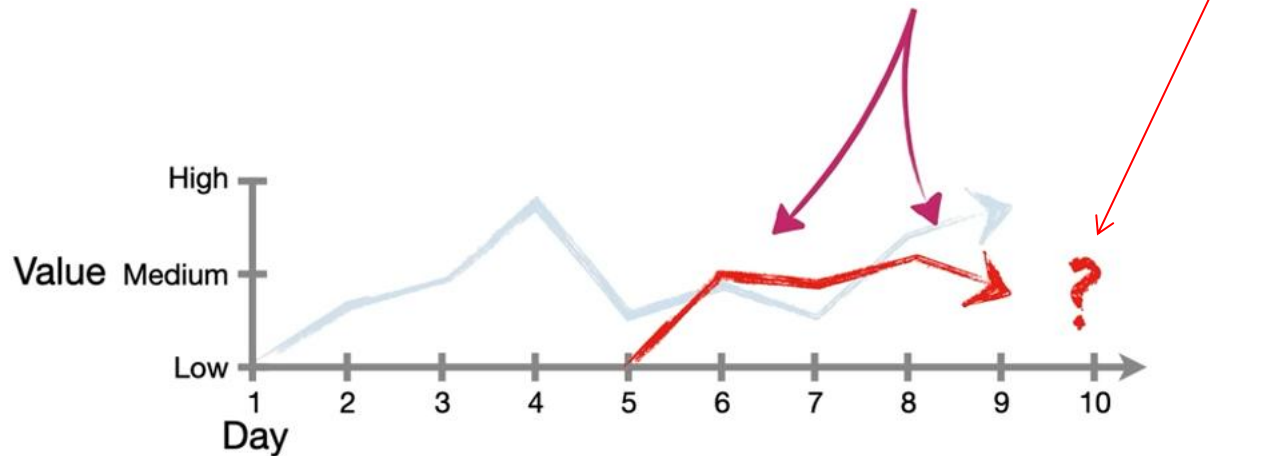


EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

En revanche, si nous voulions **prédire le prix de l'action de l'entreprise représentée par la ligne rouge au jour 10...**

...alors nous ne disposerions que des **données des 5 jours précédents.**

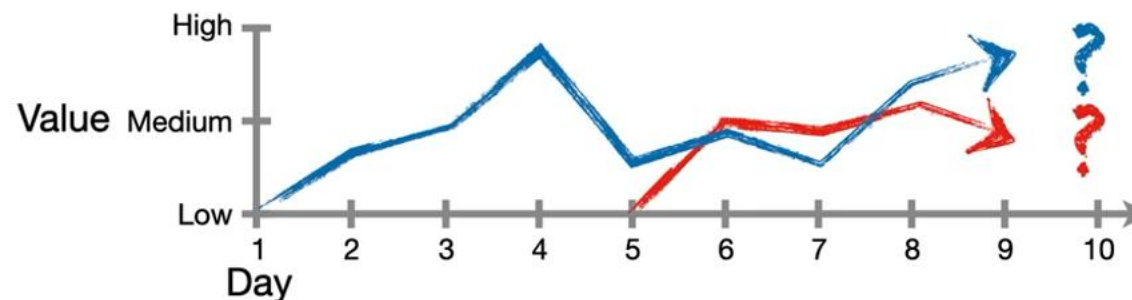


EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

Nous devons donc que le **réseau de neurones** soit **flexible** en ce qui concerne la **quantité de données séquentielles** utilisée pour faire une **prédiction**

C'est une **grande différence** par rapport aux **autres types de réseaux de neurones**.

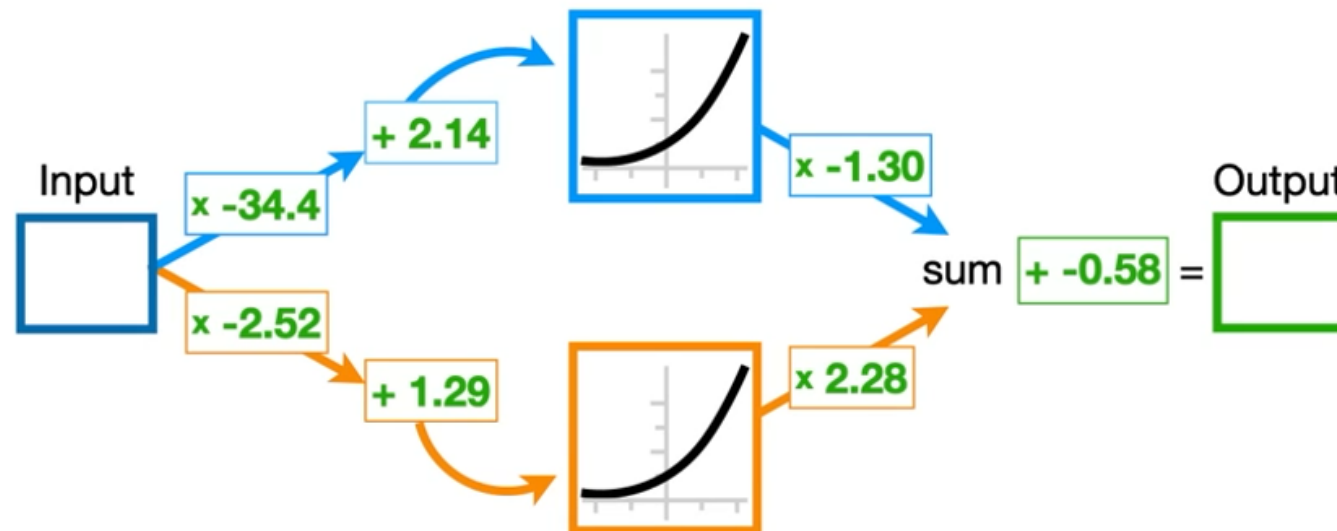


EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

Réseau de neurones qui fait des **prédictions** à partir d'une seule valeur d'entrée, ni plus ni moins.

Exemple d'un réseau de neurones Feed-Forward avec une seule entrée



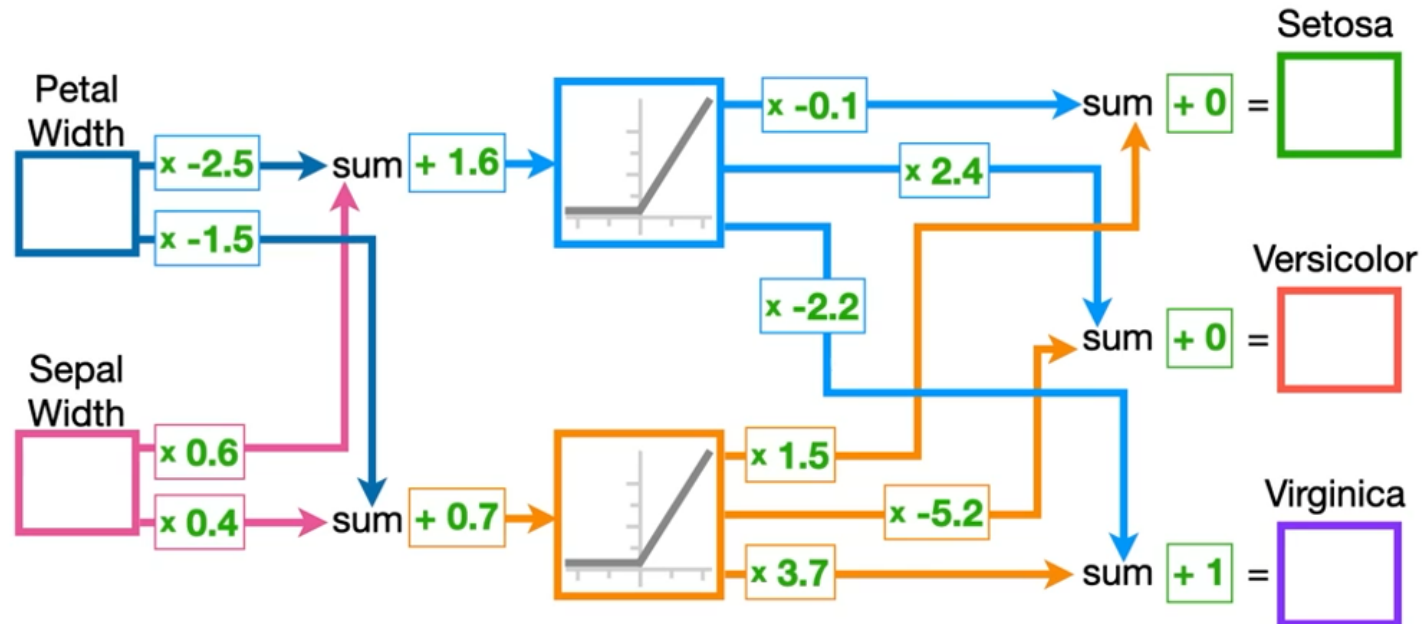
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

Réseaux de neurones avec plusieurs entrées et sorties

Vous avez vu ce réseau de neurones qui faisait des prédictions en utilisant **2 valeurs d'entrée**, ni plus ni moins.

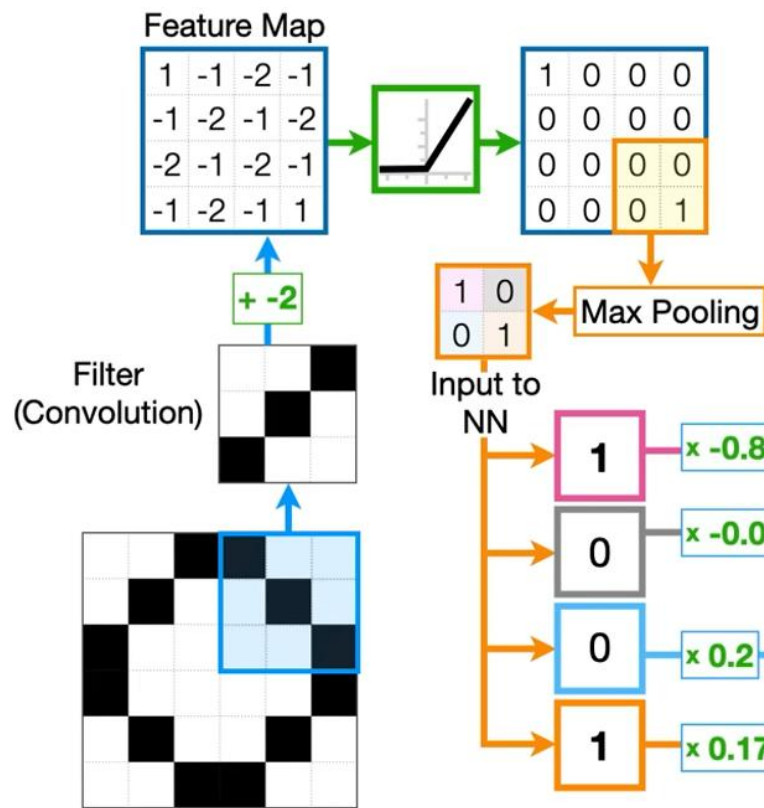
Exemple d'un réseau de neurones Feed-Forward avec deux entrées



EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

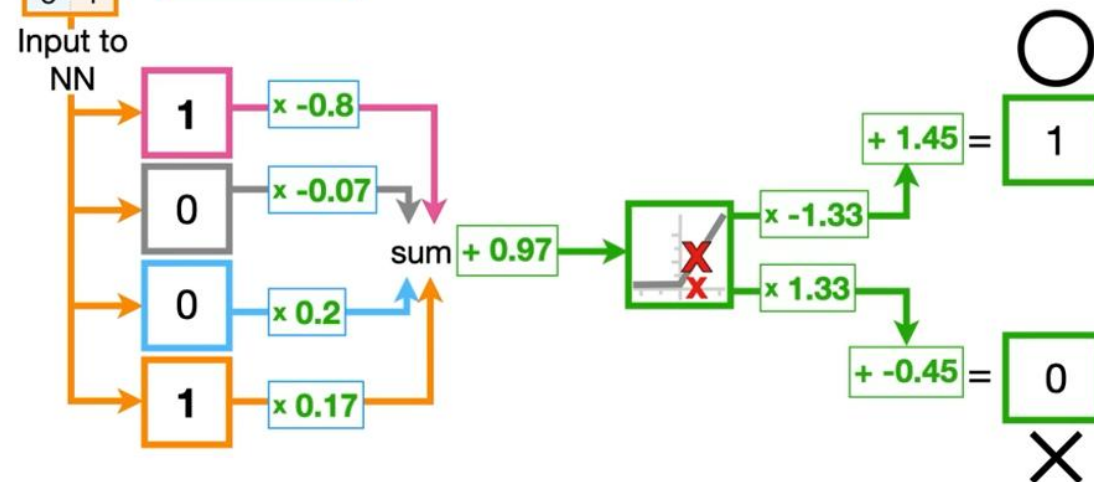
Exemple d'un CNN
avec une seule
image en entrée



Classification d'images

Réseau de neurones ayant effectué une prédiction à partir d'une image de **6 pixels sur 6**, pas plus grande et pas plus petite.

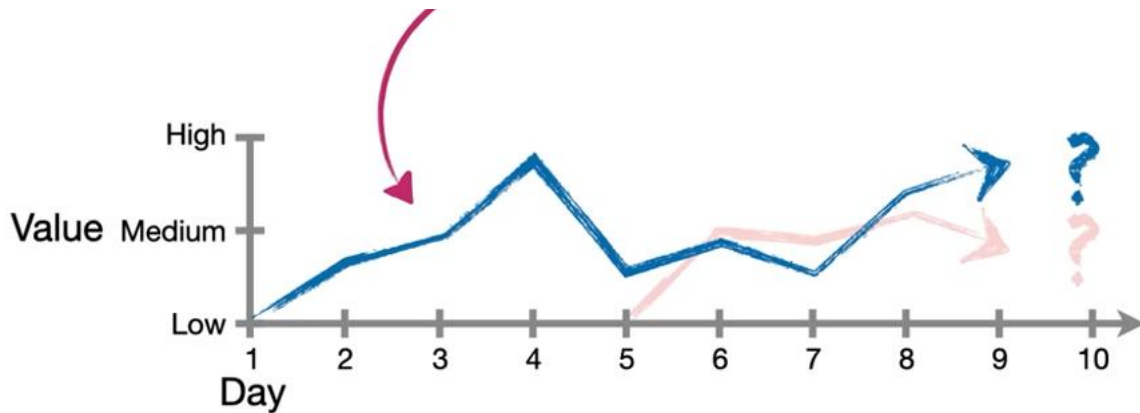
...avec des **Réseaux de Neurones Convolutifs (CNN) !!!**



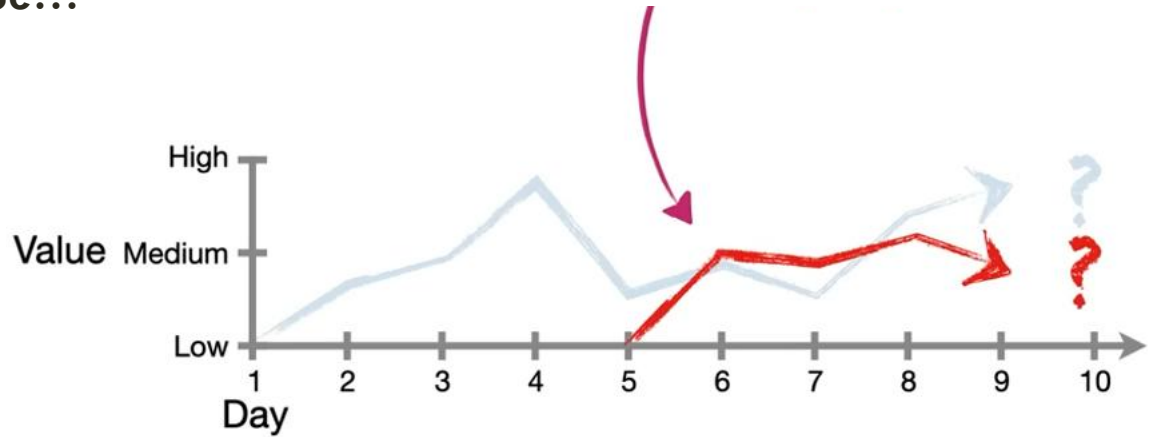
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

Cependant, nous avons maintenant besoin d'un **réseau de neurones** capable de faire une **prédiction à partir des 9 valeurs** que nous avons pour l'entreprise représentée par la ligne bleue...



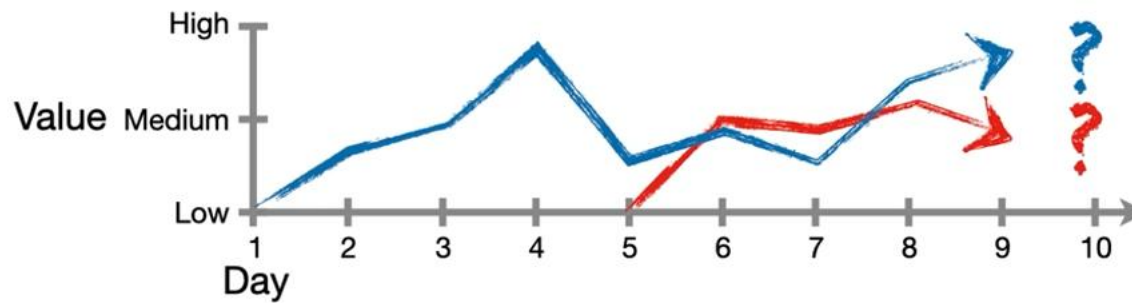
...et de faire une **prédiction à partir des 5 valeurs** que nous avons pour l'entreprise représentée par la ligne rouge.



EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

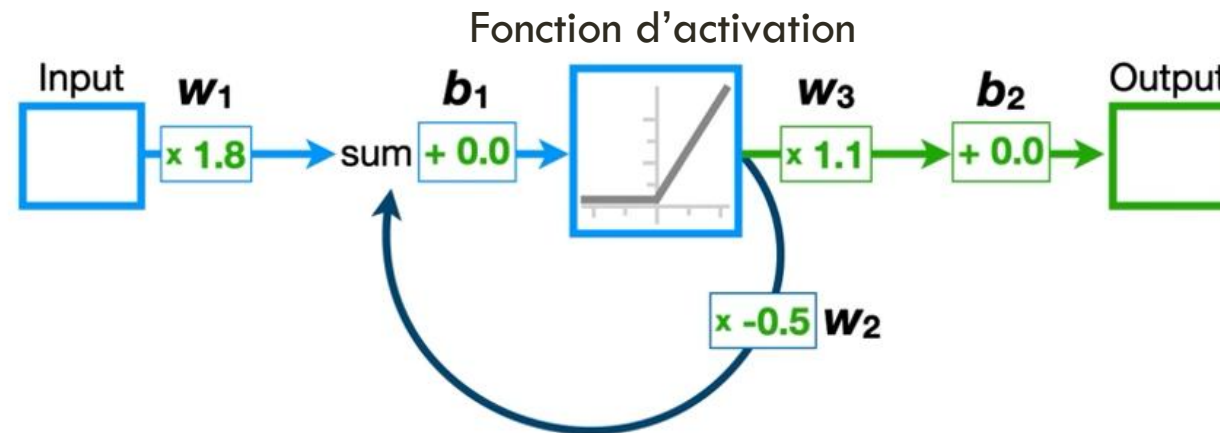
La bonne nouvelle est qu'une façon de gérer le **problème des quantités différentes de valeurs d'entrée** consiste à utiliser un **Réseau de Neurones Récurrent (RNN)**.



EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

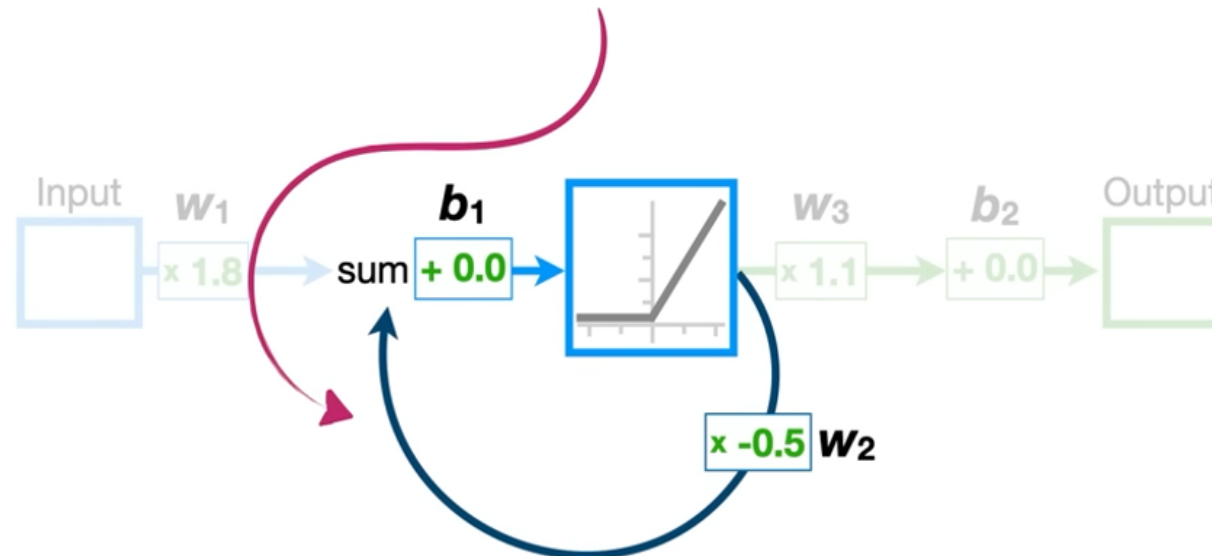
Tout comme les autres réseaux de neurones que nous avons déjà vus, les **Réseaux de Neurones Récurrents (RNN)** possèdent des **poids**, des **biais**, des **couches** et des **fonctions d'activation**.



EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

La grande différence est que les **Réseaux de Neurones Récurrents (RNN)** possèdent également des **boucles de rétroaction**.

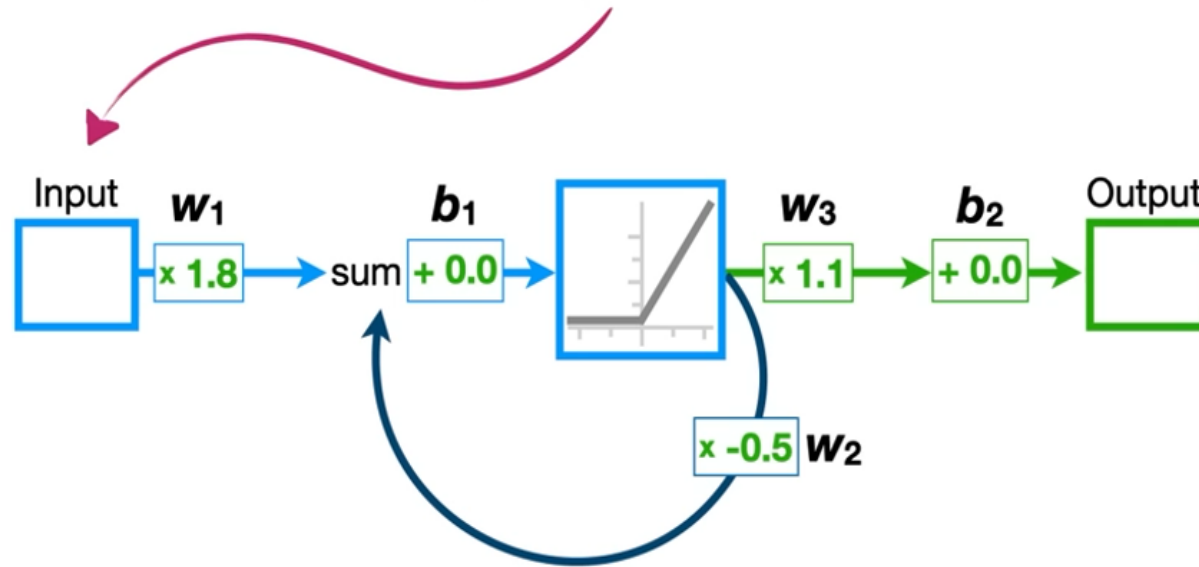


Boucle illustrant le retour du signal vers la somme précédente avec $w_2 \times -0.5$

EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

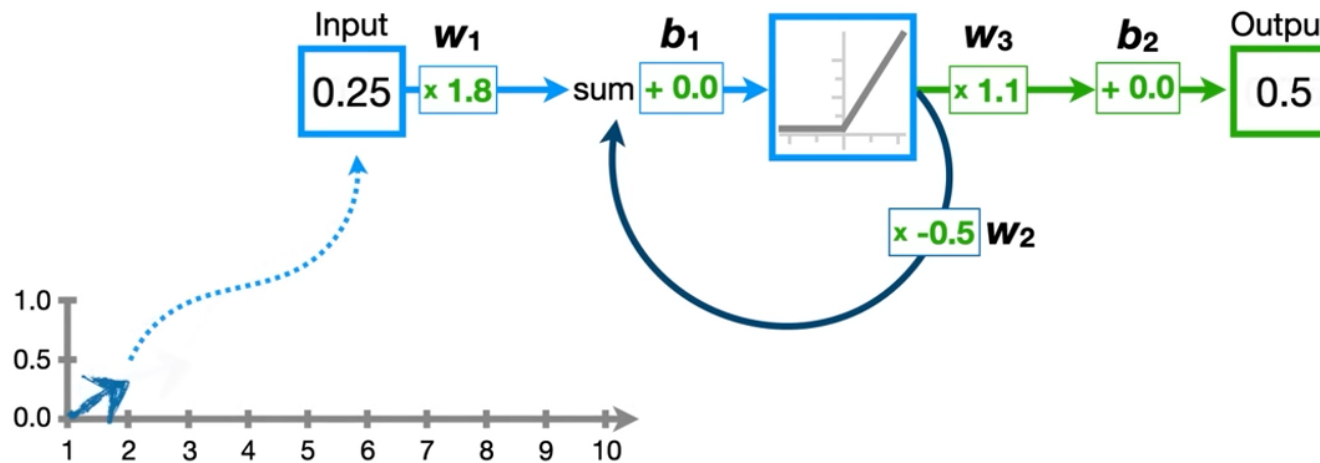
Et bien que ce **réseau de neurones** puisse sembler **ne prendre qu'une seule valeur en entrée...**



EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

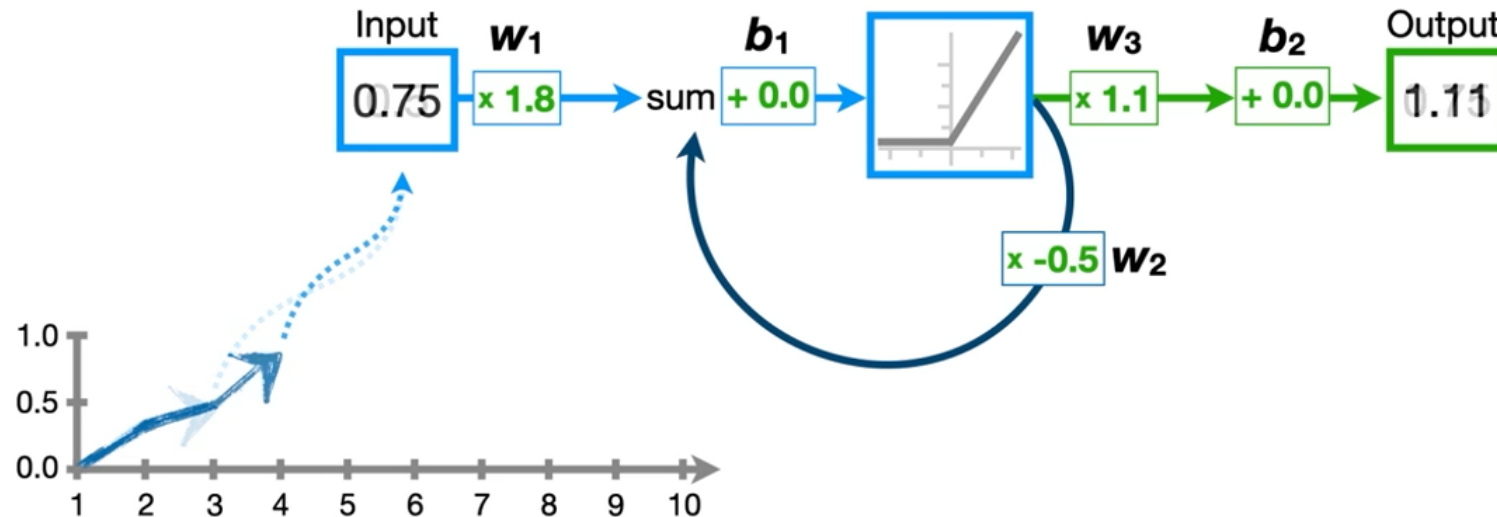
...la **boucle de rétroaction** permet d'utiliser des **valeurs d'entrée séquentielles**, comme les **prix du marché boursier collectés au fil du temps**, afin de **faire des prédictions**.



EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

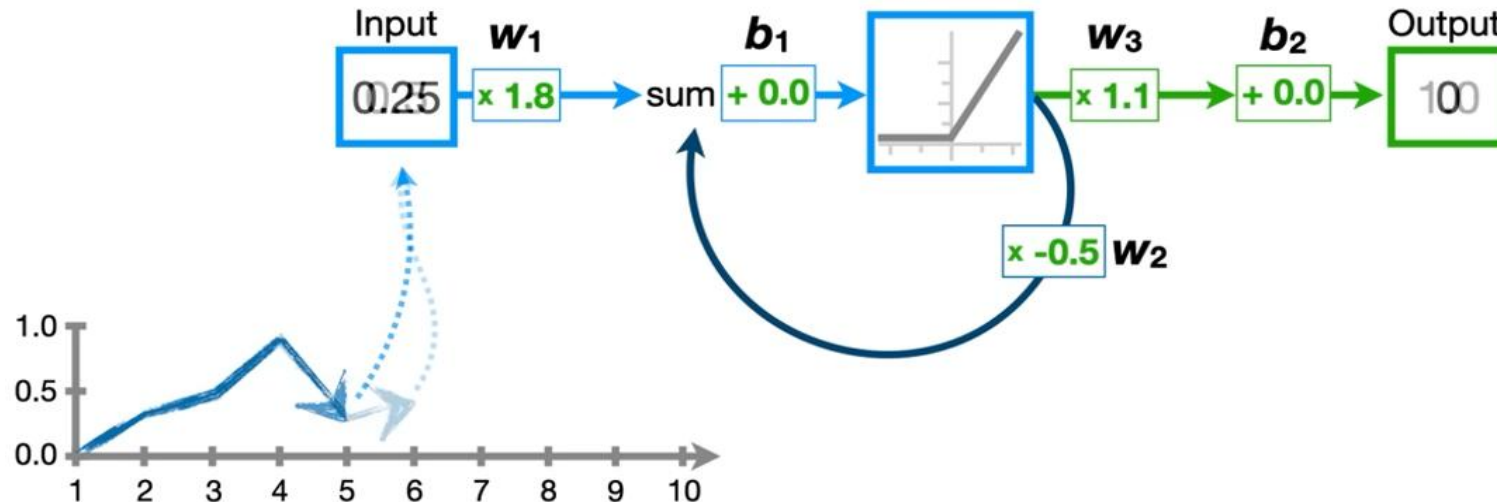
...la **boucle de rétroaction** permet d'utiliser des **valeurs d'entrée séquentielles**, comme les **prix du marché boursier collectés au fil du temps**, afin de **faire des prédictions**.



EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

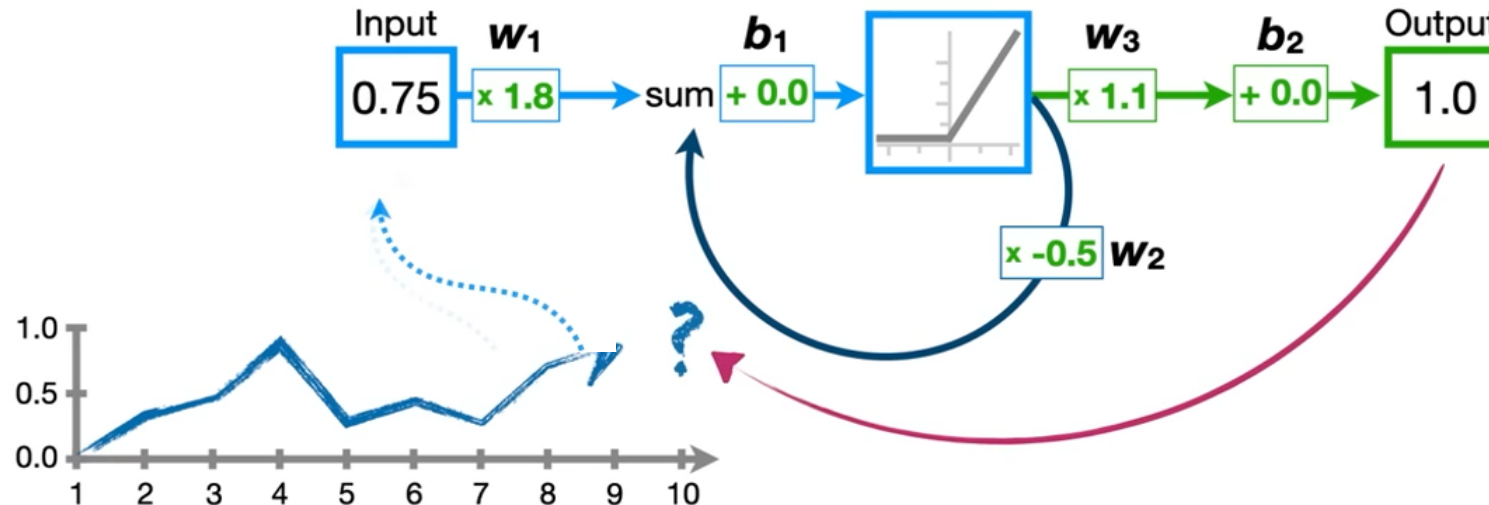
...la **boucle de rétroaction** permet d'utiliser des **valeurs d'entrée séquentielles**, comme les **prix du marché boursier collectés au fil du temps**, afin de **faire des prédictions**.



EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

...la **boucle de rétroaction** permet d'utiliser des **valeurs d'entrée séquentielles**, comme les **prix du marché boursier collectés au fil du temps**, afin de **faire des prédictions**.



EXEMPLE : MARCHÉ BOURSIER

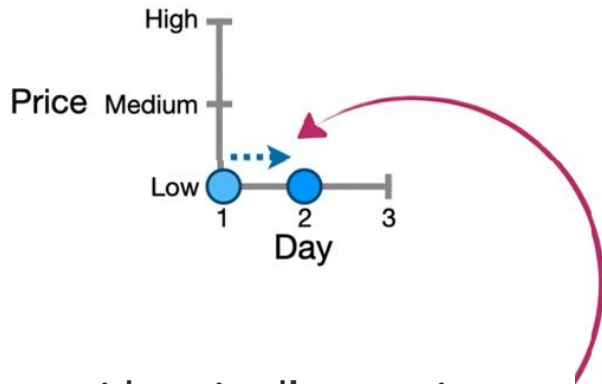
UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

Pour comprendre **comment**, exactement, ce **Réseau de Neurones Récurrent (RNN)** peut faire des **prédictions** à partir de valeurs d'entrée séquentielles...

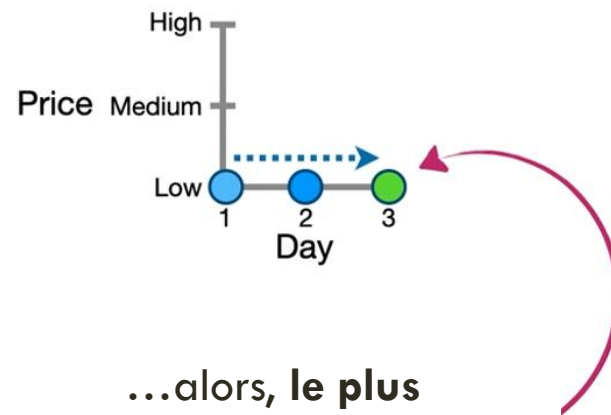
...faisons passer certaines des **données boursières** à travers ce réseau.

EXEMPLE : MARCHÉ BOURSIER

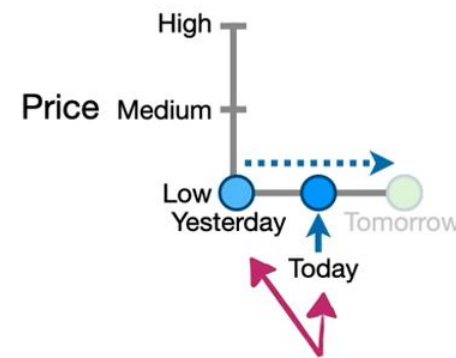
UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



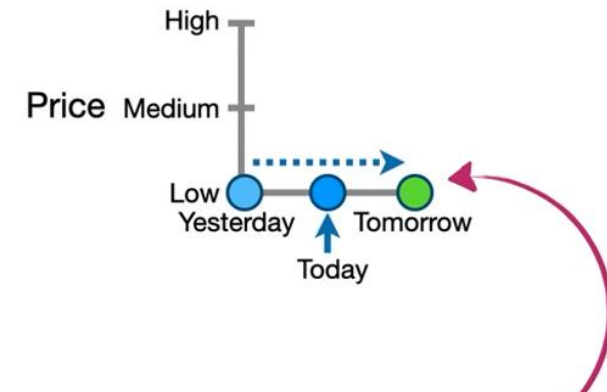
si le prix d'une action
reste **bas** pendant deux
jours consécutifs...



...alors, le **plus**
souvent, le prix reste
bas le jour suivant



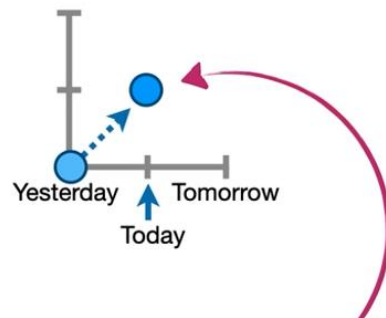
En d'autres termes, si le **prix**
de l'action d'hier et celui
d'aujourd'hui sont **bas**...



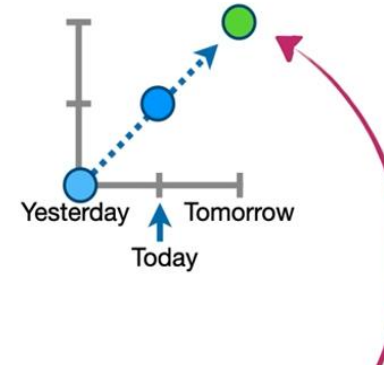
...alors le **prix de demain**
devrait également être **bas**

EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



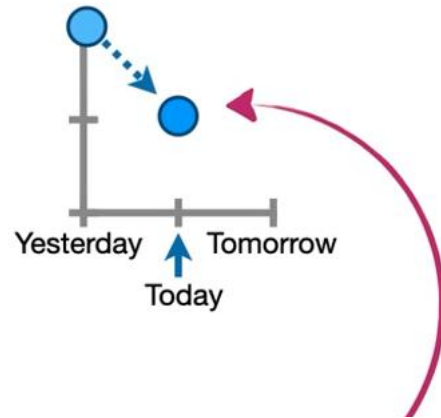
En revanche, si le **prix d'hier** était **bas** et que le **prix d'aujourd'hui** est **moyen**...



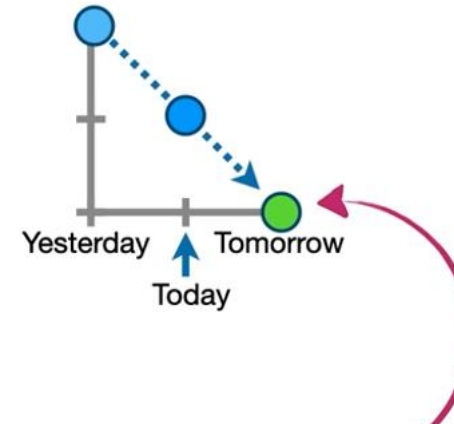
...alors le **prix de demain** devrait être **encore plus élevé**.

EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



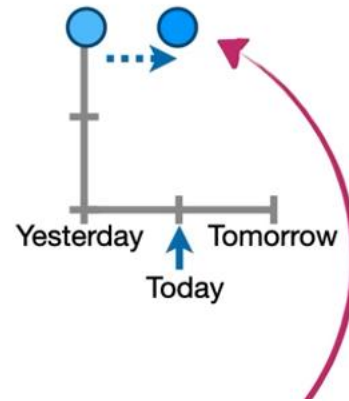
Et lorsque le **prix baisse**
de élevé à moyen...



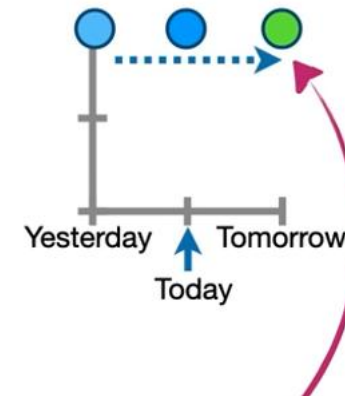
...alors le **prix de demain**
sera **encore plus bas**.

EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



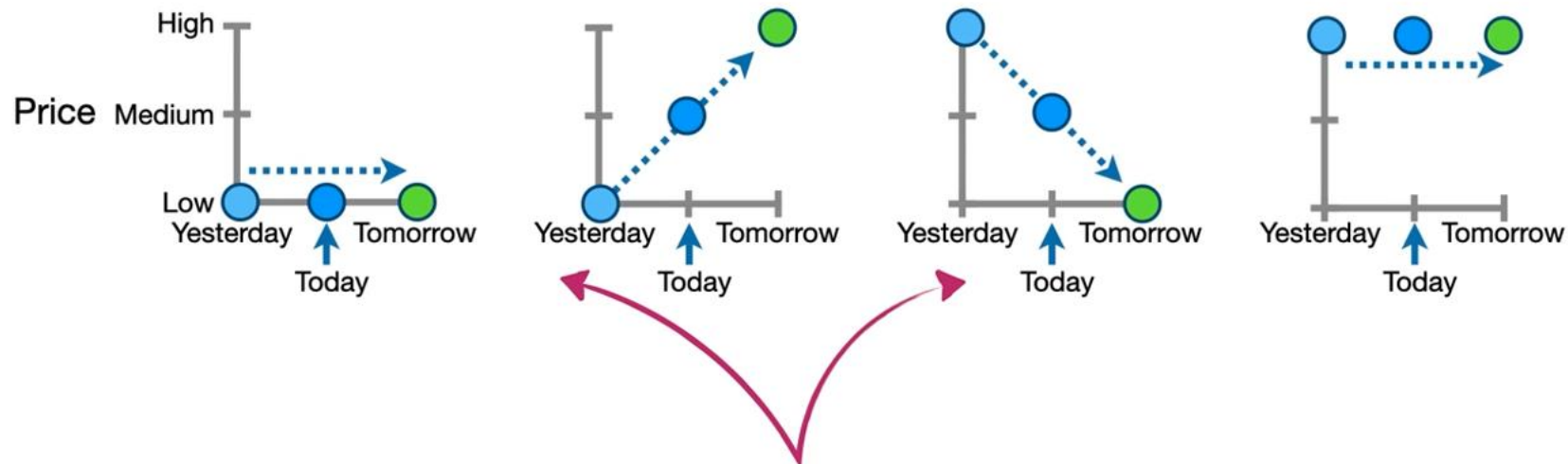
Enfin, si le **prix reste élevé pendant deux jours consécutifs...**



...alors le **prix sera élevé demain.**

EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

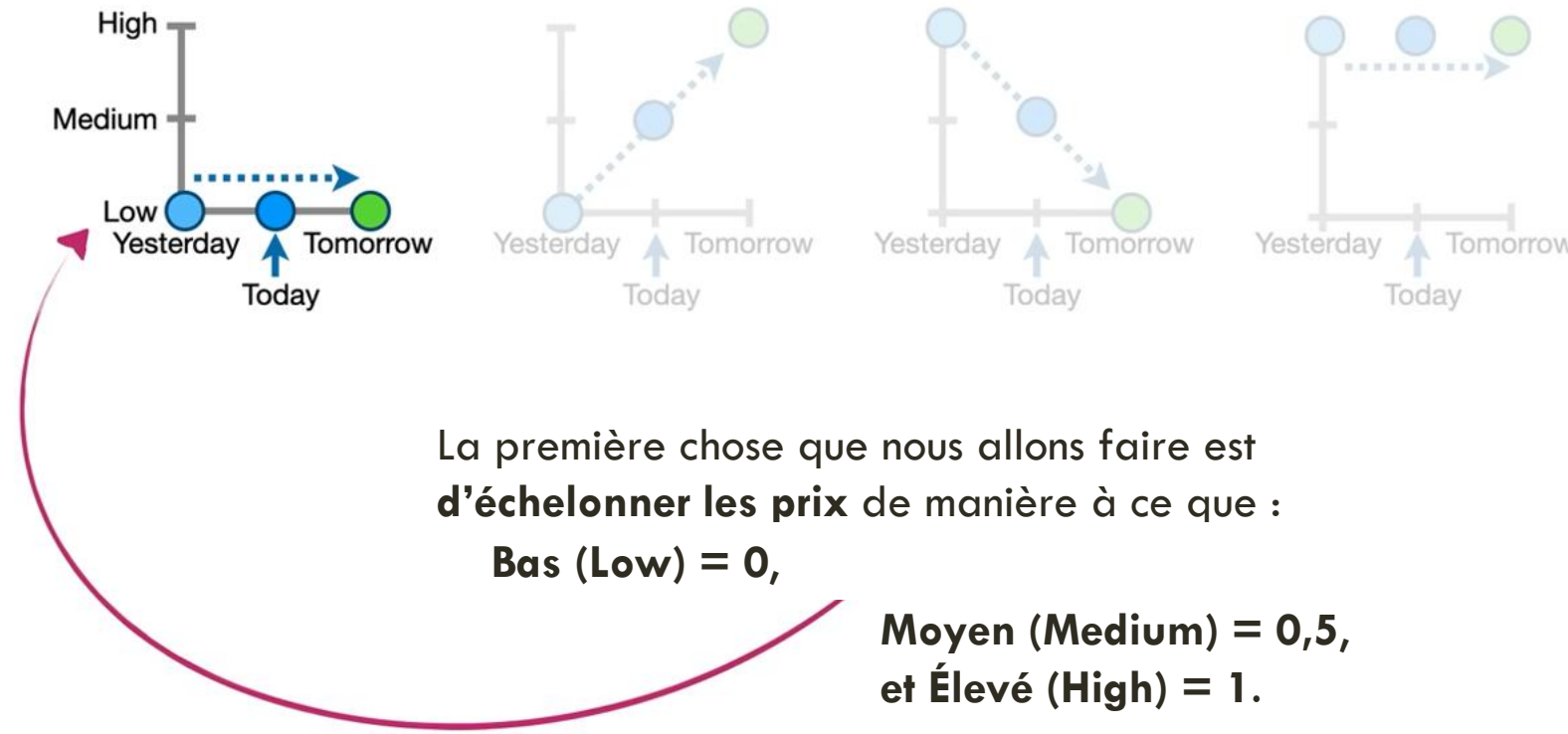


Maintenant que nous voyons
les **tendances générales**
des **prix des actions**

...nous pouvons expliquer comment utiliser les
données d'hier et d'aujourd'hui dans un **réseau**
neuronal récurrent (RNN) pour **prédire le prix de**
demain.

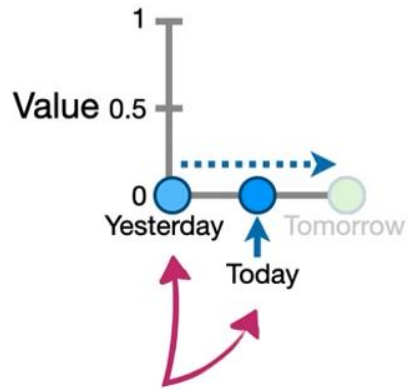
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



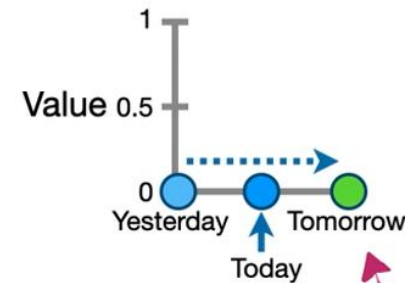
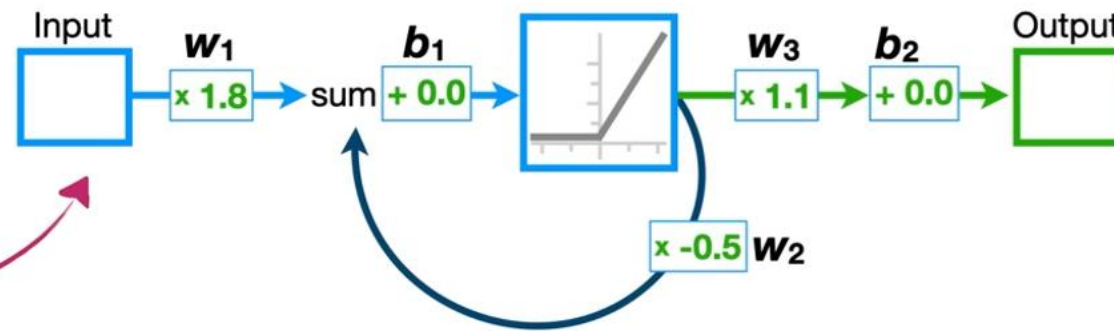
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



Passons maintenant
les **valeurs d'hier**
et **d'aujourd'hui**
dans le réseau...

...à travers ce
Réseau Neuronal
Récurrent (RNN)...

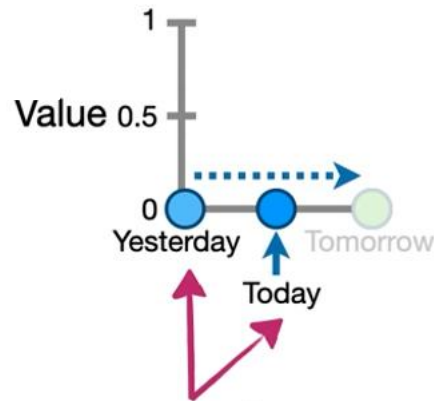


...et voyons s'il
peut **prédire**
correctement la
valeur de demain.

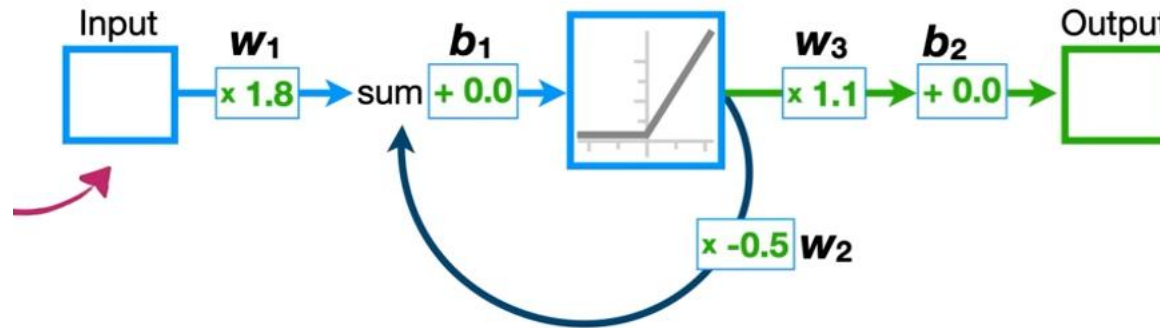
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

Maintenant, comme le réseau neuronal récurrent possède une **boucle de rétroaction**...

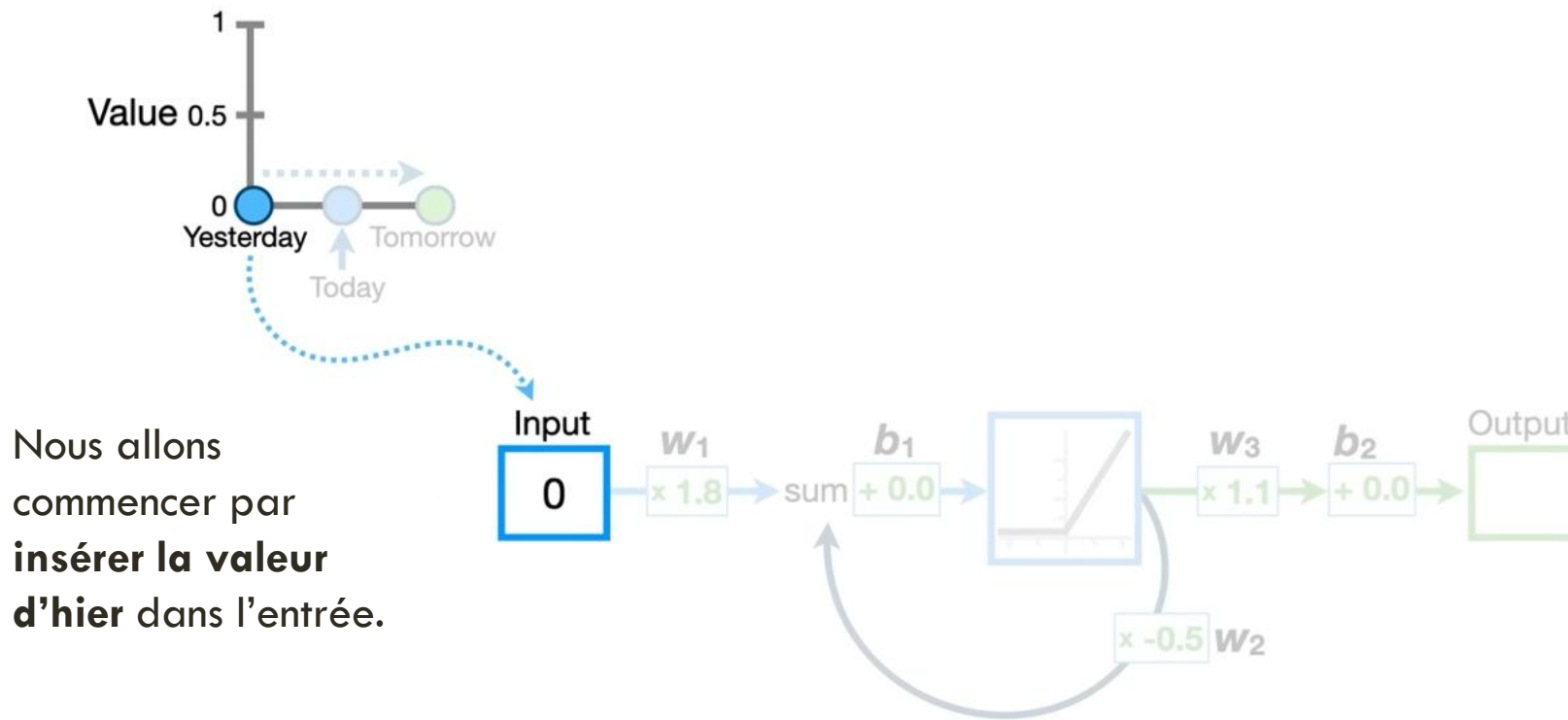


...nous pouvons saisir les **valeurs d'hier et d'aujourd'hui** dans l'entrée de manière séquentielle.



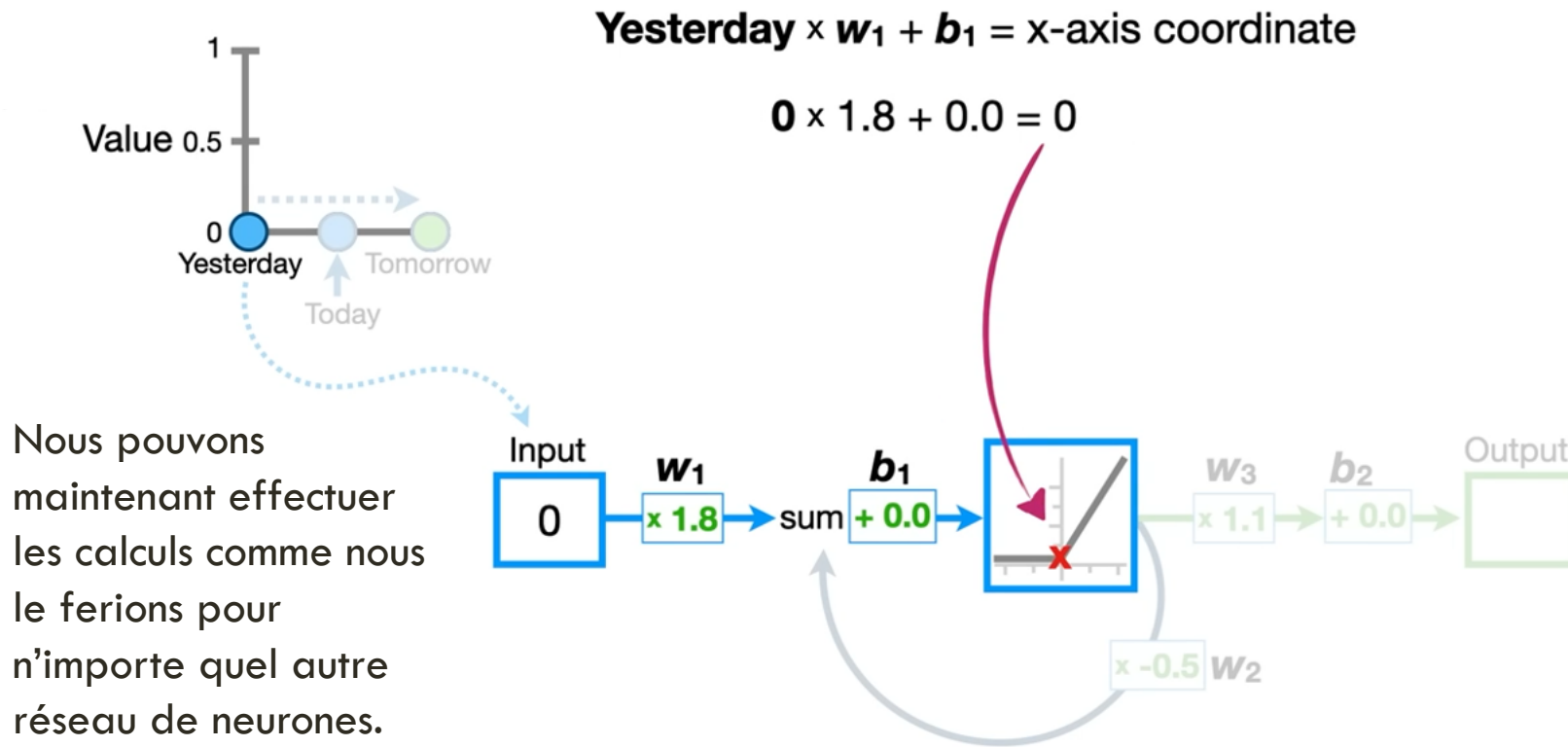
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



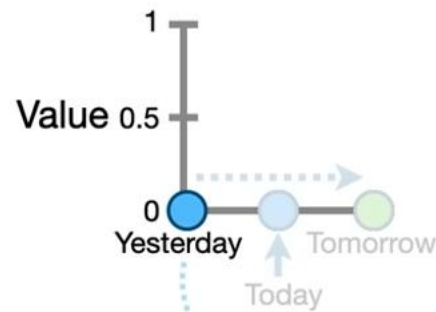
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



Nous pouvons maintenant effectuer les calculs comme nous le ferions pour n'importe quel autre réseau de neurones.

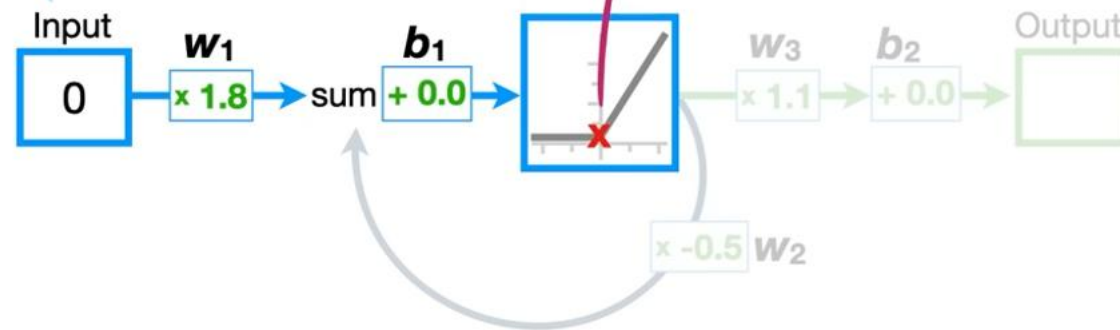
$$\text{Yesterday} \times w_1 + b_1 = \text{x-axis coordinate}$$

Linear combination

$$0 \times 1.8 + 0.0 = 0$$

$$f(x) = \max(0, x) = \text{y-axis coordinate}$$

ReLU (Rectifier activation function)



Remember:

Common activation functions:

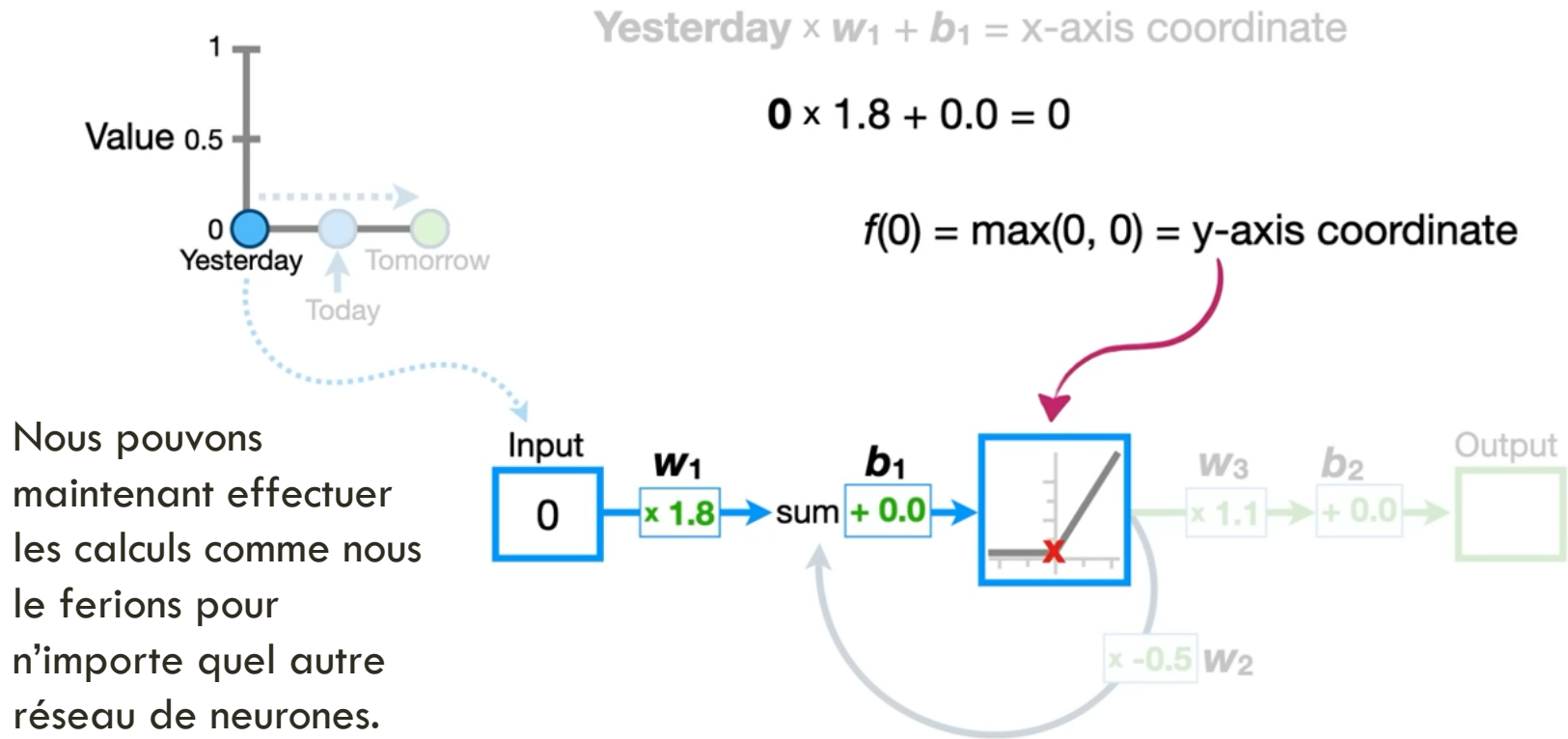
- ReLU: $f(x) = \max(0, x) \rightarrow$ used in the figure.
- Sigmoid: $f(x) = \frac{1}{1+e^{-x}}$ (maps outputs to range (0,1)).
- Tanh: $f(x) = \tanh(x)$ (maps outputs to range (-1,1)).
- Softmax: used for multi-class classification at the output layer.
- Linear (identity): $f(x) = x$, often used in regression.

Every neuron in a neural network does **two main steps**:

1. Weighted sum + bias: $z = \sum(w_i \cdot x_i) + b$
2. Apply non-linear activation: $y = f(z)$

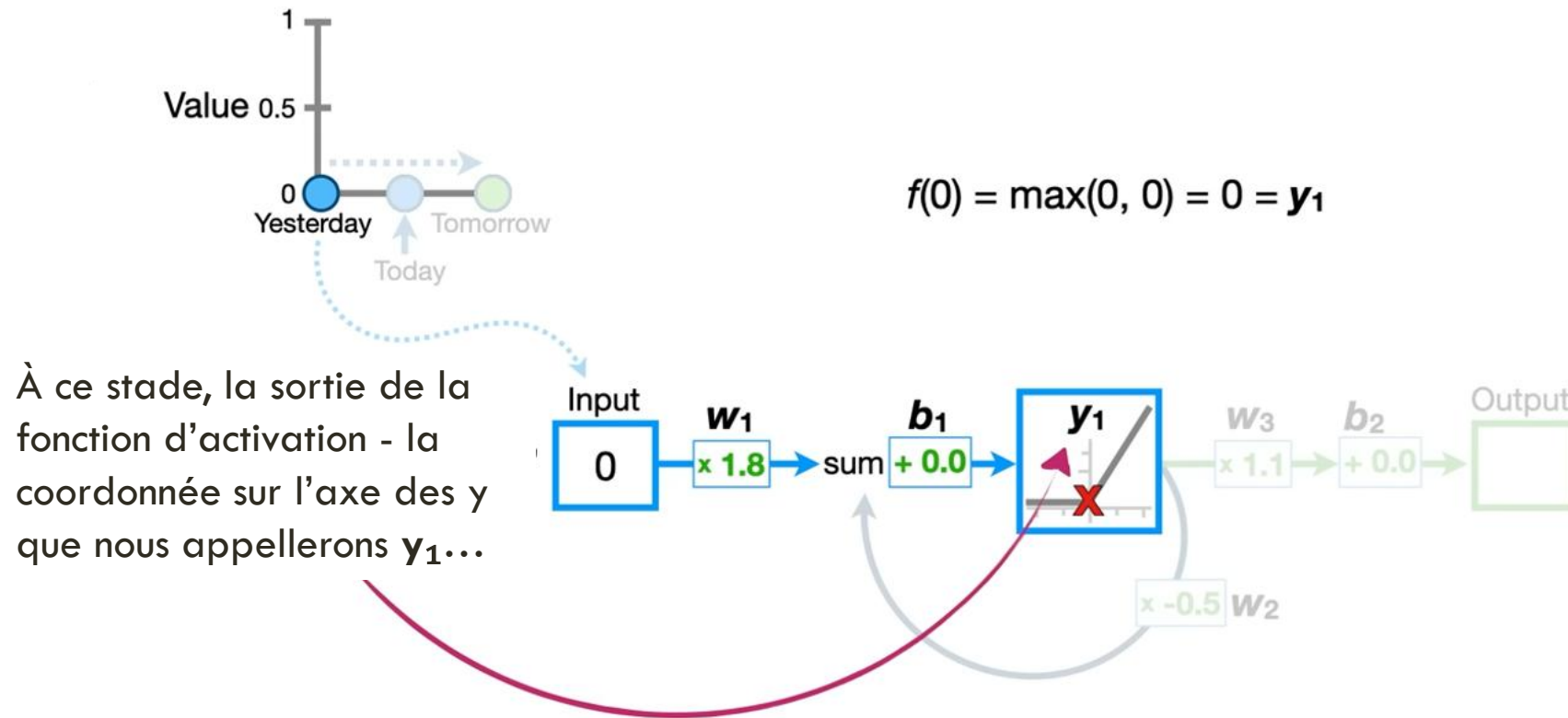
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



EXEMPLE : MARCHÉ BOURSIER

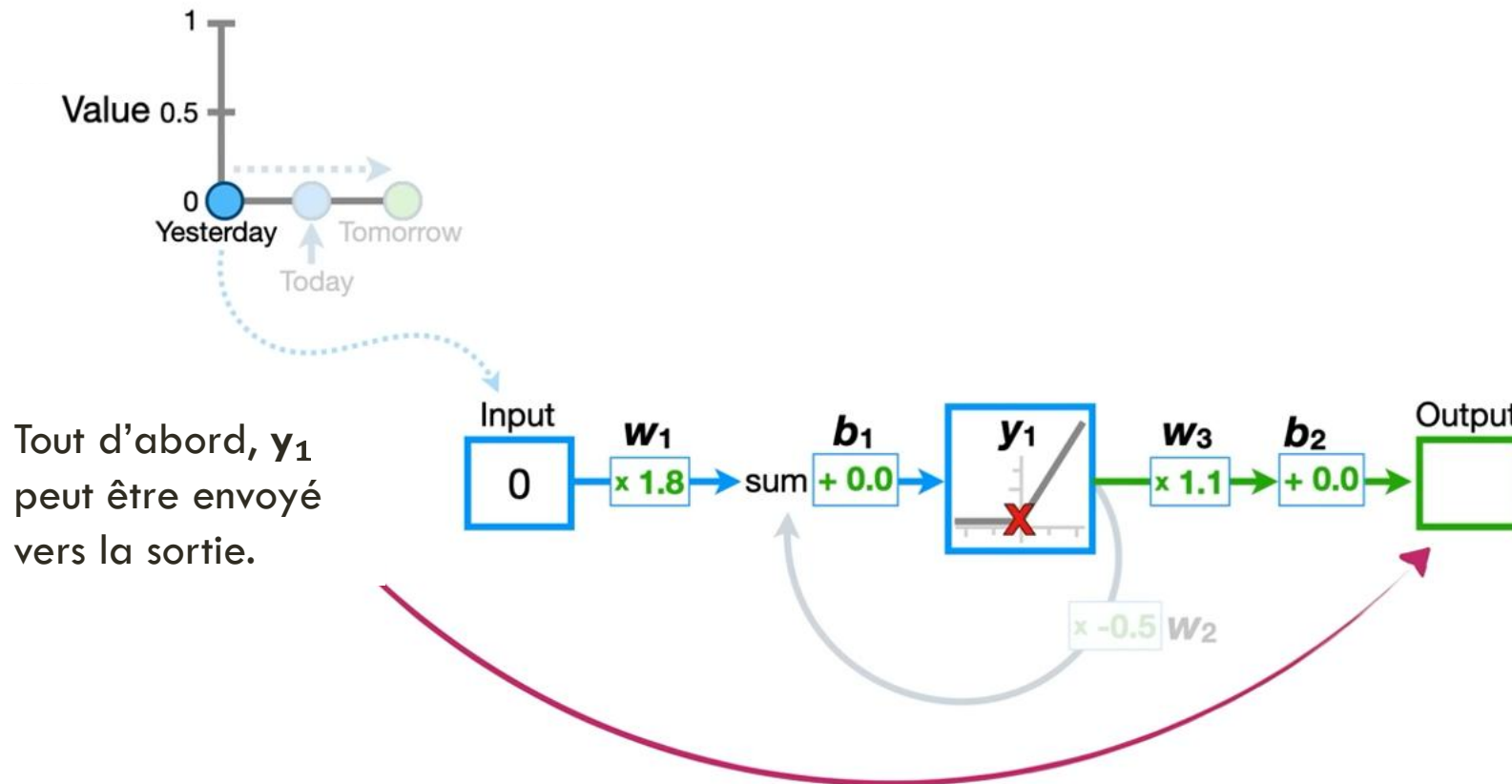
UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



À ce stade, la sortie de la fonction d'activation - la coordonnée sur l'axe des y que nous appellerons y_1 ...

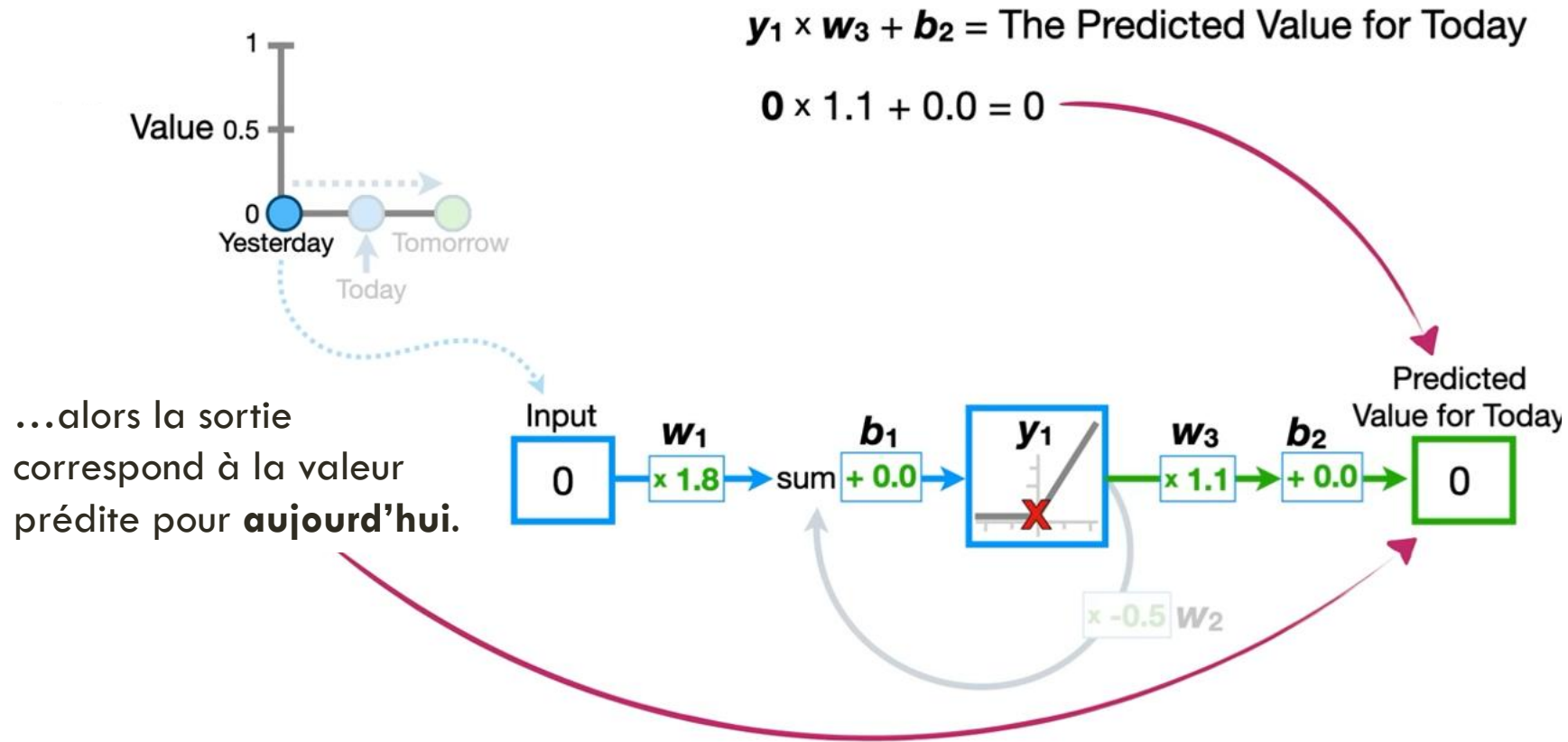
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



EXEMPLE : MARCHÉ BOURSIER

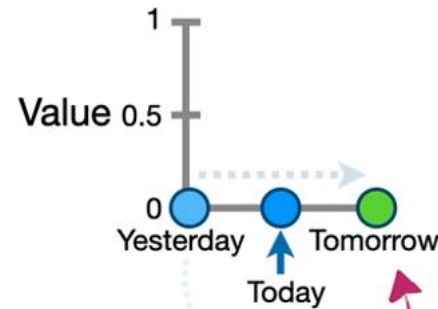
UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

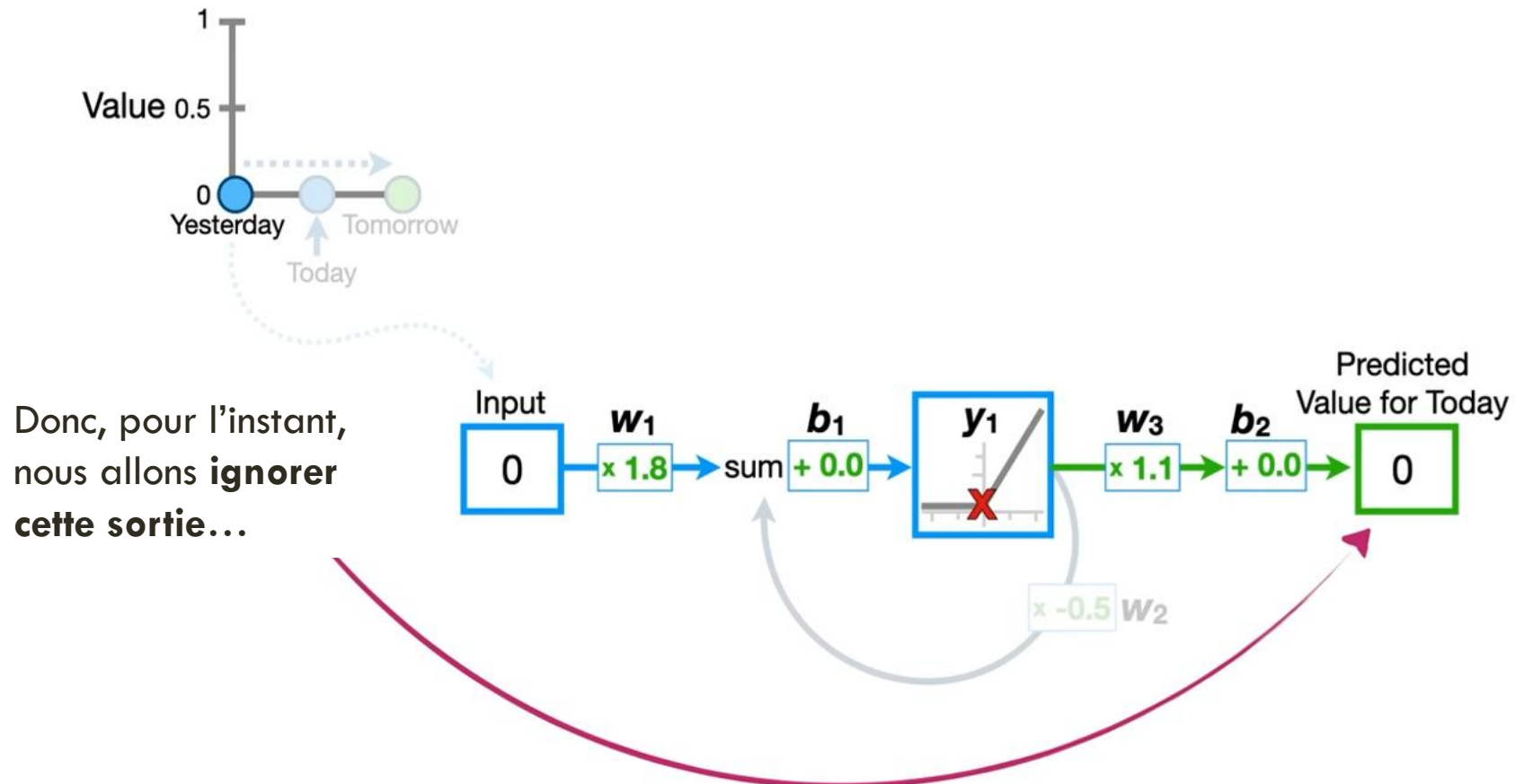
Cependant, nous ne nous intéressons pas à la **valeur prédite pour aujourd'hui**, car nous avons déjà la **valeur réelle pour aujourd'hui**.



Au lieu de cela, nous voulons utiliser à la fois la **valeur d'hier** et celle **d'aujourd'hui** pour prédire la **valeur de demain**.

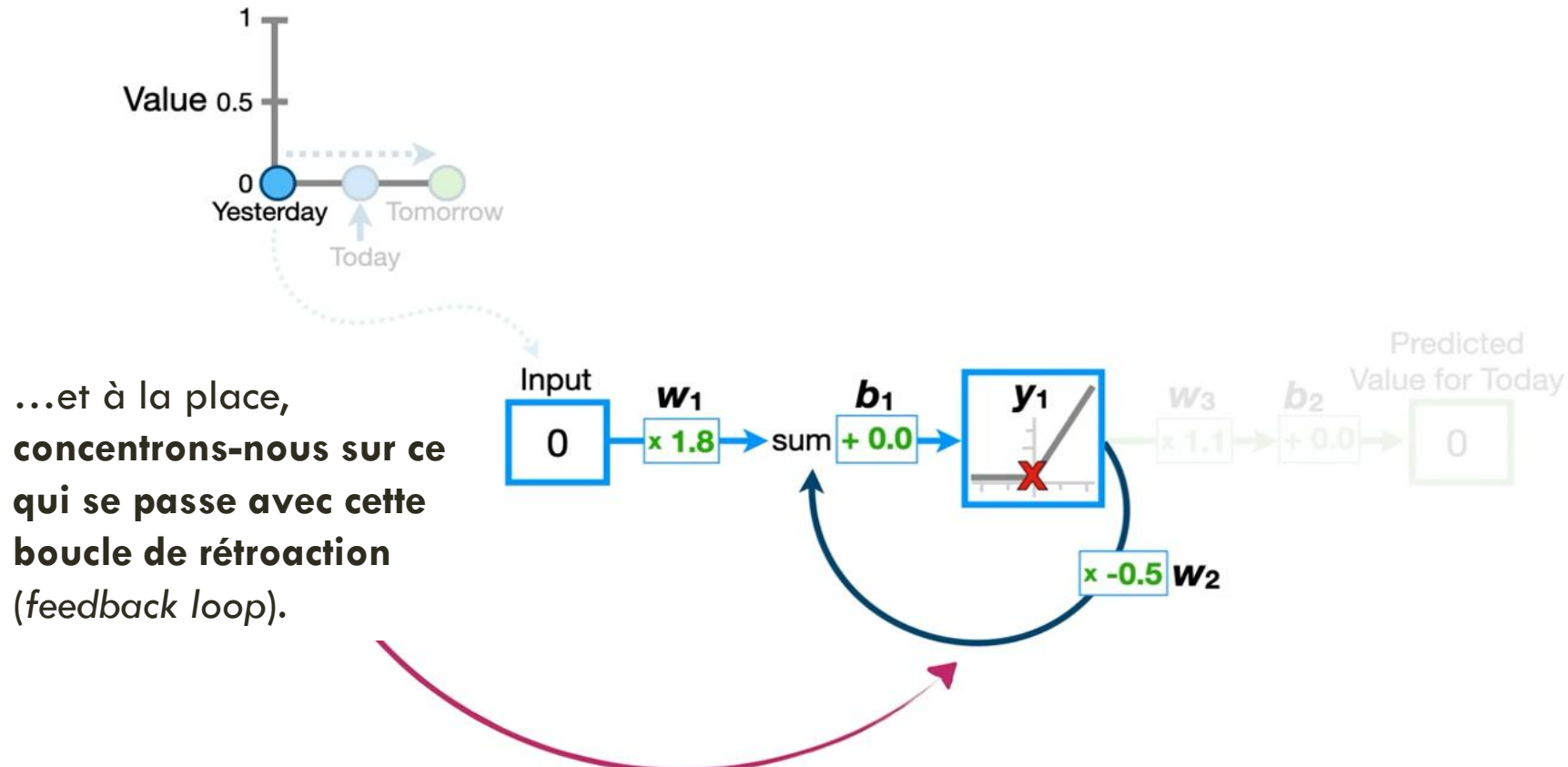
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



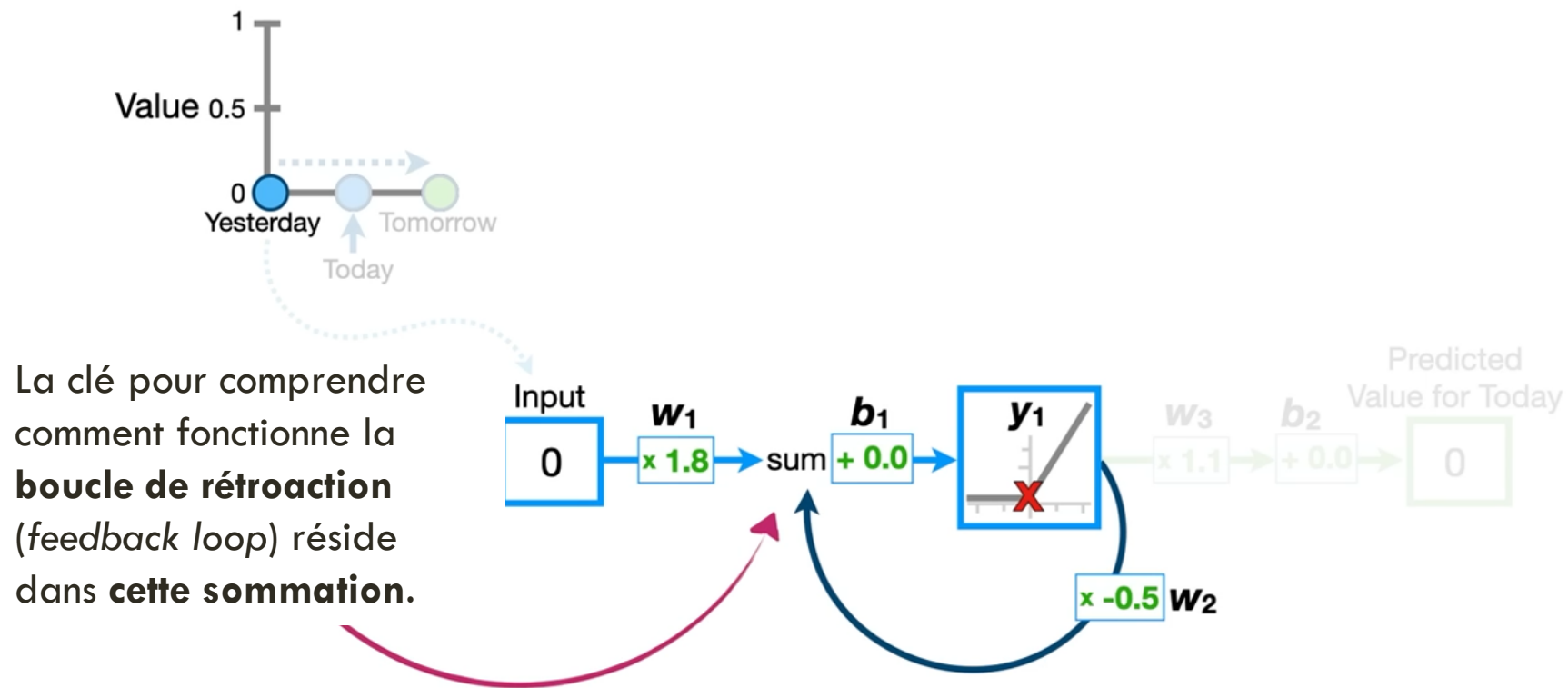
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



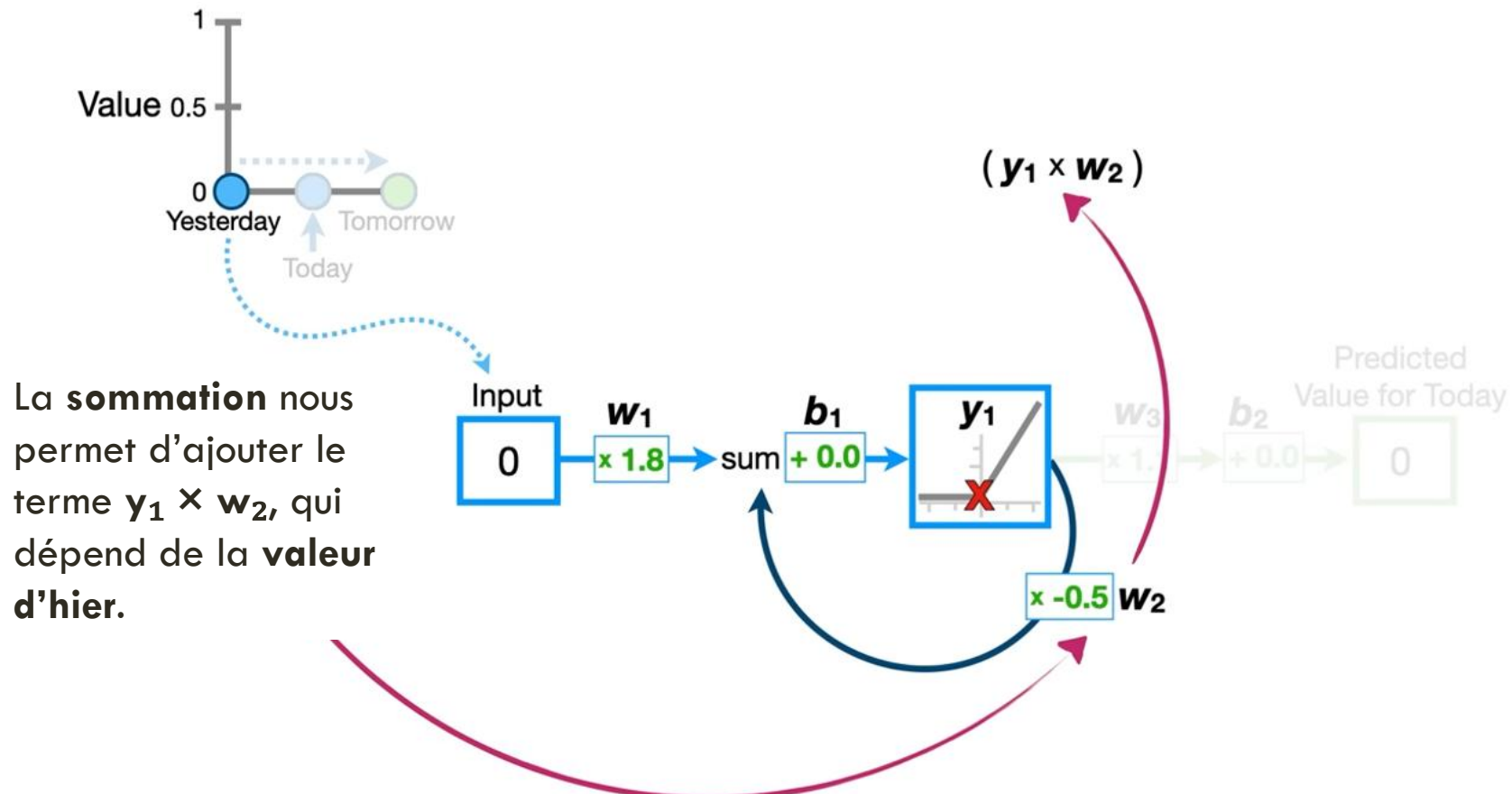
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



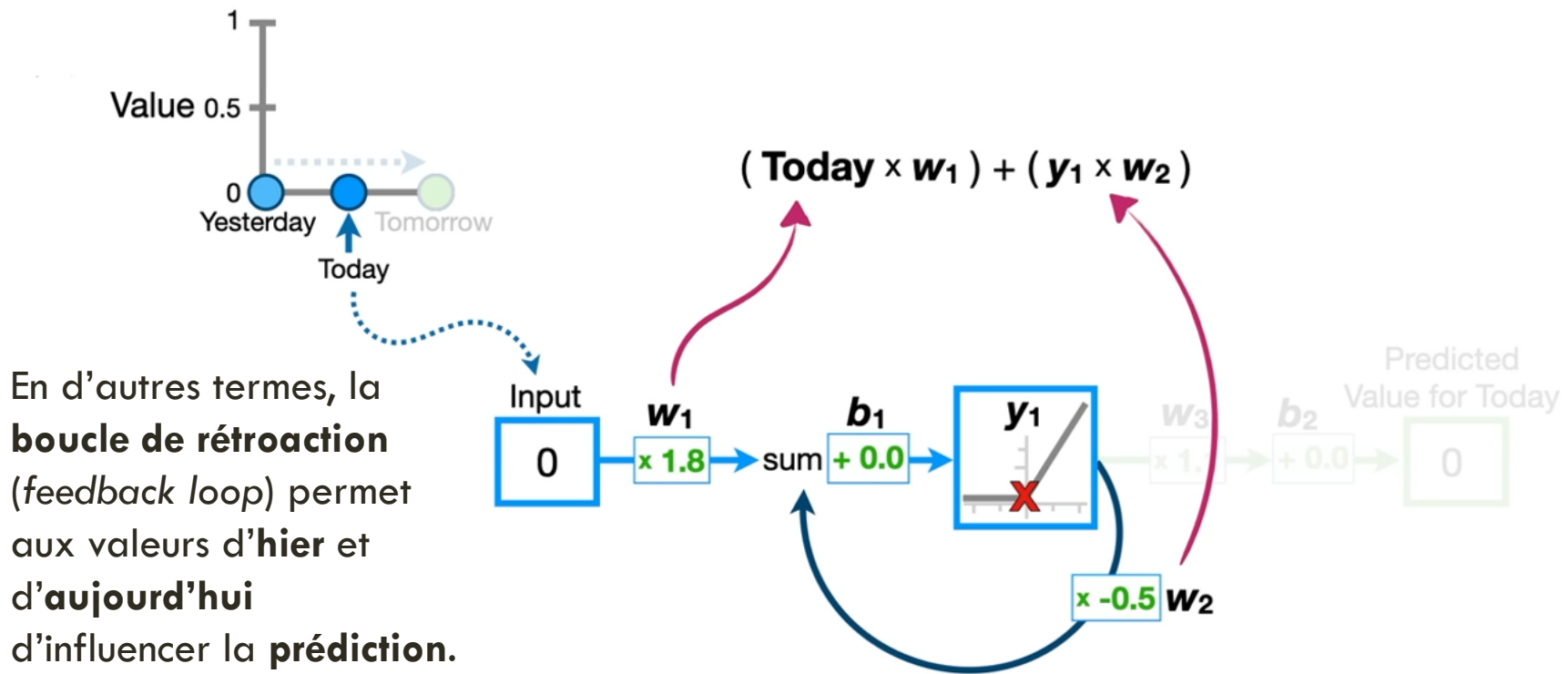
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



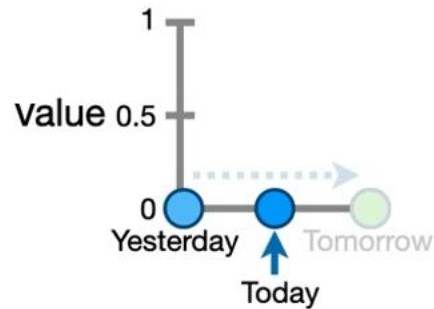
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

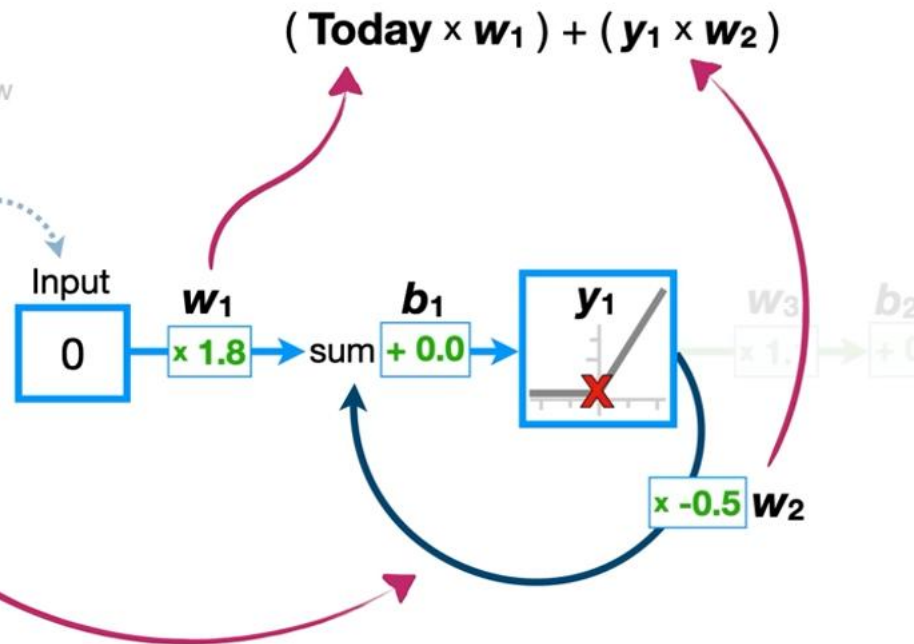


EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

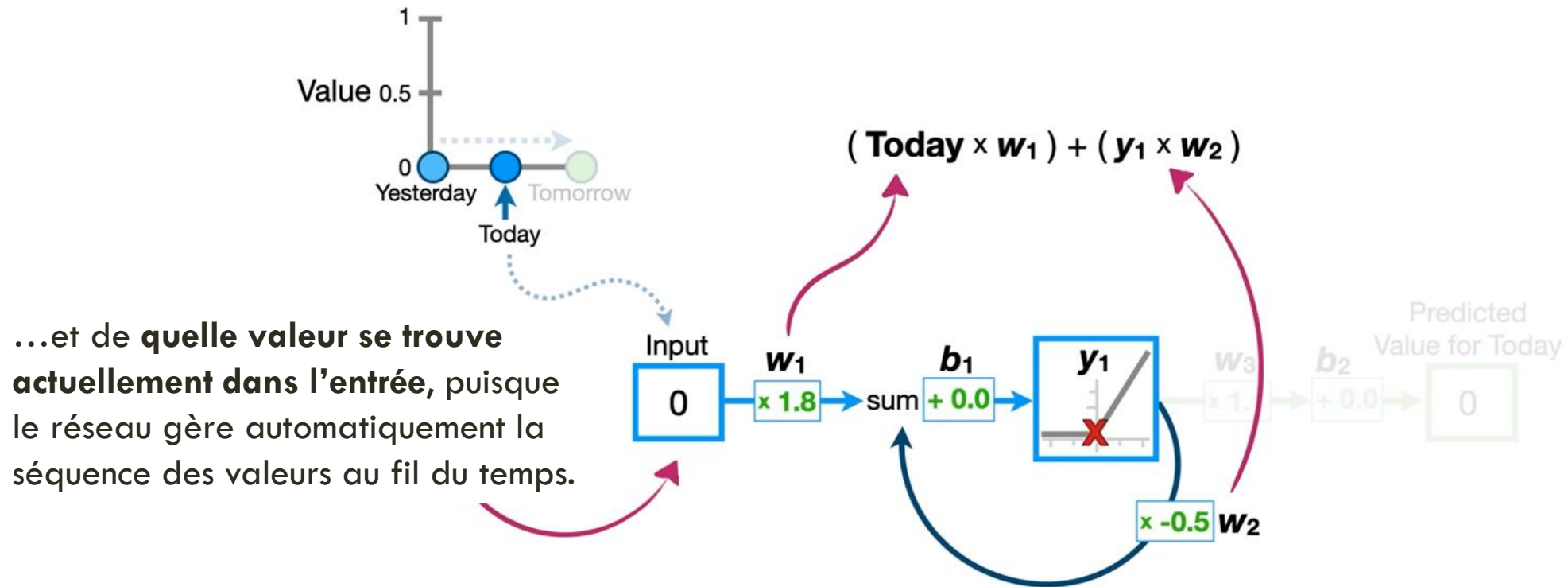


Au lieu d'avoir à se souvenir de **quelle valeur circule dans la boucle**, la structure récurrente permet de gérer automatiquement cette information.



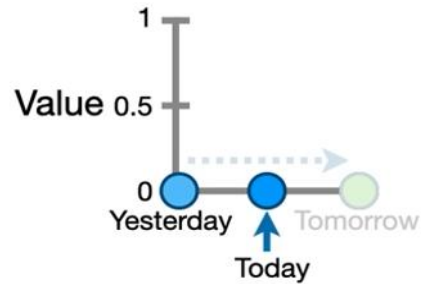
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

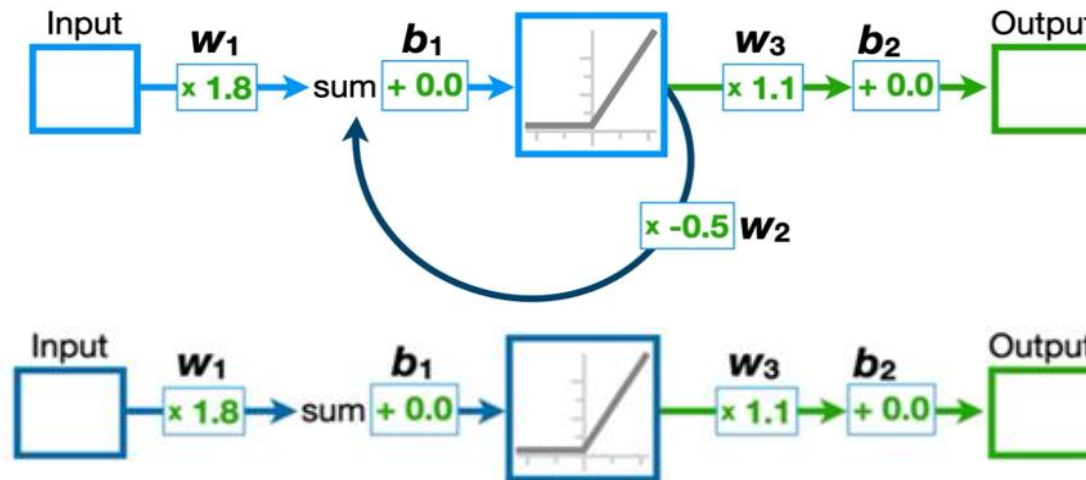


EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

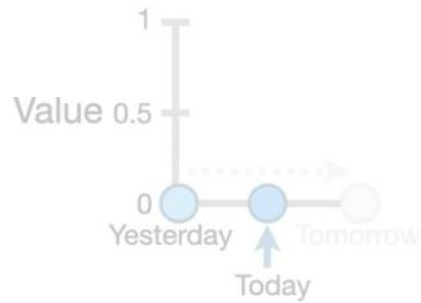


...nous pouvons **dérouler la boucle de rétroaction** (*unroll the feedback loop*) en **créant une copie du réseau de neurones pour chaque valeur d'entrée.**

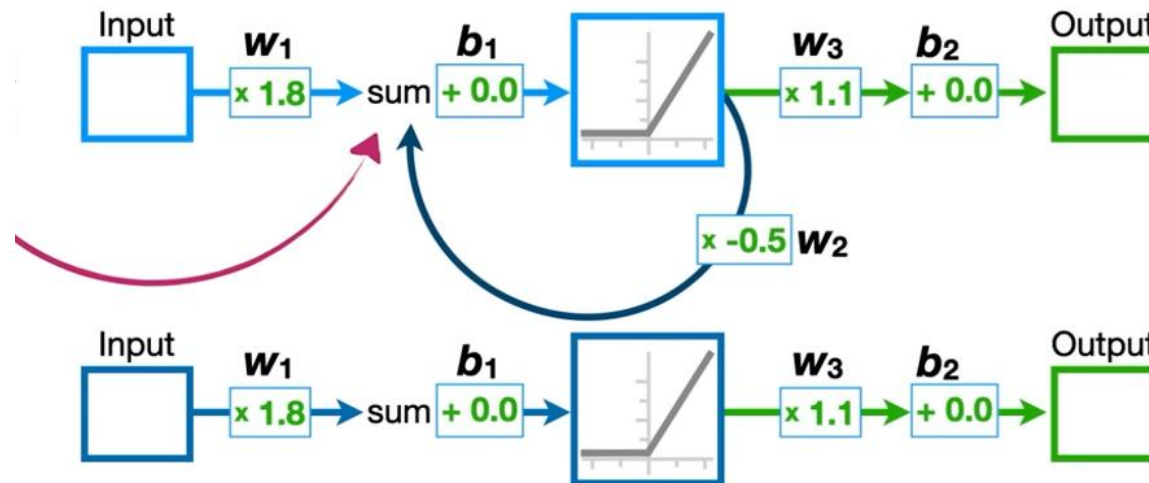


EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

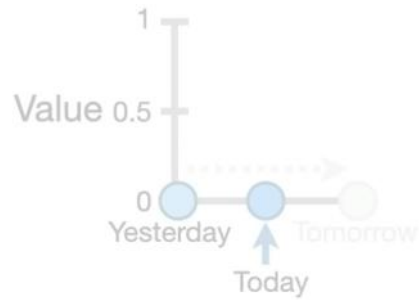


Maintenant, au lieu de faire pointer la **boucle de rétroaction** vers la somme dans la première copie du réseau...

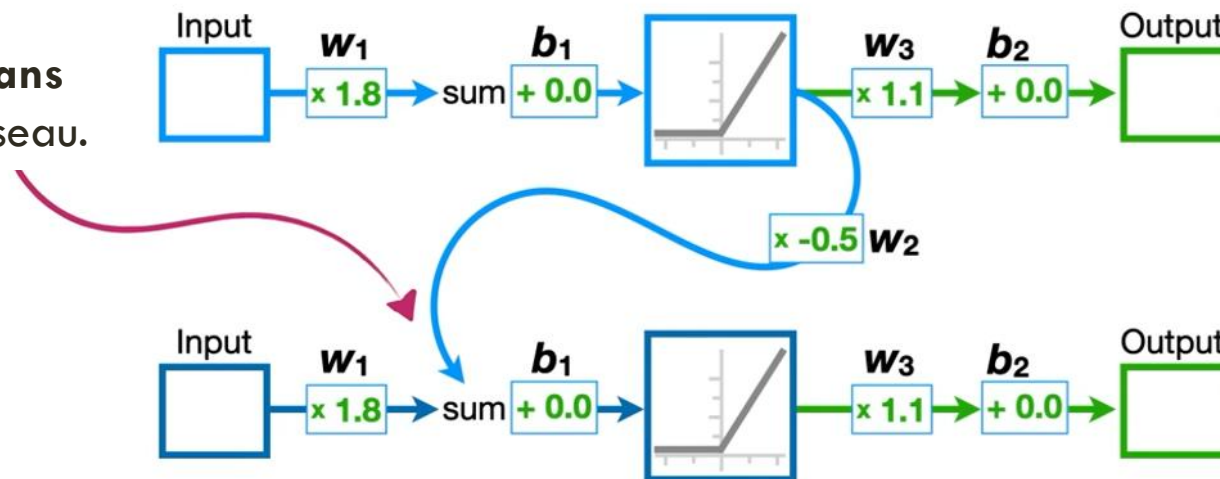


EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



...nous pouvons la **faire pointer vers la somme dans la deuxième copie** du réseau.

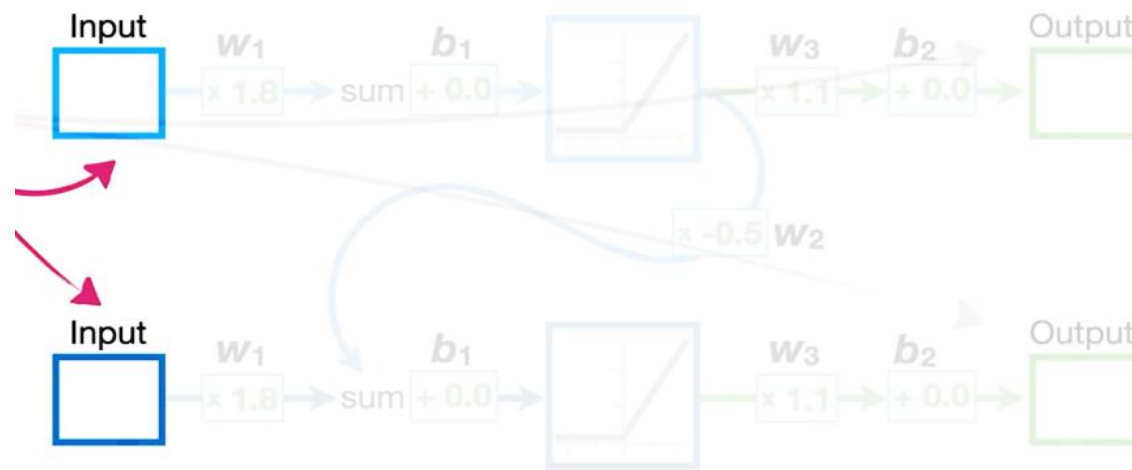


EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

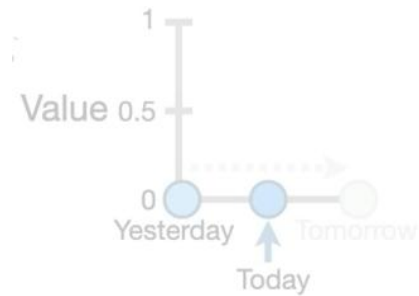


En **déroulant le réseau de neurones récurrent (RNN)**, nous obtenons un **nouveau réseau possédant deux entrées.**

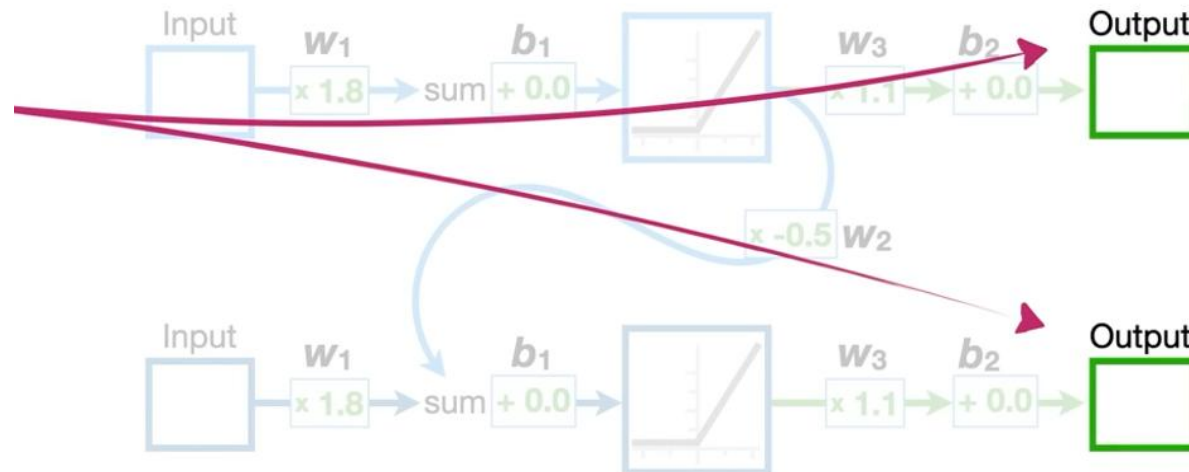


EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

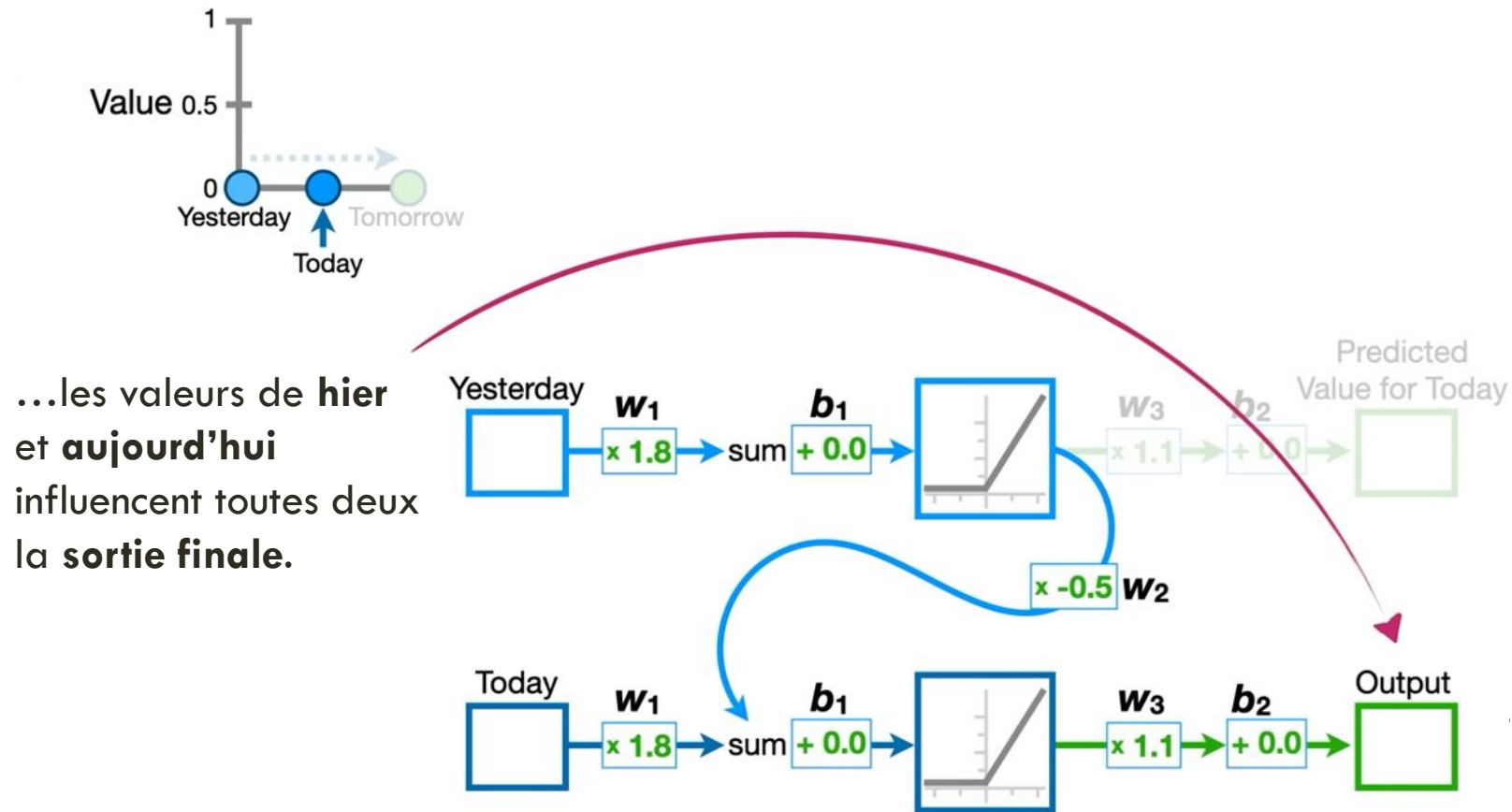


...et deux sorties.



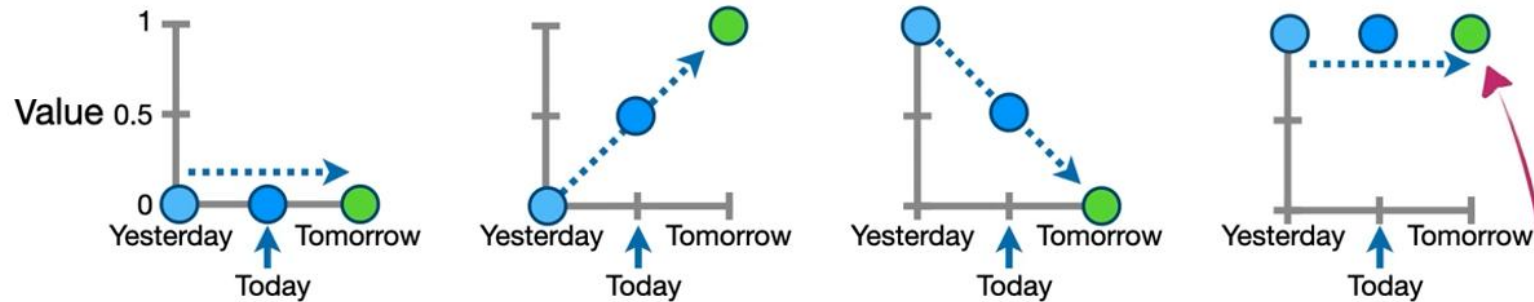
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

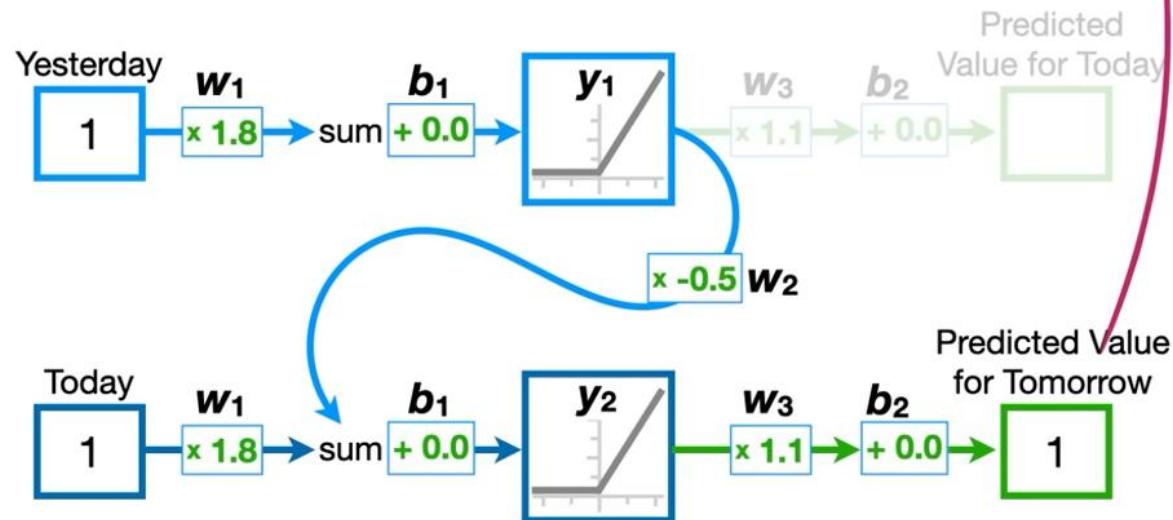


EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

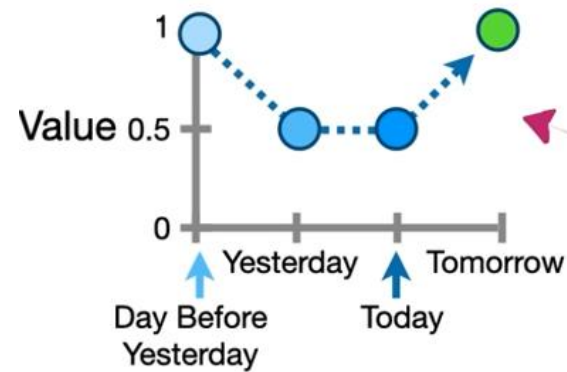


De même, lorsque nous faisons passer les valeurs de **hier** et **aujourd'hui** dans le réseau de neurones récurrent pour les autres scénarios, nous pouvons **prédire correctement les valeurs de demain**.



EXEMPLE : MARCHÉ BOURSIER

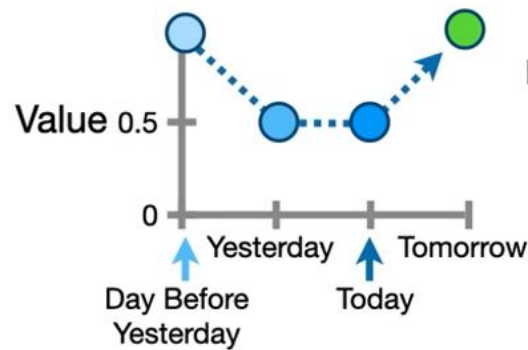
UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



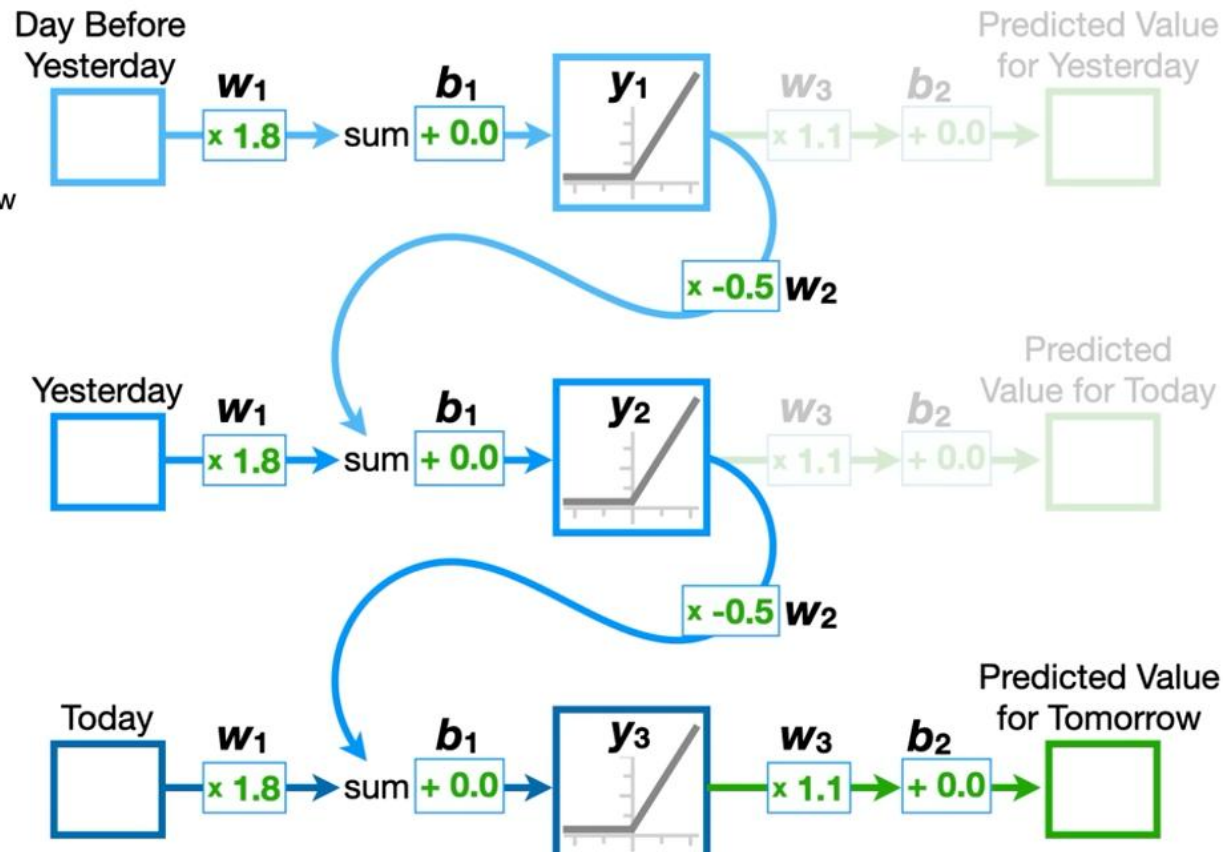
Lorsque nous voulons
utiliser **3 jours de
données** pour faire
une **prédiction sur la
valeur de demain,**
comme ceci...

EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



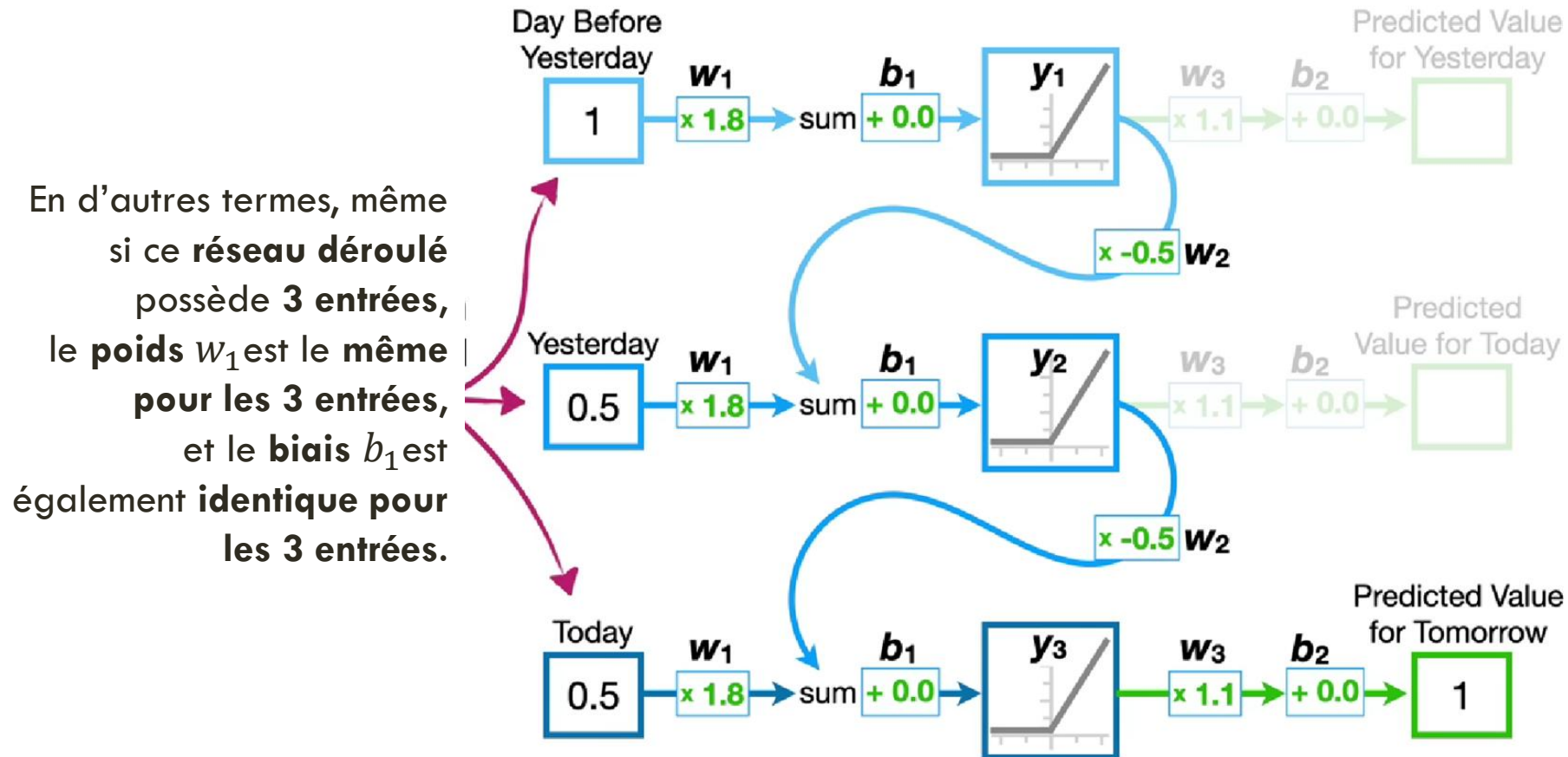
...nous entrons les valeurs dans les entrées, toujours de la plus ancienne à la plus récente.



Remarque : Peu importe combien de fois nous déroulons un réseau de neurones récurrent, **les poids et les biais sont partagés entre toutes les entrées.**

EXEMPLE : MARCHÉ BOURSIER

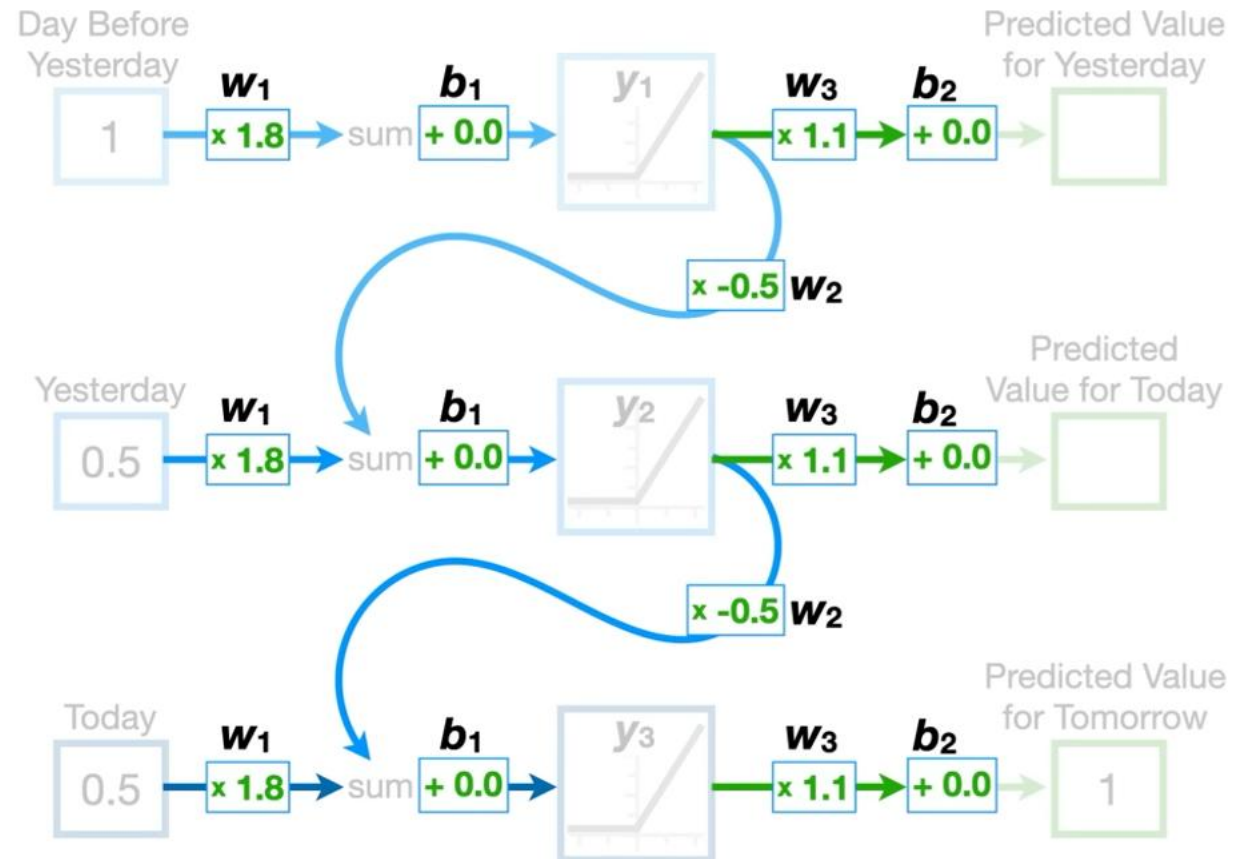
UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

De même, **tous les autres poids et biais** sont **partagés** entre les différentes étapes du réseau.



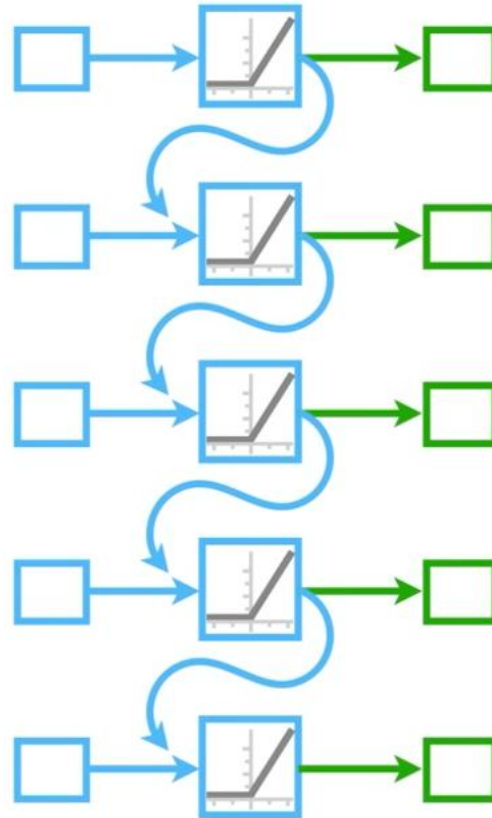
Ainsi, peu importe le nombre de fois que nous **déroulons un réseau de neurones récurrent (RNN)**, nous **n'augmentons jamais le nombre de poids et de biais à entraîner**.

EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

Pourquoi les RNN ne sont-ils pas très souvent utilisés ?

Un grand problème est que **plus on déroule un RNN, plus il devient difficile à entraîner.**

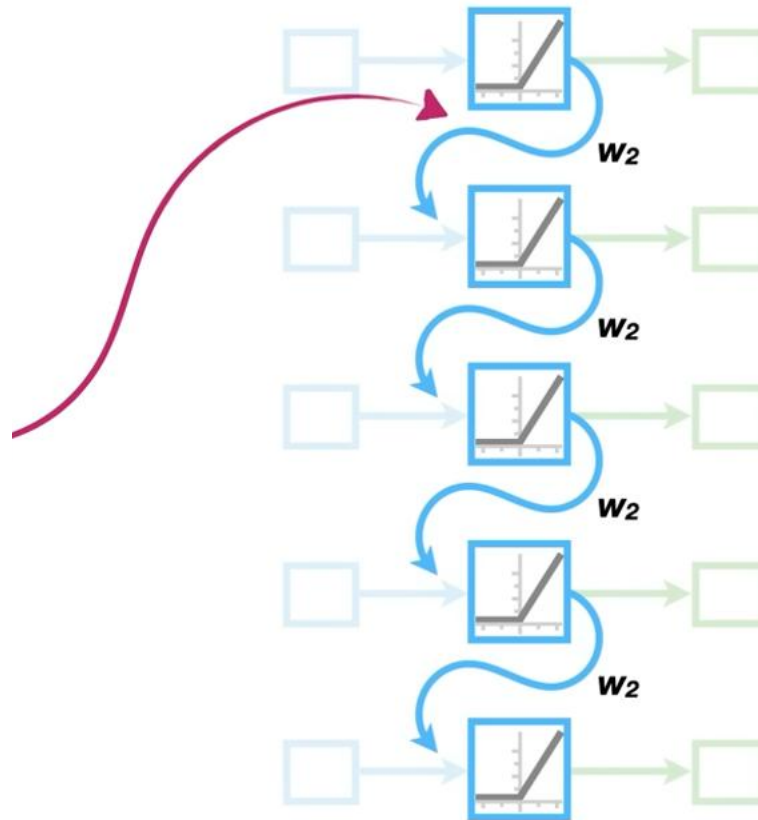


Ce problème est appelé le **problème du gradient qui disparaît ou explose (Vanishing / Exploding Gradient Problem).**

EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

Dans notre exemple, le **problème du gradient qui disparaît ou explose** est lié au **poids le long de la boucle** que nous copions à chaque fois que nous déroulons le réseau.



Remarque : Pour faciliter la compréhension du **problème du gradient qui disparaît ou explose**, nous allons **ignorer les autres poids et biais** du réseau et **nous concentrer uniquement sur w_2** .

$$f'(x) \text{ ou } \frac{df}{dx}(x)$$

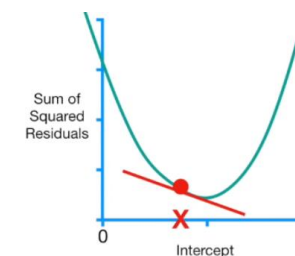
La **dérivée** d'une fonction d'une variable réelle mesure l'ampleur du changement de la valeur de la fonction (valeur de sortie) par rapport à un petit changement de son argument (valeur d'entrée).

Le Résidu est la différence entre la Valeur Observée et la Valeur Prédite.

La Descente de Gradient utilise la dérivée afin de trouver l'endroit où la Somme des Résidus au Carré est minimale.

Plus on se rapproche de la valeur optimale de l'intercept, plus la pente de la courbe se rapproche de zéro.

La descente de gradient identifie la valeur optimale en faisant de grands pas lorsque l'on est loin de la somme optimale des résidus au carré, et en effectuant de nombreux petits pas lorsqu'on est proche de la meilleure solution.



Sarah Malaeb

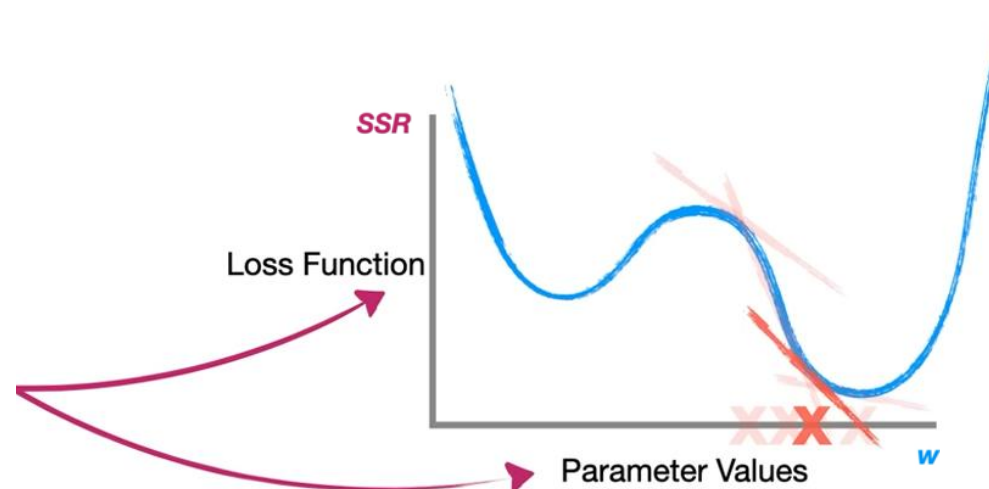
EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

Aussi, pour rappel, lorsque nous optimisons les réseaux de neurones avec la **rétropropagation (Backpropagation)**, nous commençons par calculer les **dérivées**, ou **gradients**, pour chaque paramètre.

$$\frac{d SSR}{d w_1} = \frac{d SSR}{d Predicted} \times \frac{d Predicted}{d y_2} \times \frac{d y_2}{d x_2} \times \frac{d x_2}{d w_1} + \dots$$

Nous injectons ensuite ces **gradients** dans l'algorithme de **descente de gradient (Gradient Descent)** afin de trouver les valeurs des paramètres qui **minimisent la fonction de perte (Loss Function)**, comme par exemple la **somme des résidus au carré**.



EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

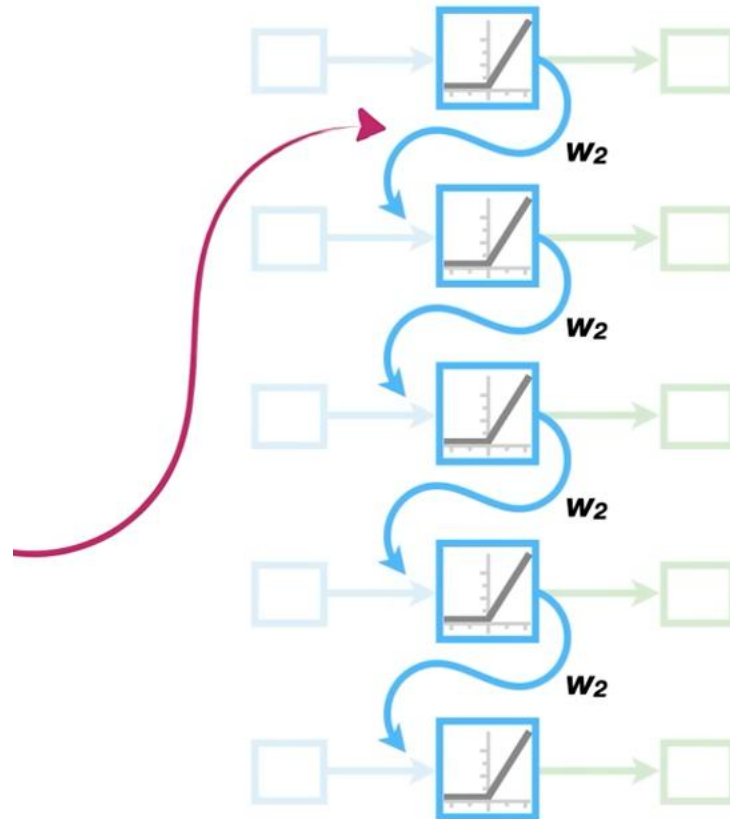
Maintenant, même si le **problème du gradient qui disparaît ou explose (Vanishing/Exploding Gradient Problem)** commence par le terme « **Vanishing** »...

...nous allons commencer par montrer **comment un gradient peut exploser**

EXEMPLE : MARCHÉ BOURSIER

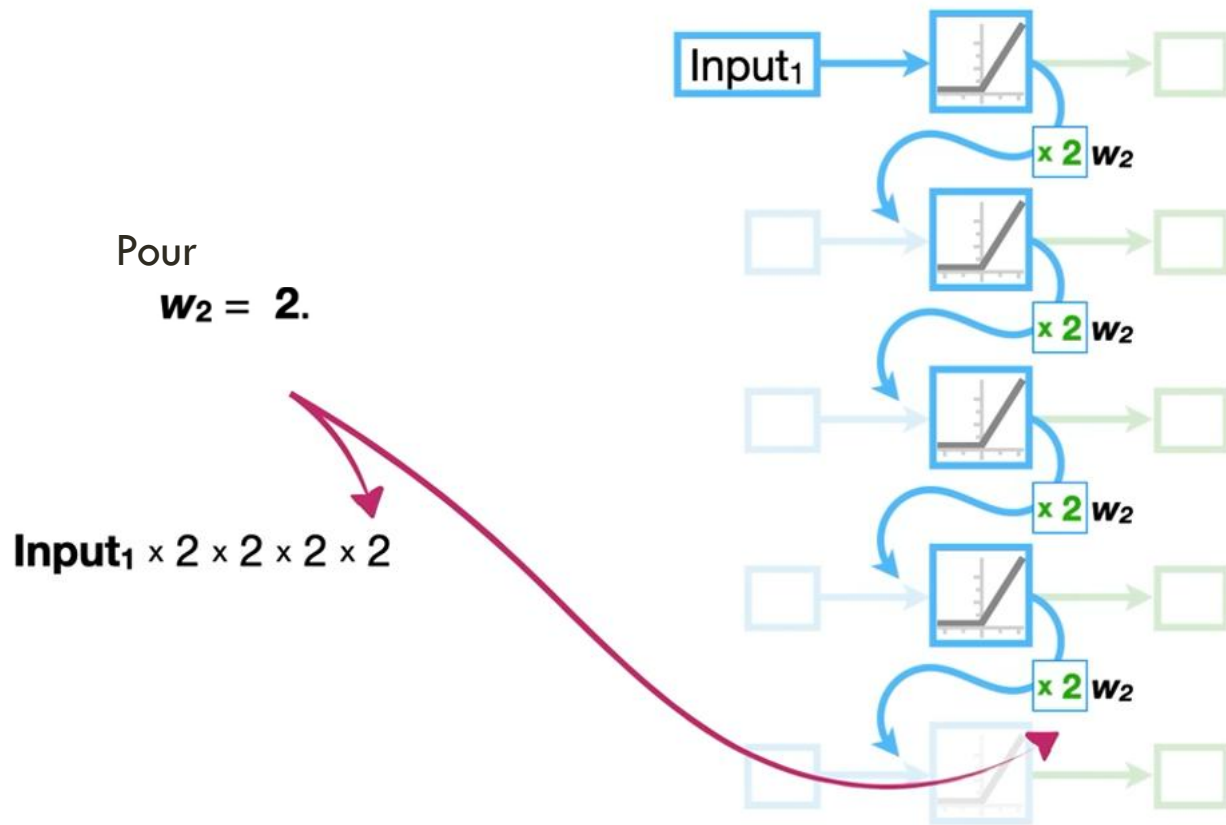
UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

Dans notre exemple, le **gradient explosera** lorsque nous fixerons w_2 à une valeur **supérieure à 1**.



EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS



Autrement dit, puisque nous avons **déplié** (**unrolled**) le réseau de neurones récurrent **4 fois**, nous multiplions la valeur d'entrée par $w_2 = 2$, élevée au nombre de fois où le réseau a été déplié, soit **4** :

$$\begin{aligned} & \text{Input}_1 \times 2 \times 2 \times 2 \times 2 \\ &= \text{Input}_1 \times 2^4 \\ &= \text{Input}_1 \times w_2^{\text{Num. Unroll}} \end{aligned}$$

Cela signifie que la **valeur d'entrée initiale est amplifiée 16 fois** avant d'atteindre la dernière copie du réseau.

$$\begin{aligned} & \text{Input}_1 \times 2 \times 2 \times 2 \times 2 \\ &= \text{Input}_1 \times 2^4 = \text{Input}_1 \times 16 \\ &= \text{Input}_1 \times w_2^{\text{Num. Unroll}} \end{aligned}$$

EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

Maintenant, si nous avons **50 jours consécutifs de données boursières**,
ce qui, honnêtement, n'est pas tant de données que ça...
...alors nous **déplierions** (**unroll**) le réseau **50 fois** :


$$= \text{Input}_1 \times 2^{50}$$

$$= \text{Input}_1 \times w_2^{\text{Num. Unroll}}$$

$$\text{Input}_1 \times 2^{50} = \text{Input}_1 \times \text{A HUGE NUMBER}$$

Et c'est à cause de ce **NOMBRE ÉNORME** qu'on appelle cela le **problème du gradient explosif**.

EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

$$\frac{d SSR}{d w_1} = \frac{d SSR}{d Predicted} \times \frac{d Predicted}{d y_1} \times \frac{d y_1}{d x_1} \times \frac{d x_1}{d w_1} + \dots$$

Si nous essayons d'entraîner ce **réseau de neurones récurrent** avec la **rétropropagation** (backpropagation), ce **NOMBRE ÉNORME** se retrouverait dans certains des gradients...

$$\begin{aligned} \frac{d x_1}{d w_1} &= \frac{d}{d w_1} \text{Input}_1 \times w_1 \times w_2^{50} \dots \\ &= \frac{d}{d w_1} \text{Input}_1 \times w_1 \times \text{A HUGE NUMBER} \dots \end{aligned}$$

$$= \text{Input}_1 \times 2^{50} = \text{Input}_1 \times \text{A HUGE NUMBER}$$

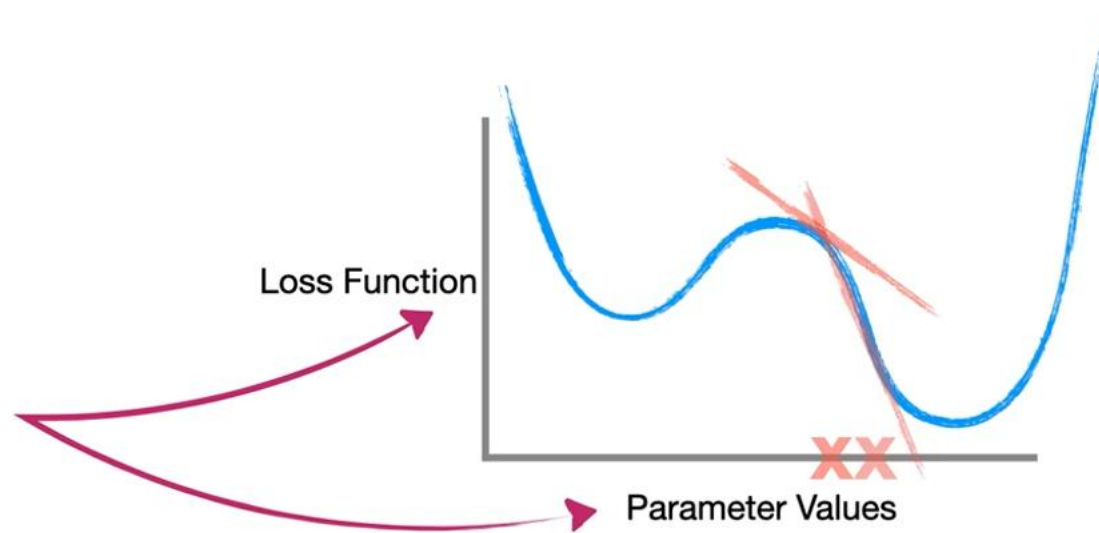
$$= \text{Input}_1 \times w_2^{\text{Num. Unroll}}$$

...et cela rendrait difficile la prise de **petits pas** pour trouver les **poids (weights)** et **biais (biases)** optimaux.

EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

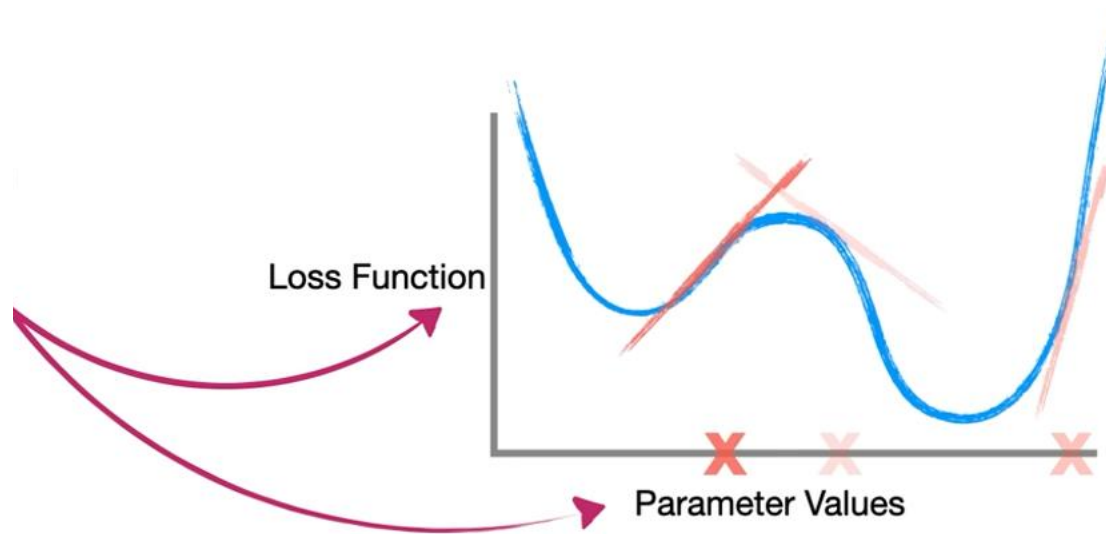
En d'autres termes, pour trouver les valeurs des paramètres qui nous donnent la **valeur la plus basse de la fonction de perte (Loss Function)**, nous cherchons généralement à effectuer des **pas relativement petits**.



EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

Cependant, lorsque le **gradient** contient un **nombre énorme**, nous finissons par faire des **pas relativement grands**. Et au lieu de trouver le **paramètre optimal**, nous allons simplement **osciller** autour de celui-ci sans jamais le stabiliser.



EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

Une façon de prévenir le **problème du gradient explosif** consiste à limiter w_2 à des valeurs **inférieures à 1**.

Cependant, cela entraîne le **problème du gradient qui s'annule (Vanishing Gradient Problem)**.

EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

Pour illustrer le
problème du gradient
qui s'annule
(Vanishing Gradient
Problem),

Pour
 $w_2 = 0.5$.



Maintenant, comme
précédemment, nous
multiplions la première
entrée par w_2 élevé au
nombre de fois où nous
déroutons (*unroll*) le réseau.

$\text{Input}_1 \times w_2^{\text{Num. Unroll}}$

EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

Ainsi, si nous avons **50 valeurs d'entrée séquentielles**, cela revient à multiplier **Input₁** par **0.5^{50}** ,

$$\text{Input}_1 \times 0.5^{50}$$

$$\text{Input}_1 \times w_2^{\text{Num. Unroll}}$$



EXEMPLE : MARCHÉ BOURSIER

UTILISER RNN POUR LA PRÉDICTION DES PRIX DES ACTIONS

...et 0.5^{50} est un nombre
très proche de 0



$\text{Input}_1 \times 0.5^{50} = \text{Input}_1 \times \text{a number super close to 0}$

$\text{Input}_1 \times w_2^{\text{Num. Unroll}}$

Parce que ce nombre est **très proche de 0**, cela s'appelle le **problème du gradient qui s'annule** (**Vanishing Gradient Problem**).

Ainsi, lors de l'optimisation d'un paramètre, au lieu de faire des pas trop grands, on finit par faire des **pas trop petits**.

Et, en conséquence, nous atteignons le **nombre maximal de pas** autorisés avant de trouver la **valeur optimale**.

PROBLÈME DU GRADIENT QUI DISPARAÎT (VANISHING GRADIENT PROBLEM)

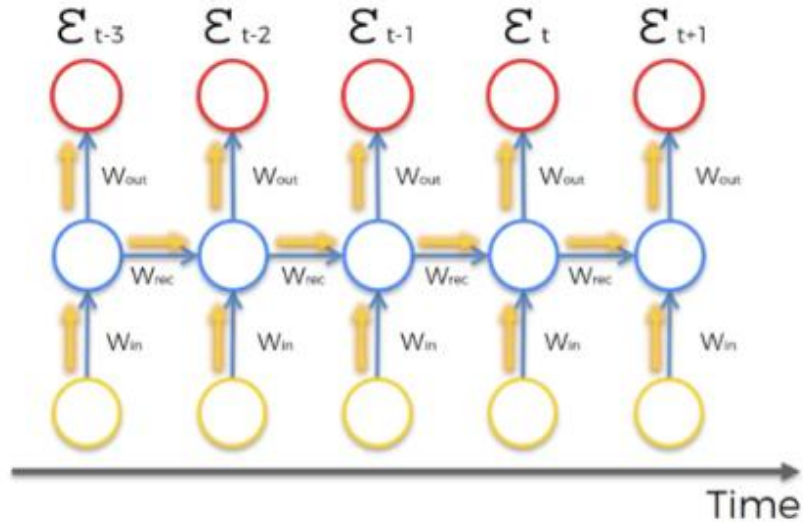
Récap:

- En apprentissage automatique, le problème du gradient qui disparaît survient lors de l'entraînement de réseaux de neurones artificiels avec des méthodes basées sur le gradient.
- Un petit gradient signifie que les poids et biais des premières couches ne seront pas efficacement mis à jour à chaque session d'entraînement.
- Plus le réseau comporte de couches, plus la valeur du produit des dérivées diminue jusqu'à ce qu'à un moment donné la dérivée partielle de la fonction de perte approche de zéro. La dérivée partielle disparaît. On appelle cela le problème du gradient qui disparaît.

Récap:

Nous calculons le terme d'erreur de perte avec la fonction de coût qui compare la sortie prédite (neurone rouge) à la sortie attendue.

Ensuite, nous propageons la fonction de coût à travers le réseau car nous devons mettre à jour les poids afin de réduire l'erreur.



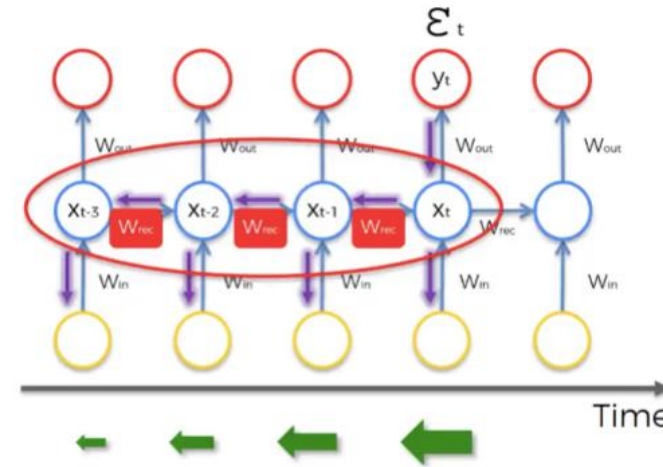
PROBLÈME DU GRADIENT QUI DISPARAÎT (VANISHING GRADIENT PROBLEM)

Récap:

Le problème est lié à la mise à jour de **wrec (weight recurring)** – le poids utilisé pour connecter les couches cachées à elles-mêmes dans la boucle temporelle déroulée.

Par exemple, pour passer de x_{t-3} à x_{t-2} , on multiplie x_{t-3} par w_{rec} . Puis, pour passer de x_{t-2} à x_{t-1} , on multiplie encore x_{t-2} par w_{rec} .

Donc, on multiplie plusieurs fois par le même poids.
Et c'est là que surgit le problème : lorsqu'on multiplie un nombre par une petite valeur, la valeur décroît très vite.



$$\frac{\partial \mathcal{E}}{\partial \theta} = \sum_{1 \leq t \leq T} \frac{\partial \mathcal{E}_t}{\partial \theta} \quad (3)$$

$$\frac{\partial \mathcal{E}_t}{\partial \theta} = \sum_{1 \leq k \leq t} \left(\frac{\partial \mathcal{E}_t}{\partial x_t} \frac{\partial x_t}{\partial x_k} \frac{\partial^+ x_k}{\partial \theta} \right) \quad (4)$$

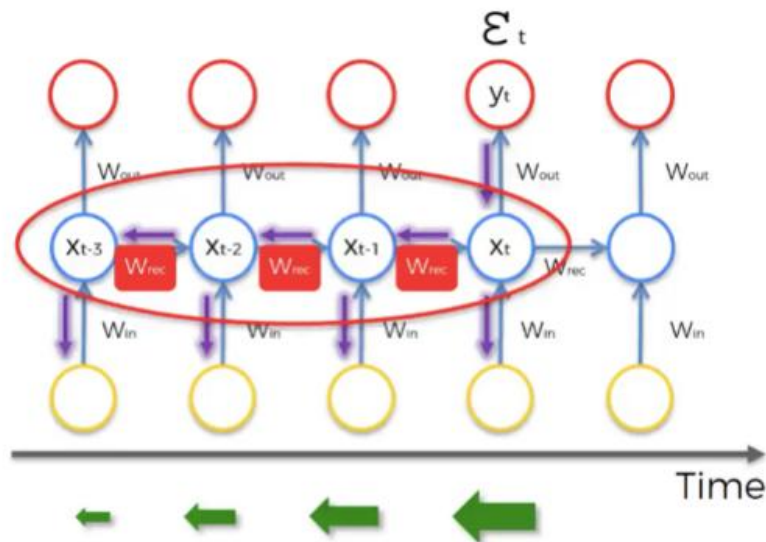
$$\frac{\partial x_t}{\partial x_k} = \prod_{i \geq t > k} \frac{\partial x_i}{\partial x_{i-1}} = \prod_{i \geq t > k} w_{rec}^T \text{diag}(\sigma'(x_{i-1})) \quad (5)$$

$w_{rec} \sim \text{small} \Rightarrow$ Vanishing
 $w_{rec} \sim \text{large} \Rightarrow$ Exploding

Formula Source: Razvan Pascanu et al. (2013)

PROBLÈME DU GRADIENT QUI DISPARAÎT (VANISHING GRADIENT PROBLEM)

Récap:



$$\frac{\partial \mathcal{E}}{\partial \theta} = \sum_{1 \leq t \leq T} \frac{\partial \mathcal{E}_t}{\partial \theta} \quad (3)$$

$$\frac{\partial \mathcal{E}_t}{\partial \theta} = \sum_{1 \leq k \leq t} \left(\frac{\partial \mathcal{E}_t}{\partial \mathbf{x}_t} \frac{\partial \mathbf{x}_t}{\partial \mathbf{x}_k} \frac{\partial^+ \mathbf{x}_k}{\partial \theta} \right) \quad (4)$$

$$\frac{\partial \mathbf{x}_t}{\partial \mathbf{x}_k} = \prod_{t \geq i > k} \frac{\partial \mathbf{x}_i}{\partial \mathbf{x}_{i-1}} = \prod_{t \geq i > k} \mathbf{w}_{rec}^T \text{diag}(\sigma'(\mathbf{x}_{i-1})) \quad (5)$$

$W_{rec} \sim \text{small} \Rightarrow \text{Vanishing}$
 $W_{rec} \sim \text{large} \Rightarrow \text{Exploding}$

Formula Source: Razvan Pascanu et al. (2013)

Plus le gradient est faible, plus il est difficile pour le réseau de mettre à jour les poids et plus le temps nécessaire pour atteindre le résultat final est long.

SOLUTIONS AU PROBLÈME DU GRADIENT QUI DISPARAÎT

Nous parlerons d'une solution populaire
appelée

**Réseaux de mémoire à long et court terme
(Long Short-Term Memory Networks ou LSTM)**

SOLUTIONS AU PROBLÈME DU GRADIENT QUI DISPARAÎT

- En cas d'explosion du gradient (exploding gradient), vous pouvez :
 - arrêter la rétropropagation après un certain point (souvent non optimal, car tous les poids ne sont pas mis à jour),
 - pénaliser ou réduire artificiellement le gradient,
 - imposer une limite maximale au gradient.
- En cas de gradient qui disparaît (vanishing gradient), vous pouvez :
 - initialiser les poids de façon à minimiser ce risque,
 - utiliser des Echo State Networks (un type spécial de RNN conçu pour résoudre ce problème),
 - utiliser des réseaux **LSTM (Long Short-Term Memory)** → **un type particulier de RNN.**

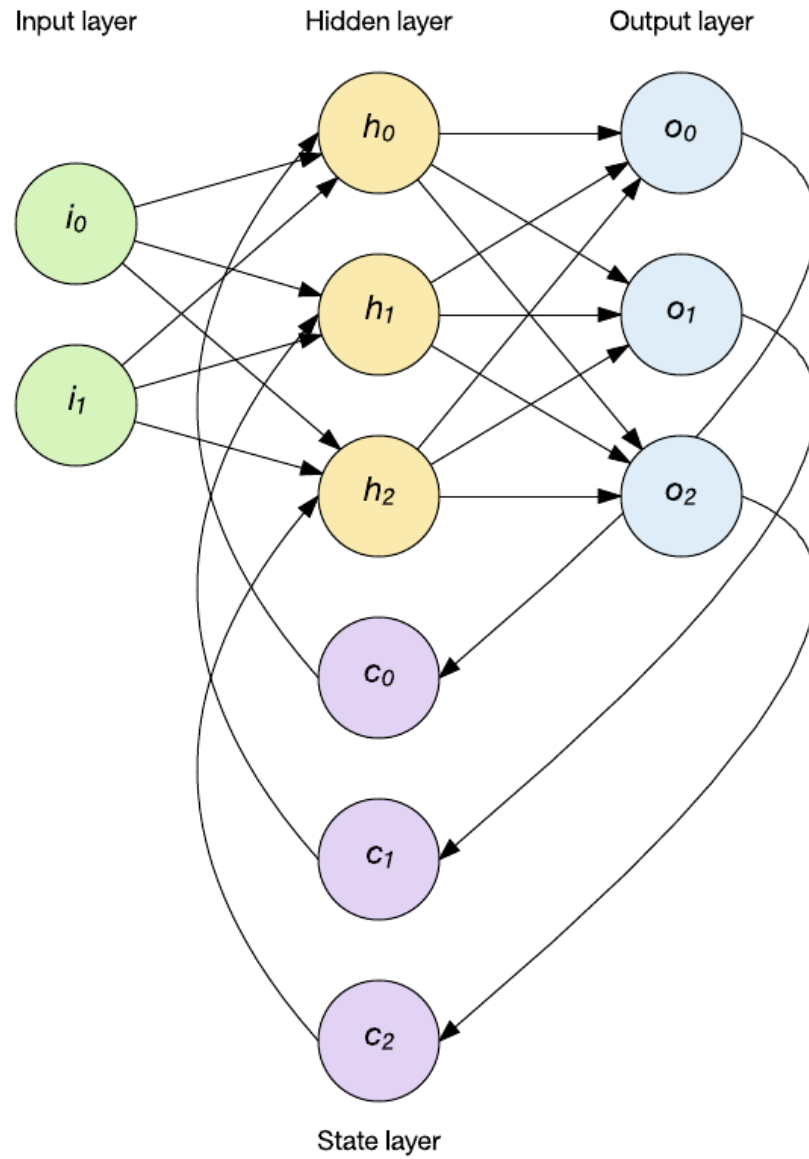
DIFFERENTES ARCHITECTURES RNN

- Il existe de nombreuses architectures de réseaux de neurones récurrents :
 - les réseaux d'Elman et de Jordan,
 - les LSTM (Long Short-Term Memory),
 - les GRU (Gated Recurrent Unit),
 - etc.
- **Toutes ces architectures, malgré leurs différences, possèdent une caractéristique commune :**
 - **elles disposent d'une mémoire utilisée pendant l'apprentissage.**
- Les différences se situent au niveau des règles d'apprentissage et de la manière dont l'information circule dans le réseau (mémoire à court, moyen ou long terme, cycles internes dans le réseau, etc.).
- Dans ce cours, nous étudierons principalement les réseaux d'Elman et de Jordan, ainsi que les LSTM.

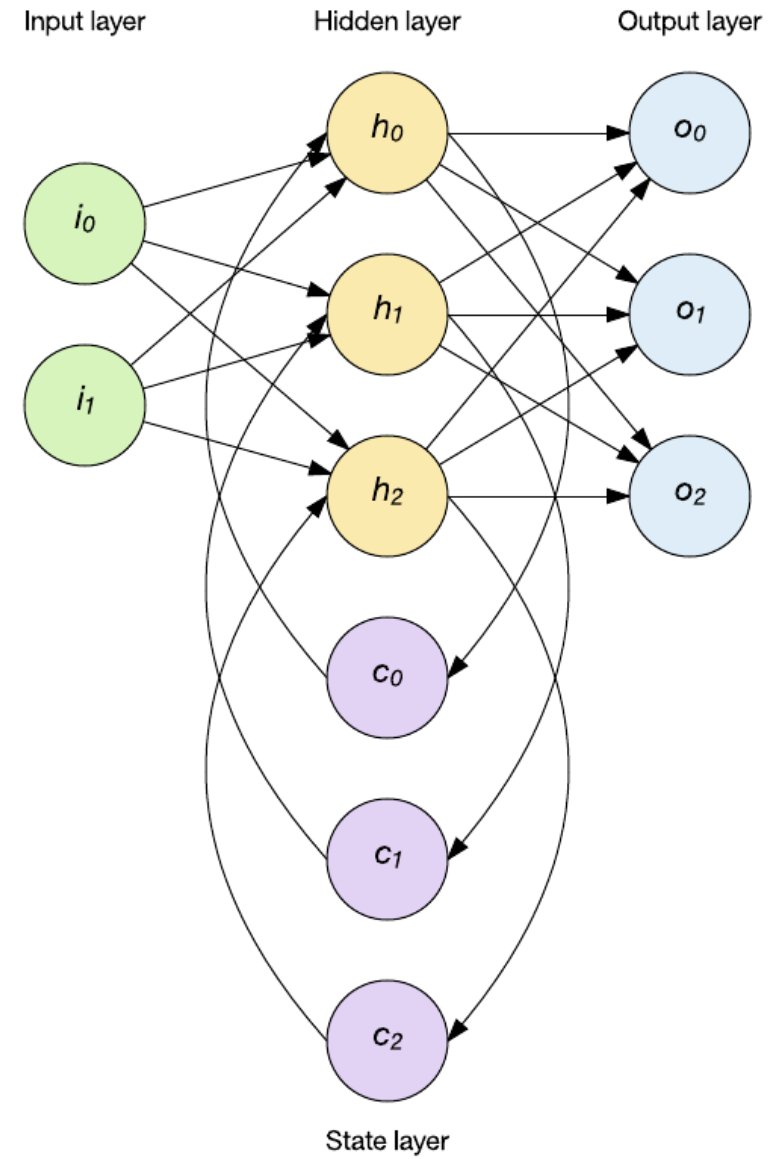
RÉSEAUX RÉCURRENTS SIMPLES

- Les réseaux récurrents simples constituent une classe populaire de réseaux récurrents qui **inclut une couche d'état permettant d'introduire une notion d'état dans le réseau.**
- La couche d'état influence la prochaine étape d'entrée et peut donc être **appliquée à des données présentant des variations temporelles.**
- Il est possible d'introduire cette étatabilité (statefulness) de différentes manières, mais **les deux approches les plus courantes sont les réseaux d'Elman et les réseaux de Jordan.**

Jordan network

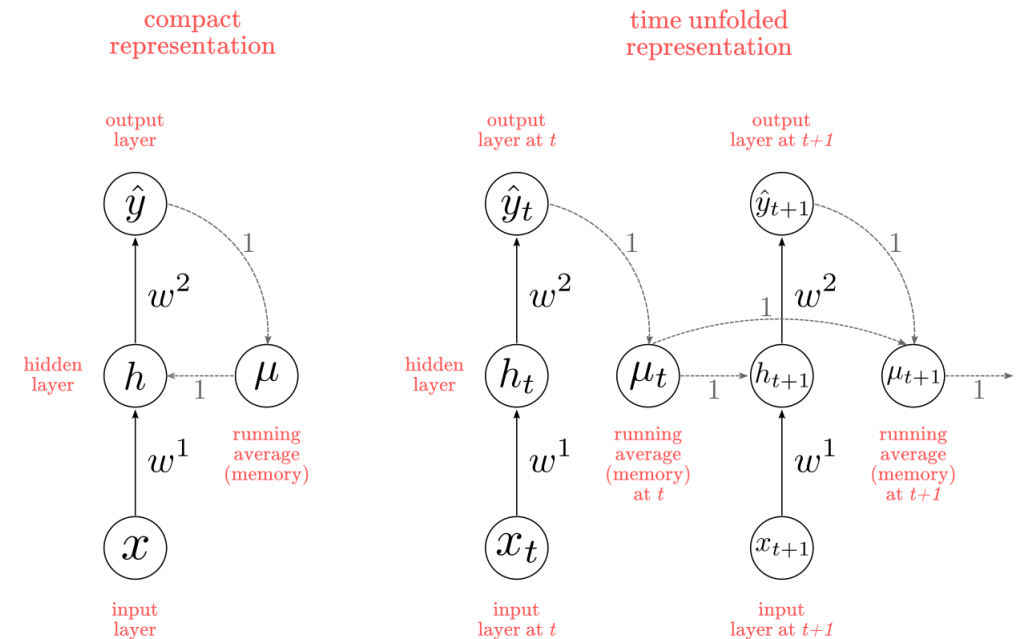


Elman network



JORDAN NEURAL NETWORK

- Le réseau de neurones de Jordan (JNN) a été proposé en 1986 par Michael I. Jordan.
- Le réseau récurrent de Jordan (Jordan RNN) est un autre type de réseau de neurones récurrent qui utilise la sortie précédente comme entrée pour le calcul courant.
- Le réseau de Jordan met en œuvre des connexions récurrentes depuis la sortie du réseau ($\hat{y}$) vers ses unités cachées (h), via une unité de mémoire (μ) équivalente à l'unité de contexte du réseau d'Elman comme illustré dans la Figure.
- En résumé, l'unité de mémoire conserve une moyenne cumulative de toutes les sorties passées : c'est ainsi que l'historique précédent est implicitement pris en compte dans chaque nouveau calcul.
- Il n'y a aucun apprentissage dans l'unité de mémoire, ce qui signifie que les poids y sont fixés à 1.

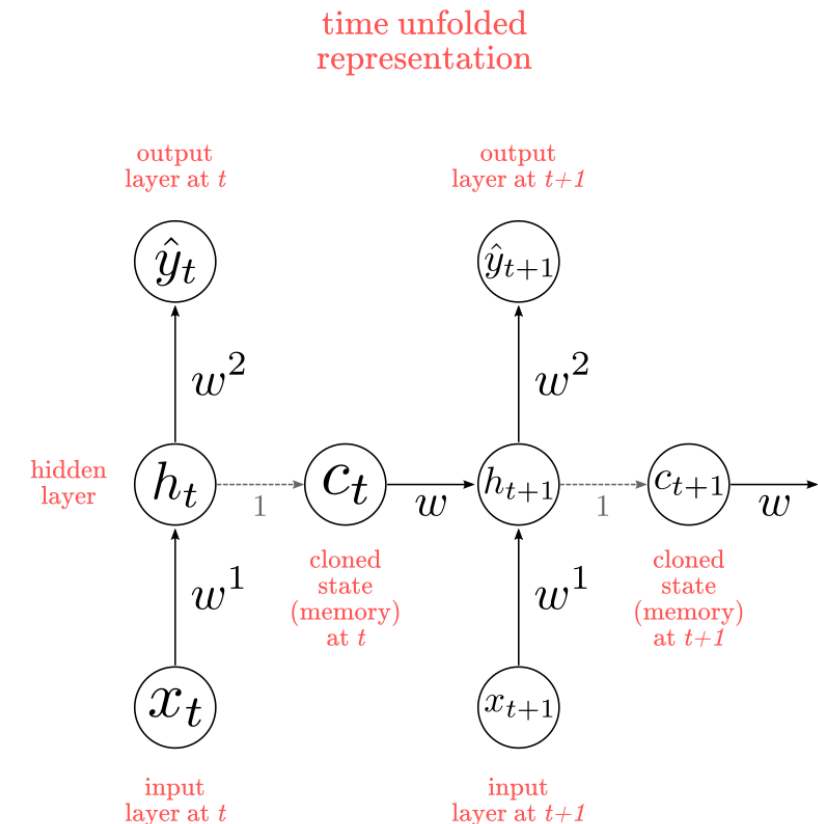
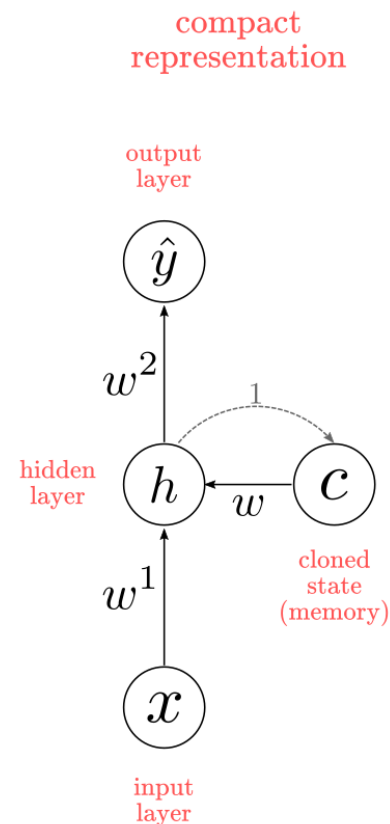


ELMAN NEURAL NETWORK

- Le premier exemple réussi d'un réseau récurrent entraîné par rétropropagation a été introduit par Jeffrey Elman : il s'agit du réseau d'Elman (Elman Network, 1990).
- Elman a fondé son approche sur les travaux de Michael I. Jordan concernant le traitement sériel (serial processing, 1986).
- **Le réseau récurrent d'Elman (Elman RNN) est un type de réseau de neurones récurrent qui inclut une couche supplémentaire d'unités cachées recevant comme entrée l'état caché précédent. Cette couche agit comme une mémoire à court terme, permettant au réseau de stocker des informations sur les entrées précédentes et d'utiliser ces informations dans le calcul courant.**

ELMAN NEURAL NETWORK

- Dans le réseau d'Elman, il existe des connexions récurrentes entre les unités cachées et les unités de mémoire (ou unités de contexte), ainsi que des paramètres entraînables reliant les unités de mémoire aux unités cachées.
- Les unités de mémoire doivent désormais mémoriser l'état passé des unités cachées, ce qui signifie qu'au lieu de conserver une moyenne glissante, elles "clonent" la valeur de l'état précédent, c'est-à-dire celle du pas de temps $t-1$.
- Les unités de mémoire doivent également apprendre des représentations utiles (poids) afin d'encoder les propriétés temporelles des données séquentielles en entrée.





VANISHING GRADIENT PROBLEM

- Le problème du gradient qui disparaît rend quasiment impossible l'apprentissage des dépendances à long terme dans les séquences.
- Imaginons que vous ayez une collection de poèmes, où la dernière phrase fait référence à la première.
- Une telle dépendance serait très difficile à apprendre pour un RNN profond, car les gradients s'atténuent progressivement à mesure que l'on remonte dans le réseau.
- Le problème du gradient qui explose, quant à lui, peut complètement perturber le processus d'apprentissage.

LONG SHORT-TERM MEMORY NETWORKS LSTM

Pour résoudre le problème du gradient qui disparaît, il faut d'abord comprendre son origine :

En réalité, on peut dire que :

$W_{\text{rec}} < 1$  Vanishing

$W_{\text{rec}} > 1$  Exploding

La solution la plus simple et la plus rapide consiste à **fixer $W_{\text{rec}} = 1$** .

Faire en sorte que le poids récurrent soit égal à 1 est **la principale idée à la base des réseaux LSTM (Long Short-Term Memory)**.

LONG SHORT-TERM MEMORY NETWORKS LSTM

La **couche cachée** dans le **bloc central** reçoit l'entrée x_t provenant de la couche d'entrée, ainsi qu'une entrée provenant d'elle-même au **temps** $t - 1$, puis elle génère la sortie h_t .

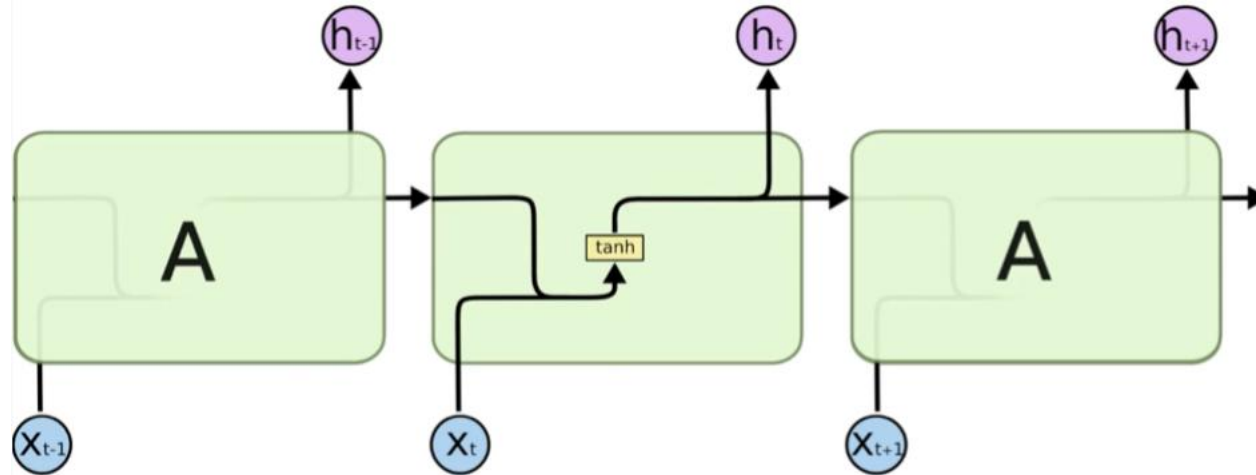


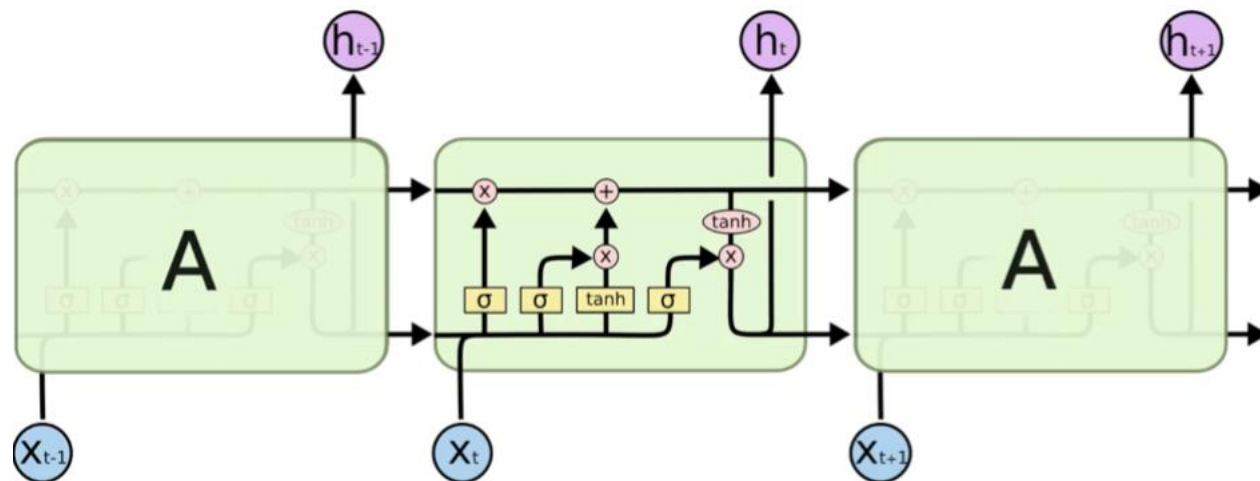
Image Source: colah.github.io

LONG SHORT-TERM MEMORY NETWORKS LSTM

Chaque bloc possède trois entrées (x_t , h_{t-1} , and c_{t-1}) et deux sorties (h_t and c_t).

Il est important de se rappeler que toutes ces entrées et sorties ne sont pas de simples valeurs, mais des vecteurs contenant de nombreuses valeurs.

- c_{t-1} stands for the input from a memory cell in time point t ;
- x_t is an input in time point t ;
- h_t is an output in time point t that goes to both the output layer and the hidden layer in the next time point.

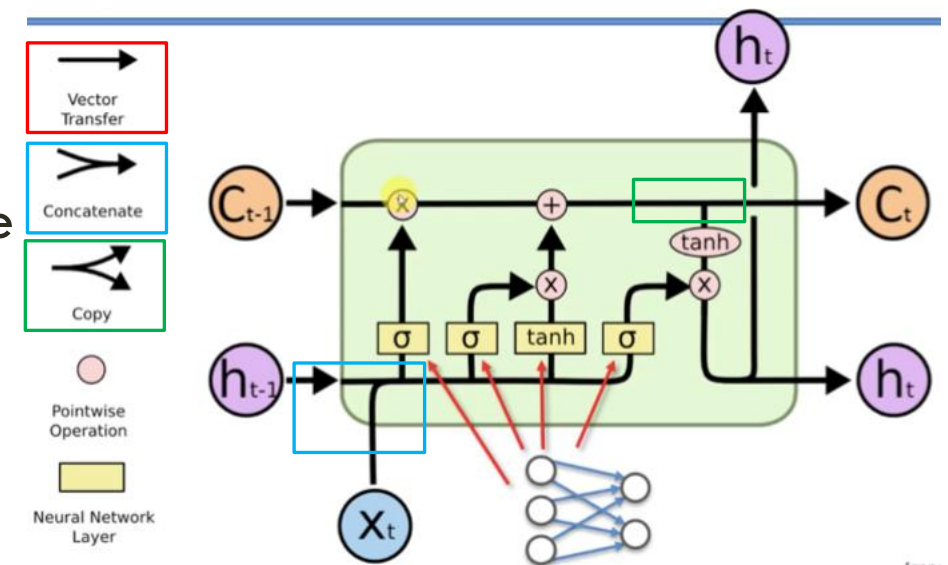


LONG SHORT-TERM MEMORY NETWORKS LSTM

Transfert de vecteurs : chaque ligne du schéma représente un vecteur.

Concaténation : deux lignes se combinent en une seule, comme par exemple les vecteurs h_{t-1} et x_t .

Copie : l'information est dupliquée et envoyée dans deux directions différentes, comme dans la partie inférieure droite du schéma, où les informations de sortie sont copiées afin d'arriver à deux couches différentes h_t .



Imax

LONG SHORT-TERM MEMORY NETWORKS LSTM

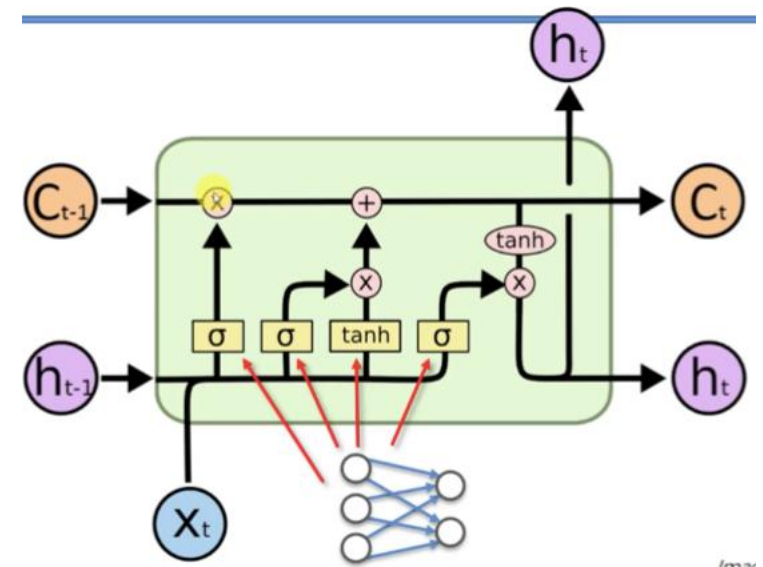
Opérations point par point (Pointwise operations) : elles sont de trois types.

Le symbole “X” ou valves (portes) représente les valves d’oubli, de mémoire, et de sortie — ce sont des points sur le schéma où l’on peut ouvrir le flux d’informations, le fermer, ou l’ouvrir partiellement.

Par exemple, la valve d’oubli, située en haut à droite du schéma ci-dessus, est contrôlée par l’opération de couche σ (la fonction sigmoïde).

- En fonction de la décision prise par cette fonction d’activation sigmoïde (dont la sortie varie entre 0 et 1), la valve sera : fermée, ouverte, ou partiellement ouverte.
- Si la valve est ouverte, la mémoire circule librement de $t-1$ à t .
- Si elle est fermée, la mémoire est interrompue, et probablement une nouvelle information sera ajoutée plus loin dans le pipeline, à un autre point d’opération point par point.

σ function controls the forget valve:
If σ outputs 1 $\rightarrow$ gate is fully open, information passes freely.
If σ outputs 0 $\rightarrow$ gate is fully closed, no information passes.
If σ outputs between 0 and 1 $\rightarrow$ partial flow of information.



Imax

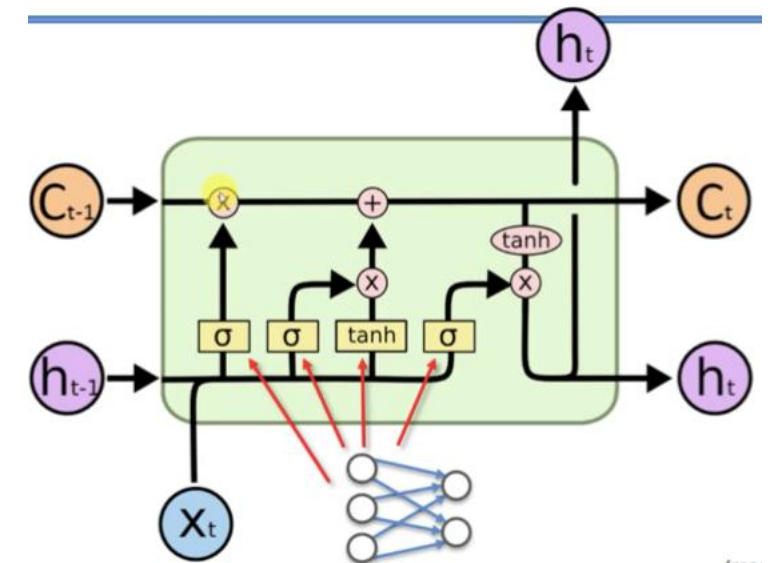
LONG SHORT-TERM MEMORY NETWORKS LSTM

Opérations point par point (Pointwise operations) :
elles sont de trois types.

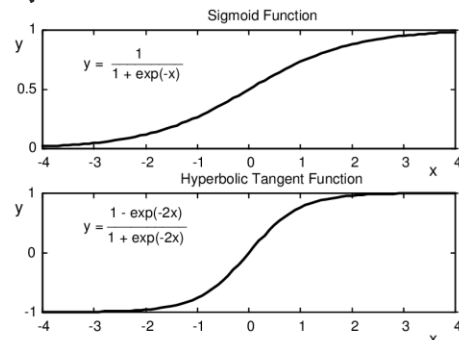
Le symbole “+” représente une **jonction en forme de T**, où la **mémoire circule** et où l’on peut **ajouter de la mémoire supplémentaire** si la **valve de mémoire**, située en dessous de cette jonction, est **ouverte**.

La fonction “**tanh**” est **responsable de la transformation des valeurs** afin qu’elles soient **comprises entre -1 et 1**, ce qui est **nécessaire pour des raisons mathématiques** (stabilisation et normalisation des activations).

Pointwise
Operation

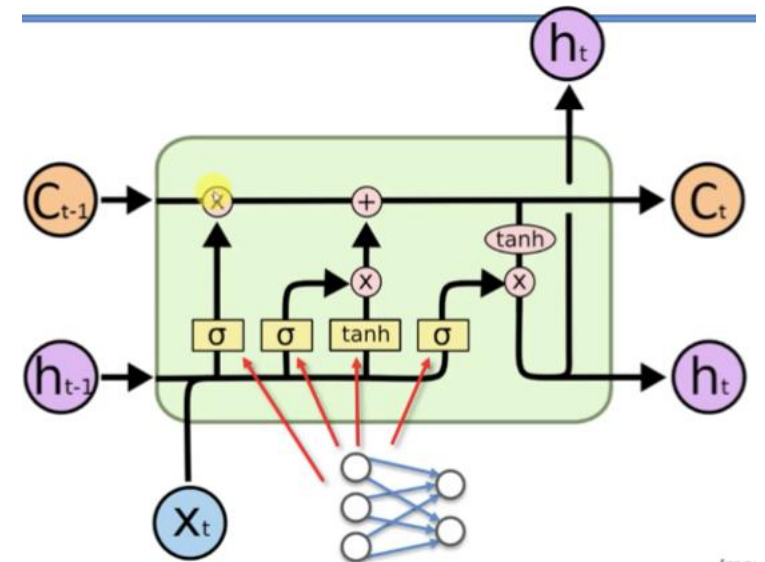
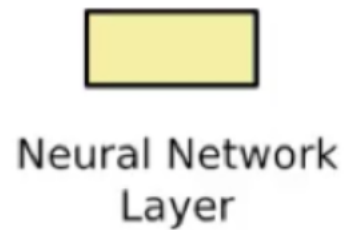


Imax



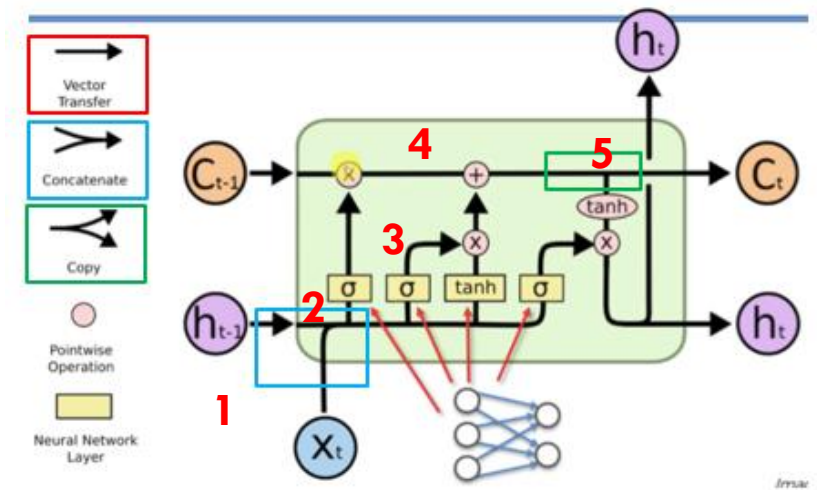
LONG SHORT-TERM MEMORY NETWORKS LSTM

Couche de réseau de neurones (Neural Network Layer) :
ce sont des opérations de couche, où une couche entre
en entrée et une autre couche en sort en sortie.



Imax

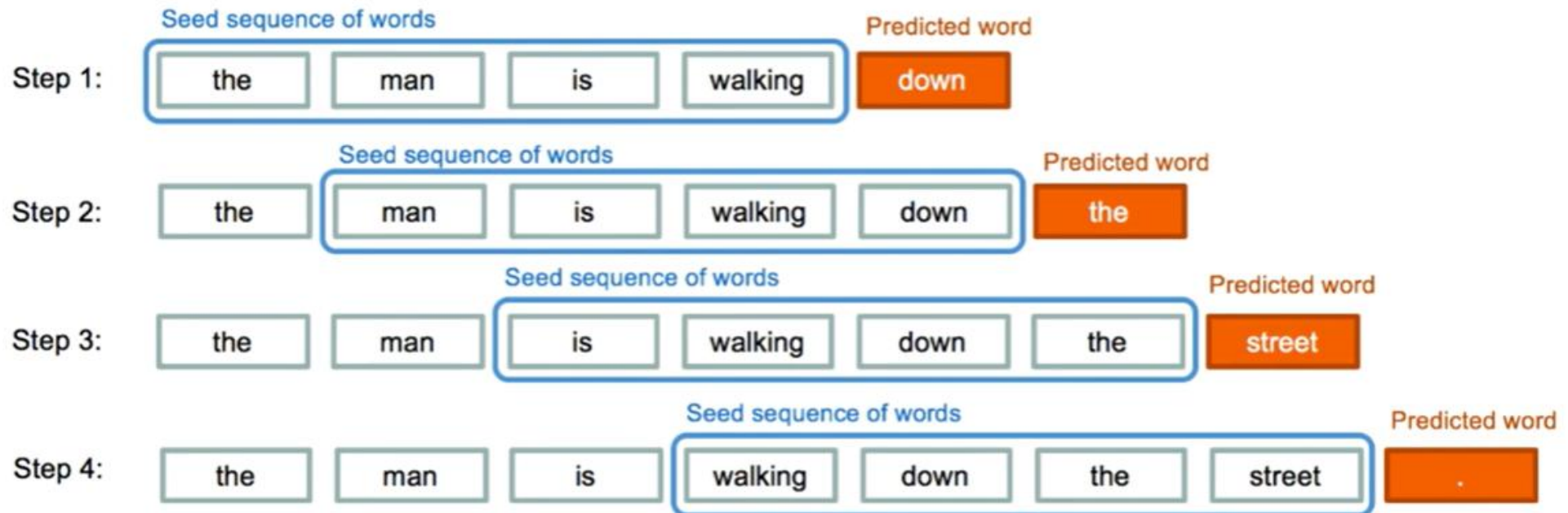
ARCHITECTURE DU LSTM



1. Nous avons une nouvelle valeur x_t et la valeur provenant du nœud précédent h_{t-1} qui arrivent en entrée.
2. Ces valeurs sont combinées et passent à travers une fonction d'activation sigmoïde, où il est décidé si la valve d'oubli doit être ouverte, fermée, ou partiellement ouverte.
3. Les mêmes valeurs — ou plus précisément des vecteurs de valeurs — passent en parallèle à travers une autre opération de couche “ $\tanh$ ”, où il est déterminé quelle valeur sera transmise au pipeline de mémoire, ainsi qu'à travers une couche sigmoïde, où il est décidé si cette valeur sera effectivement transmise à la mémoire et dans quelle mesure.
4. Ensuite, la mémoire circule à travers le pipeline supérieur. Si la valve d'oubli est ouverte et la valve de mémoire fermée, alors la mémoire ne changera pas. En revanche, si la valve d'oubli est fermée et la valve de mémoire ouverte, la mémoire sera complètement mise à jour.
5. Enfin, les valeurs x_t et h_{t-1} sont combinées pour déterminer quelle partie du pipeline de mémoire deviendra la sortie de ce module.

EXEMPLE D'APPLICATION D'UN RNN : GÉNÉRATION AUTOMATIQUE DE TEXTE

Check the tutorial video (Automatic Text Generation using TensorFlow, Keras and LSTM): <https://www.youtube.com/watch?v=VAMKuRAh2nc>



EXEMPLE D'IMPLÉMENTATION D'UN RNN POUR LA GÉNÉRATION AUTOMATIQUE DE TEXTE

Les **Réseaux de Neurones Récurrents (RNN)** peuvent être utilisés pour **générer automatiquement du texte** en apprenant les **modèles et structures linguistiques** présents dans un corpus d'entraînement.

Le modèle apprend à **prédire le mot (ou caractère) suivant** dans une séquence à partir des mots précédents.

Une fois entraîné, il peut **générer du texte nouveau**, cohérent avec le style du corpus.

Étapes principales :

1- Collecte du corpus textuel

Exemple : œuvres littéraires, articles de presse, paroles de chansons, etc.

2- Prétraitement des données

- Nettoyage du texte (ponctuation, casse, caractères spéciaux)
- Tokenisation (division en mots ou caractères)
- Encodage numérique (chaque mot/char devient un entier)
- Padding (uniformisation des séquences)

EXEMPLE D'IMPLÉMENTATION D'UN RNN POUR LA GÉNÉRATION AUTOMATIQUE DE TEXTE

Suite Étapes principales :

3- Conception du modèle RNN

- Utilisation d'un modèle séquentiel (Sequential de Keras)
- Couches possibles :
 - Embedding (conversion en vecteurs denses)
 - SimpleRNN, LSTM ou GRU
 - Dense avec activation softmax pour la prédiction du mot suivant

4- Entraînement du modèle

- Sur les séquences d'entrée (mots précédents) et la cible (mot suivant)
- Fonction de perte : `categorical_crossentropy`
- Optimiseur : `adam`

5- Génération de texte

- Fournir une phrase de départ (seed text)
- Le modèle prédit le mot suivant
- Le mot prédit est ajouté à la séquence, et le processus est répété

EXEMPLE D'IMPLÉMENTATION D'UN RNN POUR LA GÉNÉRATION AUTOMATIQUE DE TEXTE

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.preprocessing.text import Tokenizer
import numpy as np
```

Exemple de corpus textuel

```
data = """Deep learning allows machines to learn from experience.
Recurrent neural networks are powerful for sequential data.
Text generation is an exciting application of LSTM networks."""
```

1 - Tokenisation

```
tokenizer = Tokenizer()
tokenizer.fit_on_texts([data])
total_words = len(tokenizer.word_index) + 1
```

2 - Création des séquences d'entraînement

```
input_sequences = []
for line in data.split('.'):
    tokens = tokenizer.texts_to_sequences([line])[0]
    for i in range(1, len(tokens)):
        seq = tokens[:i+1]
        input_sequences.append(seq)
```

3 - Padding des séquences

```
max_seq_len = max([len(x) for x in input_sequences])
input_sequences = np.array(pad_sequences(input_sequences, maxlen=max_seq_len, padding='pre'))
```

4 - Séparation X / y

```
X = input_sequences[:, :-1]
y = input_sequences[:, -1]
y = np.eye(total_words)[y] # one-hot encoding
```

5 - Construction du modèle RNN (LSTM)

```
model = Sequential([
    Embedding(total_words, 100, input_length=max_seq_len-1),
    LSTM(150),
    Dense(total_words, activation='softmax')
])
```

6 - Compilation

```
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

7 - Entraînement du modèle

```
model.fit(X, y, epochs=200, verbose=1)
```


EXEMPLE D'IMPLÉMENTATION D'UN RNN POUR LA GÉNÉRATION AUTOMATIQUE DE TEXTE

8 - Génération automatique de texte

```
def generate_text(seed_text, next_words=10):  
    for _ in range(next_words):  
        token_list = tokenizer.texts_to_sequences([seed_text])[0]  
        token_list = pad_sequences([token_list], maxlen=max_seq_len-1, padding='pre')  
        predicted = np.argmax(model.predict(token_list, verbose=0), axis=-1)[0]  
        output_word = ''  
        for word, index in tokenizer.word_index.items():  
            if index == predicted:  
                output_word = word  
                break  
        seed_text += ' ' + output_word  
    return seed_text
```

Exemple d'utilisation

```
print(generate_text("Recurrent neural", next_words=10))
```

TP- RNN

Cas d'usage 1 : **Prédire les prix des actions de Google** (*solution partagée*)

A Approfondir :

Cas d'usage 2 : **Détection de fausses nouvelles (Fake News Detection)** à l'aide d'un RNN

Cas d'usage 3 : **Application d'un RNN à un cas NLP personnalisé**

Objective: to reinforce the understanding of sequence models and text preprocessing.

1. Sélectionner un jeu de données textuelles (par exemple : analyse de sentiments, critiques de films, classification thématique).
2. Prétraiter les données : tokenisation, padding, embedding.
3. Concevoir et implémenter un modèle RNN pour résoudre la tâche choisie.
4. Évaluer les performances et discuter les résultats.
5. Expérimenter avec des variantes de RNN ou un ajustement des hyperparamètres : essayer différentes combinaisons d'hyperparamètres pour améliorer les performances (précision, perte, F1-score, etc.).

Hyperparameter	What it does
Number of units (hidden size)	How many neurons in each LSTM layer
Number of layers	How deep the network is (single vs stacked LSTM)
Learning rate	How fast the model updates weights during training
Batch size	Number of samples per training step
Dropout rate	Fraction of neurons to randomly ignore during training
Sequence length / input window	How many time steps the LSTM sees at once